



Yazılım Mühendisliğine Giriş

- Yazılım mühendisliğine giriş,
- Yazılım mühendisliği ve etik,
- Yazılım mühendisliğinin önemi ve gereği,
- Yazılım geliştirme süreci
- Yazılım sürecinde araştırma, ölçme, planlama ve gereksinim analiz yöntemleri,
- Yazılım mühendisliğinde metodolojileri,
- Yazılım yaşam döngüsü sürecinde metotlar,
- Yazılımda standartlar, kalite teknikleri ve kalite prensipleri,
- Yazılımda proje yönetimi,
- Yazılım projelerinde başarı ve başarısızlık nedenleri,
- Bilgisayar destekli yazılım araçları ve örnek uygulamalar.

- Mesleki etik kuralların öğrenilmesi
- Gelişen teknolojiler ışığında, yazılım mühendisinde bulunması gereken niteliklerin belirlenmesi.
- Yazılım mühendisliği sürecini anlama, uygulama ve proje sürecini yönetebilme becerisi kazandırma.
- Geçerli yazılım mühendisliği standartlarını araştırma, öğrenme ve uygulayabilme.
- Güncel yazılımlar hakkında bilgiler vermek, uygulamaya yönelik çözümler üzerinde durmak.
- Hızla gelişen yazılım ihtiyaçlarına, kabul gören standartlarda güvenli kod geliştirebilme kabiliyeti kazandırma.

Amaç :

Yazılım mühendisliği sürecindeki

- ✓ **isterler,**
- ✓ **analiz,**
- ✓ **tasarım,**
- ✓ **gerçekleme,**
- ✓ **test,**
- ✓ **bakım,**

gibi kavramları öğrenmek, kodlama ilkelerini gözden geçirmek, günümüzün en popüler yazılım projelerinin örneklerini paylaşmak, modüler yapıların incelenmesi, kişisel gelişim ve teknolojik değişime ayak uydurmak.

Ders İçeriği ;

1. (1 .Hft) Yazılım nedir, Yazılım mühendisliği ve eğitimi,
2. (2 .Hft) Yazılım İsteklerin belirlenmesi ve geliştirme süreci,
3. (3 .Hft) Yazılım İsterleri çözümlemesi,
4. (4 .Hft) Veri Akış diyagramı geliştirme, örnek uygulama,
5. (5 .Hft) Yazılım tasarımı,
6. (6 .Hft) Arayüz tasarımı
7. (7 .Hft) Yazılım gerçekleştirimi
8. (8 .Hft) Örnek Uygulamalar
9. (9 .Hft) Yazılım testi ve bakımı
10. (10.Hft) Yazılım kalitesi ve standartları
11. (11.Hft) Proje yönetimi
12. (12.Hft) Örnek Uygulamalar
13. (13.Hft) Başarılı Projelerin Ortak Yönleri
14. (14.Hft) Bilişim Sistemlerinde Değerlendirme ve Etik.

Bu ana başlıklarla verilen ders içeriğinde kod üretme teknikleri, Yazılım Mimarileri, CobiT ve popüler yazılım proje örnekleri gibi örnek çalışmalar gösterilecektir.

- Teorik anlatım
- Uygulama sunuları
- Ödev sunumları
- Rapor

DEĞERLENDİRME SİSTEMİ		
YARIYIL İÇİ ÇALIŞMALARI	SAYISI	KATKI PAYI
Ara Sınav	1	70
Kısa Sınav	2	20
Ödev	1	10
TOPLAM		100
Yılıçının Başarıya Oranı		50
Finalin Başarıya Oranı		50
TOPLAM		100

- Yazılım geliştirme sürecine ait metodolojiyi öğrenmesi ve uygulaması,
- Yazılım projesi geliştirebilmeli ve yönetebilmeli,
- Literatür araştırması ve takibi yapabilmeli,
- Ödev ve rapor hazırlayabilmeli,
- Sunum becerisini geliştirmeli,
- ...

Gerçek yaşamda gereksinim duyulan basit yada karmaşık yazılımların tasarımını, üretimini ve bakımını, zaman ve maliyet kısıtlarını da göz önünde bulundurarak etik ve mühendislik yaklaşımıyla öğrencilere tanıtmak ve aynı zamanda çeşitli bireysel araştırmalar ve grup çalışmalarıyla bu süreçlere yönelik uygulamalar yapmalarına olanak sağlamak.

- Mühendislik mi ?
- Bilim mi ?
- Endüstri mi ?
- Ürün mü?
- Sanat mı ?



- ✓ Değişik ve çeşitli görevler yapma amaçlı tasarlanmış elektronik araçların, birbirleriyle haberleşebilmesini ve uyumunu sağlayarak, görevlerini ya da kullanılabilirliklerini geliştirmeye yarayan makina komutlarıdır.
- ✓ Elektronik cihazların belirli bir işi yapmasını sağlayan programların tümüne verilen isimdir.
- ✓ Var olan bir problemi çözmek amacıyla bilgisayar dili kullanılarak oluşturulmuş anlamlı ifadeler bütünüdür.

- ✓ Bir bilgisayarda donanıma hayat veren ve bilgi işlemde kullanılan programlar, yordamlar, programlama dilleri ve belgelemelerin tümü.

- Bilgisayar programlarının tasarımı, geliştirilmesi, test edilmesi ve bakımı konularını ele alan mühendislik dalıdır.
- Temel mühendislik prensiplerinin bu dalda da uygulanması, önceden tahmin edilebilir ve tekrarlanabilir sonuçların daha çok elde edilmesiyle yazılım mühendisliği gerçek bir mühendislik dalı olma yolunda ilerlemektedir.
- Diğer mühendislik dallarıyla karşılaştırıldığında çok yeni olan bu alanda sürekli yeni yöntemler geliştirilmekte ve konu yavaş yavaş belli bir olgunluğa ulaşmaktadır.
- Gelişmekte olan bu alanda halen ihtiyaç çok büyük olduğundan geleceğin mesleklerinden birisi olarak gösterilmektedir.

- Bilgisayar sistemleri artık günlük hayatın her alanında yoğun ve etkin bir şekilde kullanılmakta olduğundan, bankacılıktan otomotiv sanayisine, sağlık bilgi sistemlerinden şirket yönetimine, telekomünikasyon sistemlerinden hava taşımacılığına, çok geniş alanlarda kullanılan bilgisayar sistemlerinin çok önemli ve kritik bir parçasını oluşturuyor.
- Bilgisayar Yazılım Mühendisliği 1968 yılında NATO tarafından gerçekleştirilen bir konferans esnasında ortaya çıkan yeni bir kavram ve yeni bir mühendislik alanı olup, yazılım sistemlerinin mühendislik prensipleri çerçevesinde tasarımı, üretimi ve işletilmesini hedeflemektedir.

“Yazılım Mühendisliği tüm disiplinlerde uygulamaları olan bir alandır. ”

- Yazılım mühendisliği disiplini 1968 yıllarından bu yana oldukça gelişme kaydetmiş; yazılım geliştirme metodolojileri, programlama paradigmaları, programlama dilleri ve çeşitli araçların geliştirilmesiyle hayli ilerleme katetmiştir.
- Gelişime , IEEE (IEEE Computer Society) ve ACM (Association for Computing Machinery) gibi mesleki kuruluşların önemli etkisi olmuştur.

- Ayrıca bu kuruluşlar son yıllarda, yazılım mühendisliği çekirdek bilgisinin tanımlanması ve bu bilgilerle uyumlu yazılım mühendisliği eğitim programlarının geliştirilmesine yönelik çalışmalar da yapmaktadır.
- Bu bağlamda, diğer mühendislik dallarında olduğu gibi yazılım mühendisliği için de ayrı eğitim programlarının oluşturulması gündeme gelmiştir. Yazılım mühendisliği disiplinin olgunlaşma sürecinde yazılım mühendisliği eğitimi özel bir önem kazanmıştır.

Özel bir önem kazanan bu eğitim programı için özel projelere başlanmıştır. Genelde kısaltılmış adlarıyla karşımıza çıkabilecek olan bu projeler;

SWEBOK(Software Engineering Body of Knowledge) :

Yazılım mühendisliği çekirdek bilgisinin tanımlanması.

SWCEPP(Software Engineering Code of Ethics and Professional Practice) :

Yazılım mühendisliği etiklerinin tanımlanması.

SWEEP (Software Engineering Education Project) :

SWEBOK ile uyumlu olarak örnek bir eğitim programı tanımlanması, olarak sıralanmaktadır.

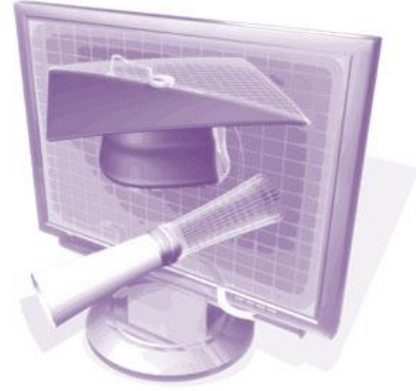
Sonuç ;



Yazılım Mühendisliği eğitiminde teknik bilgi ve beceriler yanında hukuki kavramlar, etik değerler, takım çalışması, proje yönetimi gibi soyut fakat önemli kavramların da kişiye kazandırılması amaçlanmıştır.

- ✓ Sarıdoğan , E., Yazılım Mühendisliği, Papatya Yayınevi,2004.
- ✓ Arifoğlu, A., Doğru, A., Yazılım Mühendisliği, Sas Bilişim Yayınları,2004.
- ✓ Ders Notları.

Yazılım Mühendisliği Eğitimi



Bir yazılım mühendisliği (lisans) mezununun sahip olması gereken yetenekler şunlardır :

1. Yazılım ürünleri geliştirmek için **bir takımın parçası** olarak çalışmak,
2. **Kullanıcı gereksinimlerini** belirlemek ve onları **yazılım gereksinimlerine** çevirmek,
3. Çelişen amaçları düzenlemek, maliyet, zaman, bilgi ve organizasyon kısıtlamaları içinde kabul edilebilir uzlaşmalar bulmak.
4. Bir veya daha çok uygulama alanı için, etik, sosyal, yasal ve ekonomik ilgileri bütünleştiren mühendislik yaklaşımlarını kullanarak uygun çözümler tasarlamak.
5. Yazılım tasarımı, geliştirilmesi, gerçekleştirimi ve doğrulanması için bir temel sağlayan mevcut teorileri, modelleri ve teknikleri anlamak ve uygulayabilmek.
6. Tipik bir yazılım geliştirme ortamında etkin olarak çalışmak, gerekli olduğunda liderlik yapabilmek ve kullanıcılarla iyi iletişim kurabilmek.
7. Yeni modelleri, teknikleri ve teknolojileri öğrenebilmek.

Yazılım Mühendisliği Eğitimi Bilgi Alanları ;

- Temeller
- Profesyonel Uygulama
- Gereksinimler
- Tasarım
- Yazılım oluşturma
- Yazılım sına ve doğrulama
- Yazılım gelişimi
- Yazılım Süreci
- Yazılım Kalitesi
- Yazılım Yönetimi

Yazılım Mühendisliği Eğitimi Bilgi Alanları

Temeller:

Yazılım mühendisliğinin ürettiği ürünlerin niteliklerini anlatan teorik ve bilimsel temellerden, bu ürünleri modellemeyi ve tanımlamayı kolaylaştıran matematiksel temellerden ve öngörülebilir sonuçlar üreten ana ilkelerden oluşur.

Buradaki ana nokta, kaynakları belirlenmiş bir amaca dönüştürmek için mühendislik tasarımı ve mühendislik biliminin uygulanmasıdır.

Yazılım Mühendisliği Eğitimi Bilgi Alanları

Profesyonel Uygulama:

Yazılım mühendislerinin, yazılım mühendisliğini profesyonel ve etiğe uygun olarak uygulayabilmeleri için sahip olmaları gereken bilgi, beceri ve davranışlarla ilgilidir.

Profesyonel uygulamalar bilgisi teknik iletişim, psikoloji ve sosyal ve mesleki sorumlulukları içerir.

Yazılım Mühendisliği Eğitimi Bilgi Alanları

Gereksinimler:

Bir sistemin amacını ve hangi içerikte kullanılacağını tanımlar.

Gereksinimler, kullanıcıların gerçek gereksinimleri ile yazılım ve diğer bilgisayar teknolojileri arasında köprü oluşturur.

Gereksinimlerin belirlenmesi, sistemin fizibilite çalışmasını, kullanıcıların gereksinimlerinin analizi, sistemin ne yapacağını ve ne yapmayacağını kısıtlamalar göz önünde alınarak belirlenmesini ve bu bilginin kullanıcılar tarafından doğrulanmasından oluşur.

Yazılım Mühendisliği Eğitimi Bilgi Alanları

Tasarım:

Bir bileşenin veya bir sistemin nasıl gerçekleştirileceğini belirlemek için kullanılan teknikler, stratejiler, gösterimler ve desenlerle ilgilidir.

Tasarım, kaynaklar, performans, güvenilirlik ve güvenlik gibi kısıtlamalar göz önüne alınarak işlevsel gereksinimlere uygun olmalıdır.

Ayrıca, yazılım bileşenleri arasındaki içsel ara yüzler, mimari tasarım, veri tasarımı, kullanıcı ara yüzü tasarımı, tasarım araçları ve tasarımın değerlendirilmesi de bu alanın kapsamındadır.

Yazılım Mühendisliği Eğitimi Bilgi Alanları

Yazılım Oluşturma:

Bu alan, tasarımda belirlenmiş yazılım bileşenlerinin geliştirilmesiyle ilgili bilgileri içermektedir.

Bu kapsamda, bir tasarımın bir gerçekleştirim diline çevrilmesi, bileşen sınamaları ve program belgelemeleri incelenmektedir.

Yazılım Mühendisliği Eğitimi Bilgi Alanları

Yazılım Sınama ve Doğrulama:

Elde edilen programın hem belirlenen gereksinimleri sağladığını hem de gerçekleştirimin beklenenlere uygun olduğunu kontrol etmek için statik ve dinamik sınama teknikleri kullanır.

Statik teknikler, yazılımın tüm yaşam döngüsü boyunca elde edilen gösterimlerin analizi ve kontrolüyle ilgilenirken, dinamik teknikler sadece gerçekleştirilmiş sistemi içerir.

Yazılım Gelişimi:

Yazılımın kullanıma verilmesinin öncesindeki ve sonrasındaki aşamalarda etkin bir maliyetle desteklenmesini sağlar.

Bu destek, gelişen sistemi oluşturan versiyonların veya sürümlerin her biri için hazırlık aktivitelerine gerek duyar.

Bu aktiviteler, planlama, ölçüt desteği, regrasyon sınama ve karmaşıklık kontrolünü içermektedir.

Bu aktiviteleri desteklemek için kullanılan teknikler, program anlama, sürüm planlaması, değişiklik tanımlaması, yeniden mühendislik, tersine mühendislik, bakım, sistemin kullanımına son verilmesini içerir.

Yazılım Mühendisliği Eğitimi Bilgi Alanları

Yazılım Süreci:

Yaygın olarak kullanılan yazılım yaşam döngüsü süreç modellerinin tanımlanmasıyla ilgili bilgileri ve kurumsal süreç standartlarını;

yazılım süreçlerinin tanımlanmasını,

gerçekleştirilmesini,

ölçülmesini,

bakımını,

yönetimini,

değiştirilmesi ve iyileştirilmesini ;

ve yazılım geliştirme ve bakımı için gereken teknik ve yönetsel aktiviteleri gerçekleştirmek için tanımlı bir süreç kullanımını kapsamaktadır.

Yazılım Yönetimi:

Tüm yazılım yaşam döngüsü aşamalarının planlanması, düzenlenmesi ve izlenmesiyle ilgili bilgileri içermektedir.

Yazılım geliştirme projelerinin başarısı için, farklı organizasyonel birimlerdeki işlerin koordinasyonu için, yazılım versiyonlarının bakımı için, kaynakların gerekli oldukları zaman var olabilmesi için, projedeki işlerin uygun olarak bölünebilmesi için, iletişimin kolaylaşması için kritik önemdedir.

Yazılım Kalitesi:

Yazılım geliştirmenin ve bakımın tümünü etkileyen ve tümünden etkilenen bir kavramdır.

Hem geliştirilen ürünlerin kalitesini hem de bu ürünleri geliştirmek için kullanılan süreçlerin kalitelerini içerir.

Ürün kalite nitelikleri,

1. kullanılabilirlik,
2. güvenilebilirlik,
3. güvenlik,
4. bakıma uygunluk,
5. esneklik,
6. Etkinlik,
7. performans

gibi kriterleri kapsamaktadır.

Temel Bileşenleri

1. Ekip olarak çalışmak için gerekli bilgiyi sağlayan **Etkin İletişim ve Grup Çalışması Yetenekleri.**
2. Öğrencileri gerçek hayat problemlerine hazırlayan **Bir Uygulama Alanında Deneyim.**
3. Öğrencileri gereksinimlerin değişmesi, proje yönetimi, konfigürasyon yönetimi, araç kullanımı gibi konulara hazırlayan **Bir Ekip Projesi.**
4. Öğrencileri gerçek hayat ortamına hazırlayan **Çalışma Ortamında Deneyim.**
5. Ders notu veya ders gibi belirli bir düzende sağlanmayan bilgiyi aramak, değerlendirmek ve kullanmak için gerekli beceriyi kazandırmayı amaçlayan **Yaşam boyu Öğrenme Araçları.**

6. Yazılım ve donanım ürünleri hakkında temel teknik bilgi ve becerileri sağlayan **Bilgisayar Bilimi Temelleri**. Bunlar, programlama dilleri, modelleme, veritabanları, işletim sistemleri, ağ sistemleri, algoritmalar olarak sayılabilir.
7. Yazılımı ve ilgili belgelemeyi oluşturmak ve bakımı yapmak için gerekli teknik bilgi, yetenek ve araçları sağlayan **Yazılım Mühendisliği Temelleri**. Bunlar arasında, yazılım süreçleri, yaşam döngüsü modelleri, yazılım ölçütleri, mimari ve tasarım yöntemleri bulunmaktadır.
8. Genel sistem ilkelerinin, ekonominin ve mühendislerin görev ve sorumluluklarının anlaşılmasını sağlayan **Mühendislik Uygulamaları ve Etiği**

Yazılım mühendisliği

6 seneye yayılmış bir Eğitim programıdır.

Bu program 2 bölüme ayrılır:

- Temel Yazılımcılık
- Uzman Yazılımcılık

Temel yazılımcılık :

Temel yazılımcılığın süresi 4 Dönemdir.

Bu dört dönem içerisinde yazılımcı adayı bilgisayar yazılımcılığı hakkında temel bilgiler edinir.

- Algoritma geliştirme,
- Sistem analizi,
- Veri tabanı analizi,

Yazılım dilleri hakkında genel bilgiler....

- C ----- C++ ----- Visual C ---- Borland C#
- Assembly (8/16/32/64)
- Sistem analizi
- Veri Tabanı analizi
- Delphi ----- Pascal----Delphi400
- Java --- JavaScript---- Perl ---- Web Tasarım --- CQI --- CGI --- ClientSERVER
- Grafik
- Animasyon
- Projeksiyon --Promasyon --- Prodüksiyon
- . net
- AS400 ----- RPG400
- Ağ yönetimi
- Cracker
- Hacker
- ...

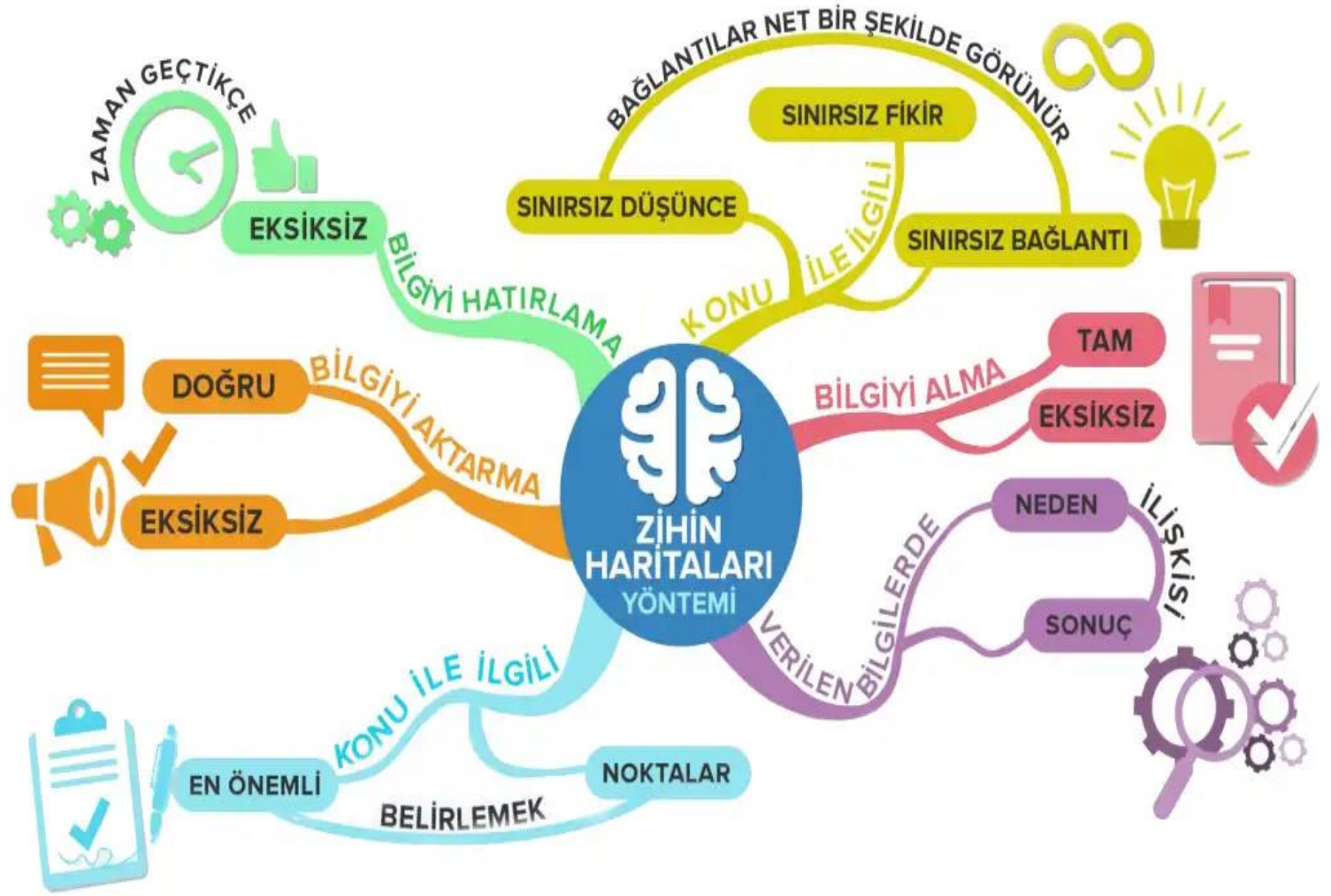
Uzman Yazılımcılık:

Yazılım Mühendisi adayı yukarıda belirtilen konulardan herhangi birini seçer. Daha sonra bu konu hakkında ayrıntılı bilgi edinirler ve uzmanlaşırlar.

“Böylece üretilen işin kalitesi gün geçtikçe artar.”

**Orta büyüklükte bir Yazılım şirketi
en az şu yazılımcıları bulundurmak zorunda kalacak.**

- Sistem Analisti Yazılım Mühendisi
- Hacker Yazılım Mühendisi
- Cracker Yazılım Mühendisi
- Algoritma Yazılım Mühendisi
- Veri Tabanı Analisti Yazılım Mühendisi
- Grafiker Yazılım Mühendisi
- Animasyon Yazılım Mühendisi
- Assembly Yazılım Mühendisi
- Herhangi bir dil kullanan yazılım Mühendisi







Uygulama ...

Kaynak :

N.Y. Topaloğlu

Ege Üniversitesi

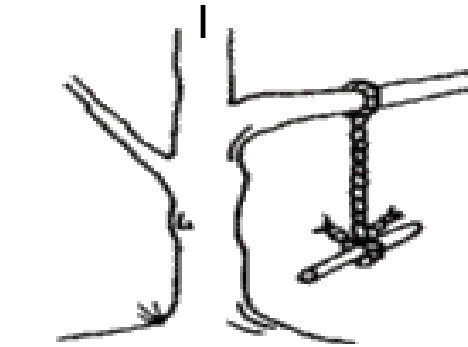
Mühendislik Fakültesi

Bilgisayar Mühendisliği Bölümü

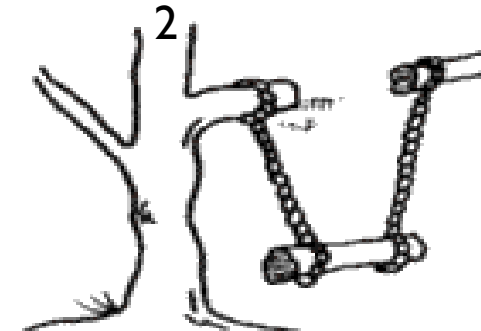
Yazılım Mühendisliği



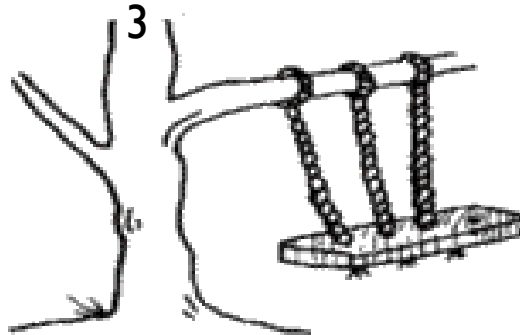
Ekip çalışması ☺



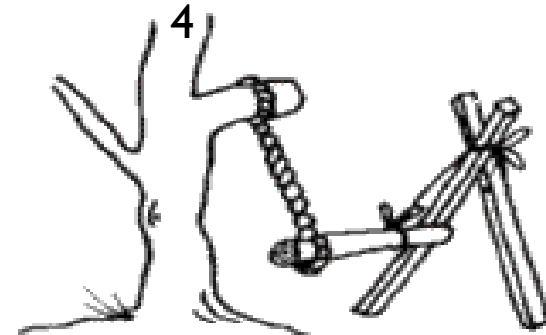
What the user asked for



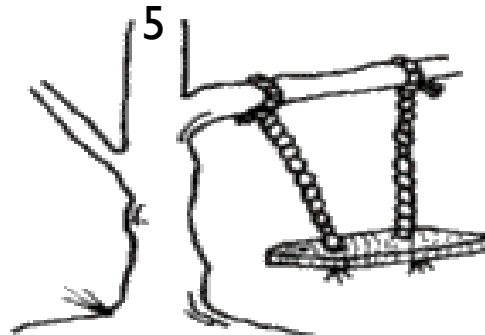
How the analyst saw it



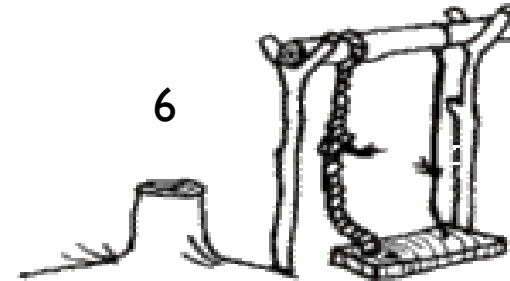
How the system was designed



As the programmer wrote it

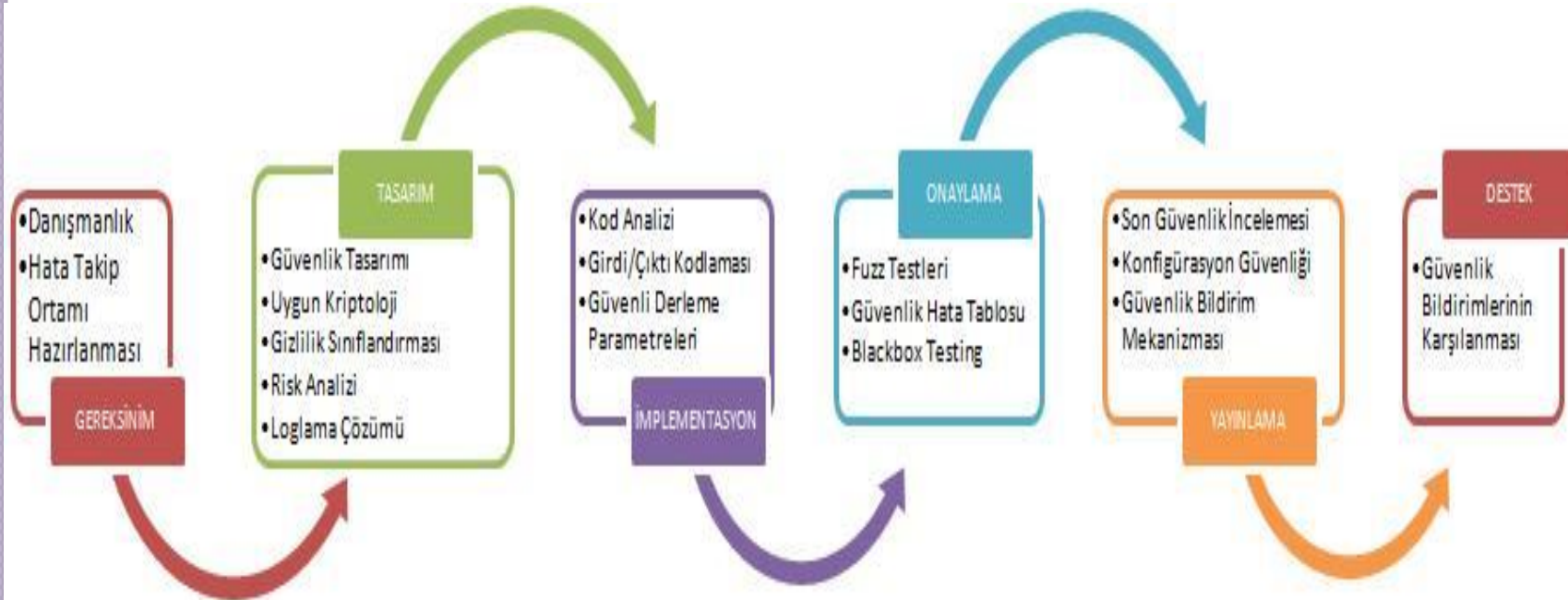


What the user really wanted



How it actually works

Yazılım Yaşam Döngüsü (YYD)

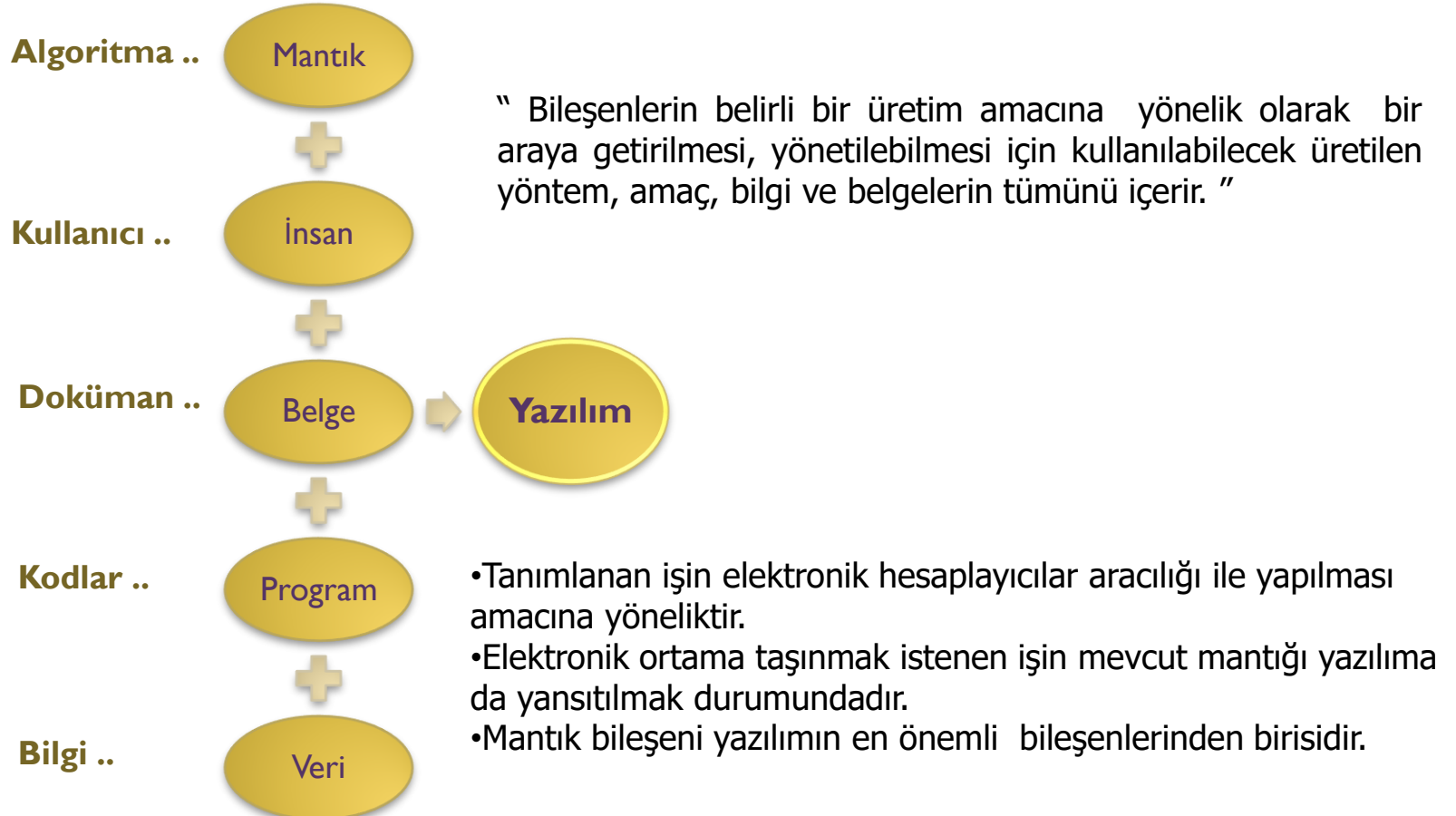


Temel Kavramlar ;

Yazılım: Bir sistemin donanım bileşenleri dışındaki her şey olarak tanımlanabilir.

Yazılım; Bilgisayar program parçası yada programlar grubu olarak an

Yazılımı oluşturan bileşenler



- Her tür yazılım mutlaka bir veri üzerinde çalışma durumundadır.
- Veri dış ortamdan alınabilir yada yazılım içerisinde üretilebilir. 'veri' yi 'bilgi' ye dönüştürme en önemli amaçtır.
- Yazılım üretimi, bir mühendislik disiplini gerektirir.
- Yazılım Yasam Döngüsü (YYD) mühendisler tarafından üretim sırasında kullanılan yasam döngüsünden esinlenerek oluşturulmuştur.
- Yazılım üretimi sırasında, birçok aşamada yapılan ara üretimler, bilgi belge üretimidir.
 - Planlama bilgileri,
 - Çözümleme bilgileri,
 - Tasarım bilgileri,
 - Gerçekleştirim bilgileri

Yazılımın insan bileşeni iki boyutludur.

1. Yazılımı geliştirenler
2. Yazılımı kullananlar

“Çok kişili ekiplerle geliştirilmektedir. Temel nedeni yazılımın yaygınlaşması ve boyutlarının büyümesidir.”

Yazılımın ana çıktısı sonuçta bir bilgisayar programıdır.

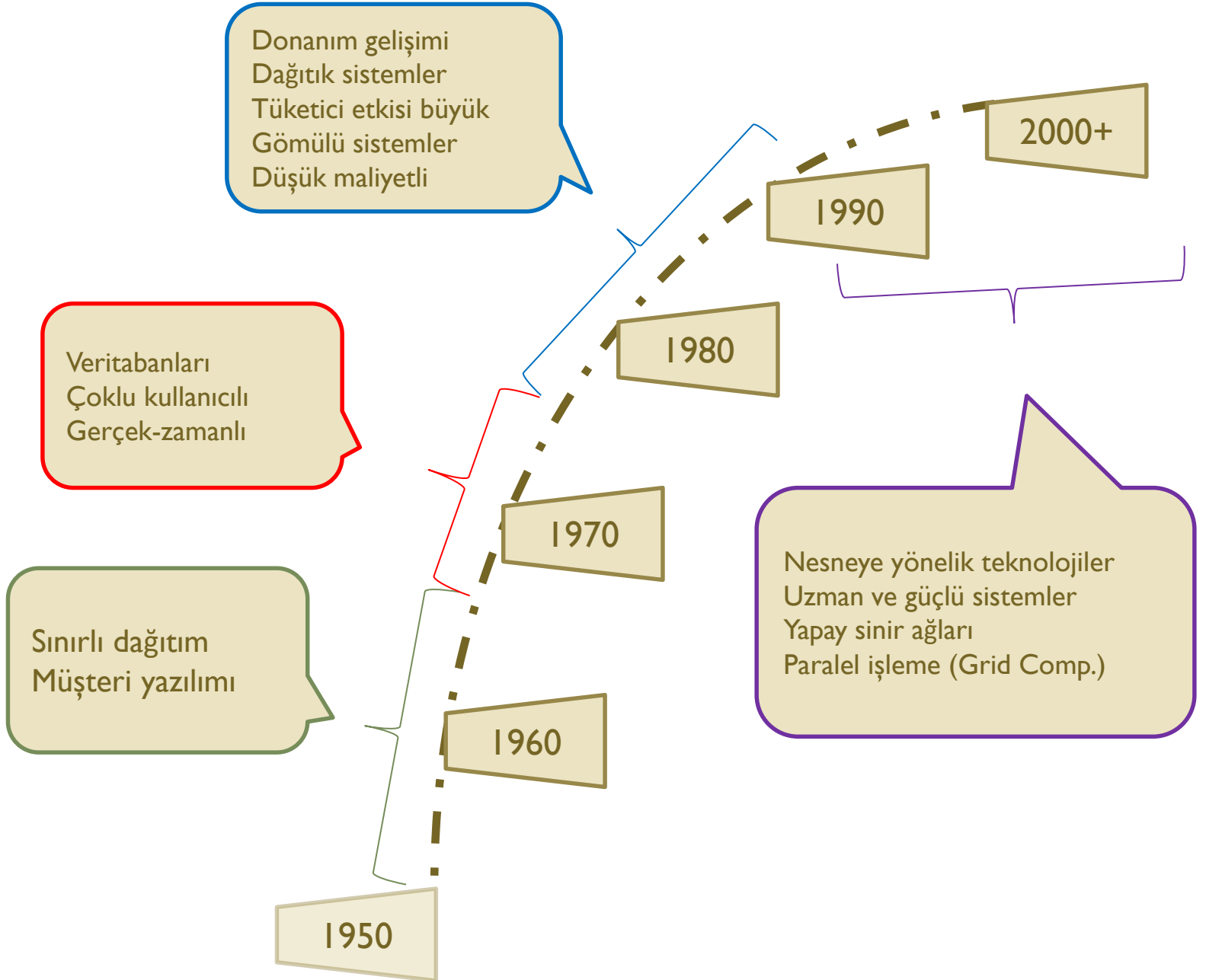
- İşletime alınan programın hemen ardından bakım çalışmaları sürekli olarak gündeme gelir.
- Hiçbir program bütünüyle her olasılık göz önüne alınarak sınanamaz, dolayısıyla hata ihtimali her zaman mevcuttur.
- İşletmeler doğaları gereği dinamik bir yapıya sahiptir, dolayısıyla mevcut sistemin sürekli olarak yeni istek ve gereksinimleri ortaya çıkar.
- Ortaya çıkan talepler ve değişiklikler aynı disiplin içerisinde sisteme eklenmelidir.

“Değişik bilgisayar bilimi teknolojilerinin ve kişilerin bir bilgi yada yazılım sistemi oluşturmak amacıyla bir araya getirilmesinde bir bütünleştirici gibi çalışır.”

“Yazılım mühendisi bir programcı değildir, ancak programcının tüm yeteneklerine sahiptir.”

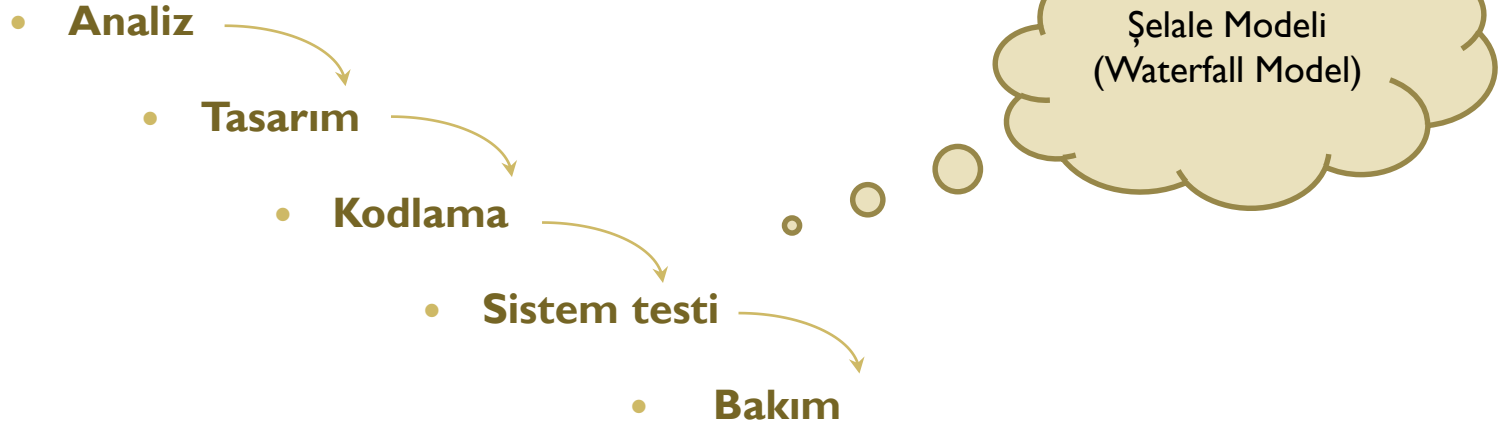
“Yazılım projelerinde temel hedefi, söz konusu üretimin az maliyet, yüksek nitelik ve talebe uygun yapılmasıdır.”

Yazılım Gelişim Süreci :



Yazılım geliştirme adımları

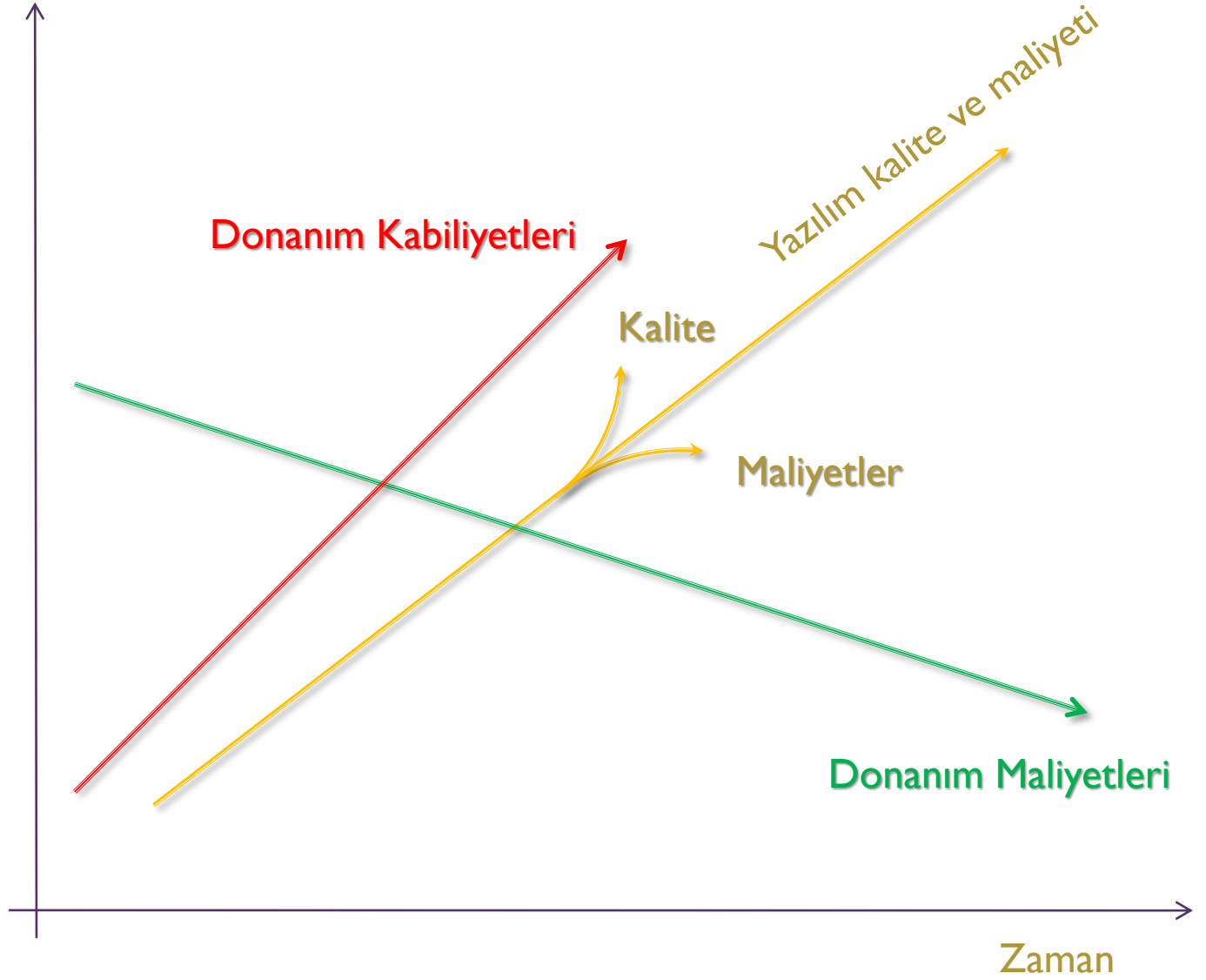
- Gereksinim analizi (requirements analysis)
 - Fonksiyonel ve fonksiyonel-olmayan gereksinimler (functional & non-func. reqs.)
- Tasarım (design)
 - Sistem tasarımı (system design): subsystems
 - Detaylı tasarım (detailed design)
- Kodlama ve birim testi (unit testing)
 - Birleşenlerin ayrı ayrı gerçekleştirilmesi ve birim testi
- Bütünleme testi ve sistem testi
 - (integration & system testing)
- Bakım ve güncelleme (maintenance)
 - Hataların giderilmesi, performans iyileştirme,
 - servislerin geliştirilmesi, değişikliklere uyum, vs.



Yazılım üretiminin, mühendislik yöntemleriyle yapılmasını öngören ve bu yönden yöntem, araç, teknik ve metodolojiler üreten bir disiplindir.

- Yöntemler
- Araçlar
- Teknikler

“Yazılım üretiminde (Yaşam döngüsü) belirtilen aşamaların sistematik olarak izlenmesi ve gerçekleştirilebilmesi yazılım mühendisliği için ön şarttır.”



Problemler !

Büyük çaptaki uygulamalar ne kadar test edilsede, hiçbir zaman %100 hatadan arındırılmaz.

Yazılım geliştirme (çoğunlukla) ;

- Proje temellidir
- Yoğundur (bilgi yoğun)
- Genellikle bütçelerini aşar
- Öngörülen zamandan uzun süren
- Zorluklarla doludur
- Zaman alıcıdır
- Hesapta olmayan masraflar gerektirir
- Sürekli devam eden süreçtir.
- ...

Problemler !

- Değişik yetenekte bir çok personel (raportör, programcı, test sorumlusu, çözümleyici),
- Yeniliğe ve değişime tepki gösteren kullanıcı ve yöneticiler,
- Standart ve yöntem eksiklikleri,
- Yeterince tanımlanmamış, oldukça karmaşık kullanıcı beklentileri,
- Personel sirkülasyonunun fazla olması,
- Yüksek eğitim maliyetleri,
- Dışsal ve içsel kısıtlar (maliyet, zaman, işgücü),
- Verimsiz kaynak kullanımı,
- Mevcut yazılımlardaki yetersizlik ve kalitesizlik,
- Üretim maliyetlerinin yüksek olması,

Hatalar

Yazılım üretiminde Hataların Dağılımı :

- Mantıksal Tasarım %25
- İşlevsel Tasarım %15
- Kodlama %35
- Belgeleme ve Diğer işlevler %25

Yazılım Üretiminde Hata Düzeltme Maliyetleri :

- Çözümleme %1
- Tasarım %5
- Kodlama %15
- Test %30
- Kabul Testi %40
- Uygulama %100

Mühendislik !

Yazılımın mühendislik olarak ifade edilmesi demek ;

Belirli standartlara uyması ve **ölçülebilir olması** gerekir.

- Kalite standartları,
- Kalitenin ölçülebilmesi
- Verimin standartları,
- Verimin ölçülebilmesi

Kalite

- Hata sayısının düşük düzeyde olması
- Kullanıcı isterlerine cevap oluşturabilme (tamamını gerçekleştirebilme)
- Arızalar arası zamanın uzunluğu
- Destek ve gelişme.

Yazılımlar

- Sistem yazılımı
- Gerçek-zamanlı yazılım
- İş yazılımı
- Mühendislik ve bilimsel yazılım
- Gömülü yazılım
- Kişisel bilgisayar
- Yapay zeka yazılımı
- ...

Sınıflama

- İşlevlerine göre
- Zamana dayalı ve uygulama alanlarına göre
- Boyutlarına göre

Örnekler

- Hesaplama (Nümerik Çözümleme)
- süreç temelli (Gömülü sistemler)
- Veri isleme (finans sektörü)
- CAD (Sinyal işleme)
- Kural Temelli (Robotik, Yapay Zeka)

Yazılım Mühendisliğinde İçerik Bilgileri



Bilgisayar Temelleri

- Algoritmalar, veri yapıları, programlama dilleri
- İşletim sistemleri, matematik



Yazılım Mühendisliği

- Mühendislik disiplini, çözümleme, tasarım,
- Gerçekleştirim(kodlama), test, bakım, geliştirme



Yazılım Yönetimi

- Süreç yönetimi, Risk yönetimi, Kalite yönetimi
- Geliştirme yönetimi, Kazanım yönetimi



Yazılım Alanları

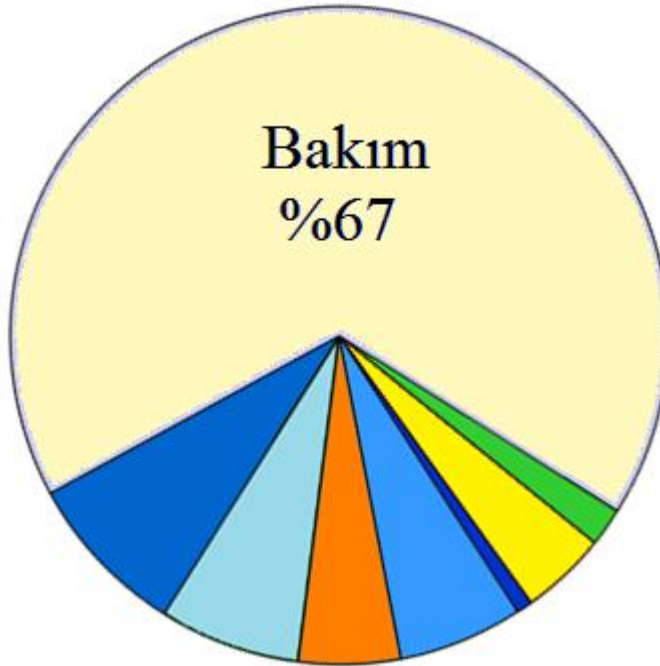
- Veri tabanı, mühendislik, Gömülü sistemler
- Yapay zeka, ...

Süreç bilgi alanları

- Üretimi ve işlemleri içine alan mühendislik disiplini
- Yazılım Mühendisliği Yönetimi
- Yazılım ihtiyaç Analizi
- Yazılım Yönetimi
- Yazılım Tasarımı
- Yazılım Yapılandırılması
- Yazılım Testi
- Yazılım Mühendisliği Altyapısı
- Yazılım Mühendisliği işlemi
- Yazılım Değerlendirme ve Bakımı
- Yazılım Kalite Analizi

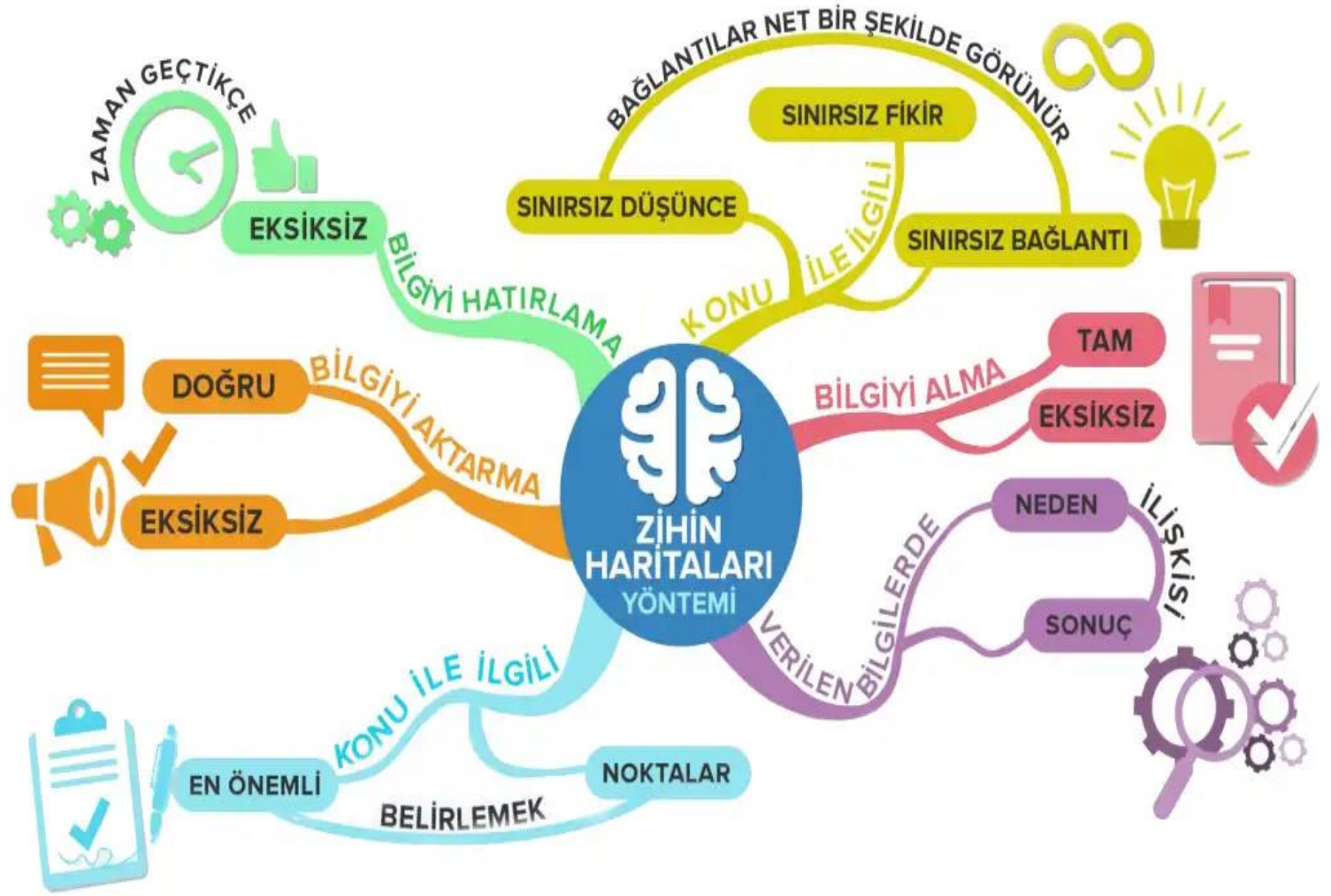
Örnek meslekler ...

- Sistem Analisti
- Yazılım Mimarı
- Yazılım Proje Yöneticisi
- Yazılım Programcısı
- Yazılım Sistem Yöneticisi
- Yazılım Veri Tabanı Yöneticisi
- Yazılım Test elemanı
- Yazılım Ağ Uzmanı
- Yazılım Güvenlik Mühendisi
- Yazılım Konfigürasyon Yöneticisi
- Yazılım Kalite Yöneticisi
- ...



Göreceli maliyetlere göre
yazılım yaşam döngüsü aşamaları.

İhtiyaç belirleme	% 2
Şartname belirleme	% 4
Planlama	% 1
Tasarım oluşturma	% 6
Gerçekleştirim(kodlama)	% 5
Test süreci	% 7
Entegrasyon süreci	% 8
Bakım süreci	% 67







Uygulama ...

Kaynak :

- Roger S. Pressman, Software Engineering – A Practitioner's Approach, 6th Ed., McGraw Hill, International Edition, 2004,
- Prof. Dr. Ş.Sağıroğlu ders notları
- N.Y. Topaloğlu (Makale)
- Wikipedia



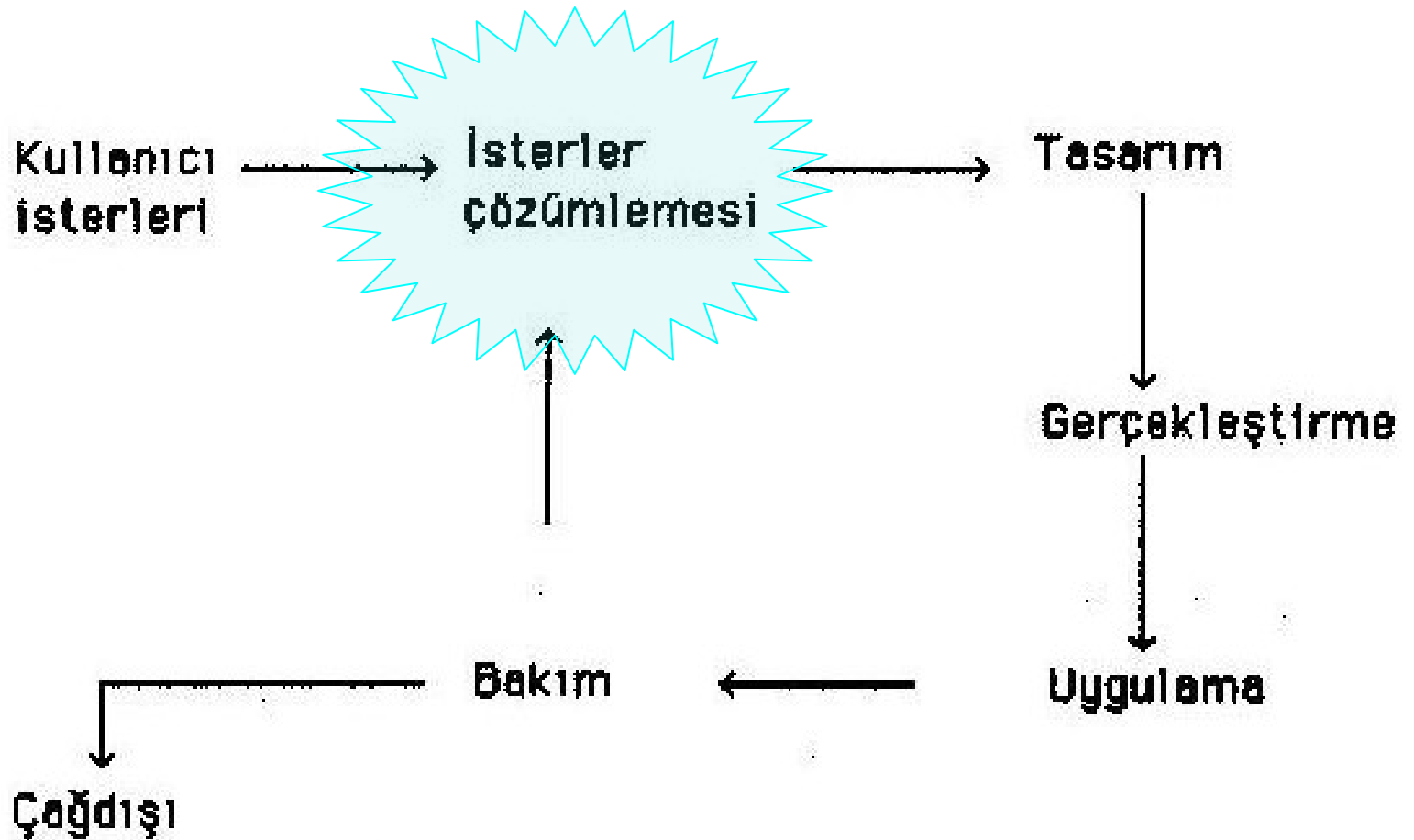
Yazılım İsterlerinin Çözümlemesi

Yazılım Yaşam Çevrimi

- “ Yazılım Yaşam çevrimin herhangi bir yazılım geliştirme uzmanı ya da deneyimli bir bilişim yöneticisi tarafından kolayca birkaç farklı biçimde ayrıntılandırılması mümkündür. ”
- “ Hangi aşamalardan geçilirse geçilsin, nasıl bir geliştirme yöntemi uygulanırsa uygulansın, kullanıcının başlıca üç noktada bu çevrime yaşamsal katkısı vardır: ”

1. **İsterlerin belirtilmesi,**
2. **Yazılımın kabulü,**
3. **Alınan sistemin kullanım sonrası değerlendirmesi.**

Yazılım İsterlerinin Çözümlemesi



Yazılım yaşam çevrimi.

Sistem geliştirme aşaması ve İşlemleri

I. Tanımlama Aşaması

Ön araştırmayla İlgili İşlemler;

1. Proje sözleşmesi
2. Hedeflerin ve yönlerin belirlenmesi
3. Öneriler, seçenekler ve beklenen yararların/sonuçların belirtildiği bir rapor
4. Ön sistem incelemesi İçin zamanlama ve maliyet tasarısı
5. {3} ve (4)'ü kullanıcılara ve yönetime sunarak bir sonraki adıma geçme yetkisi

Ön sistem İncelemesiyle İlgili aşamalar:

6. Şimdiki sistemin maliyeti ve İlgili belgelerin toplanıp incelenmesi
7. Giriş/çıkış gerekleri belirlenmesi
8. Üst düzey iş akışı çıkarılması
9. Yazılım paketleri için ön liste hazırlanması
10. Maliyet ve getiri öngörüsü
11. Bir sonraki adım için zamanlama ve maliyet
12. (11)'i kullanıcı ve yönetime sunarak bir sonraki adıma geçiş

Sistem çözümleme ile ilgili işlemler

13. Toplanan verilerin doğrulanması
14. Geliştirme, sinama ve uygulama planı
15. (9)'da hazırlanan paketler ön listesiyle işlevsel gereklerin (7,8) karşılaştırılması
16. Bir sonraki adım için sistem tasarım planı hazırlanması
17. Kullanıcı ve yöneticilerden bir sonraki aşama için onay

II. Geliştirme Aşaması Sistem Tasarımıyla ilgili İşler

18. Veri tabanı tasarımı ve değerlendirmesi
19. Donanım gerekleri belirlenmesi
20. Sınama verilerinin saptanması
21. Her bir işlevin mantık tasarımı
22. Giriş/çıkış gerekleri kesinleştirilmesi
23. Uygulama/geçiş planı ve kullanıcı işlemlerinin belirlenmesi
24. Programlama planı
25. Kullanıcı ve yöneticilere (24)'ün sunuluşu ve bir sonraki adıma geçiş

Programlama ile ilgili işler

- 26. Sistem gerekler tanımının gözden geçirilmesi ve Programcıların görevlendirilmesi
- 27. Veri tabanı tasarımı (18) ve program tanımının (21, 22, 24, 26) gözden geçirmesi
- 28. Tasarım üzerinden gidilmesi (Yapısal gözden geçirme)
- 29. Sınama verilerinin hazırlanması
- 30. Program yazımı ve düzeltimi
- 31. Program ve işletim belgeleri hazırlanması
- 32. Tüm sistem sinaması
- 33. Uygulama için yönetim onayı

III. Uygulama Aşaması Sınama ve yerleştirmeyele İlgili işler.

- 34. Geçiş ve sınama planları
- 35. Kullanıcı eğitimi
- 36. Kullanım ve işletim el kitaplarının elden geçirilmesi
- 37. Kullanım ortamında tüm sistemin sınanması
- 38. Sonuçların değerlendirilmesi ve geçişin gerçekleştirilmesi
- 39. Kullanıcı kabulü

Proje değerlendirme ile ilgili İşler

- 40. 1–6 ay içinde kullanıcı değerlendirmesi
- 41. Sistem geliştirme süreciyle ilgili öneriler geliştirilmesi.

“Yazılım yaşam çevriminin hangi aşamalara ayrıldığı ve bunların gerçekleştirilme sırası, yazılım geliştirme yaklaşımına bağlıdır.”

Geleneksel yaklaşım

1. Kullanıcı isterlerinin işin başında yeterli açıklık ve doğrulukla tanımlanabileceği,
2. Bu isterlerin tanımında kullanıcı ile geliştirici arasında anlaşmaya varılabileceği, bunun için yeterli zaman ve diğer kaynakların ayrılabilceği,
3. İsterler tanımlama ve tasarım aşamaları yeterli nitelikte hazırlık oluşturarak sonuçlanmadan programlamaya geçilmeyeceği için, geliştirme projesinin başlamasından uzun bir süre sonraya kadar kullanıcıya hiçbir ürün verilmemesinin kabul edilebileceği durumlarda olumlu sonuç verecektir.

Yazılım Projesi Gerekçesi :

1. Gereken yazılımın kısa tanımı.
2. Gereksinim nedenleri.
3. Söz konusu işin bugün nasıl görüldüğü.
4. İstenen yazılımın sağlayacağı hizmetler.
5. Bu yazılımdan yararlanacak kuruluş birimleri ve işlevleri.
6. Beklenen proje maliyeti ve öngörülen gerçekleşme süresi.

“ Yazılım projesinin gerekçesi oluşturulurken, bilişim aşamalarının gelişmesi de bilinir ve proje tanımında yol gösterici olursa, hem çalışma daha güvenilir hedeflere yönelecek, hem de yönetim katmanlarının desteği daha kolay sağlanacaktır. ”

İsterler Anlatımı

- Tüm özel terimler tanımlanmalı,
 - Önce genel, sonra artan düzeyde ayrıntıya yer verilmeli,
 - Çizelge ve resimler kullanılmalı,
 - Tüm isterler çizelgelerle ilintilendirilmeli,
 - İsterler somut olarak ölçülebilecek en üst (ayrıntısız) düzeyde tanımlanmalı,
 - Belgenin bölümleri arasında tutarlılık sağlanmalı; bütünlük içinse:
 - Kullanıcı gerekleri ve istenen başarımlar düzeyleri belirtilmeli, (bunları sağlayacak yazılımın nasıl kurulacağı değil,)
 - İşlevsel isterler (ne sağlanacağı),
 - Veri tabanı (hangi bilgilere dayanılacağı),
 - Başarım isterleri-ölçütler (Kullanıcı sayıları, veri hacim ve sıklıkları) belirtilmeli,
 - Kullanılacak donanım ve destek yazılımı ile ilgili kısıtlayıcı ya da belirleyici özellikler belirlenmeli,
- 📁 geliştirmeye ilgili kısıtlar (zaman, öncelik sırası, yazılım mülkiyeti, v.b.) varsa belirtilmelidir.

İsterler Anlatımı

“İsterler belgesinin bütünlüğü, örneğin, yazılımın olabilecek tüm veri girişlerinde, görülecek tepkisini tanımlamak anlamına gelir.

Yalnızca doğru ve tutarlı verilerle ne yapılacağı anlatılıp hatalı, eksik ya da çelişkili verilere nasıl tepki gösterileceğinin belirlenmemesi, bu açık noktanın tasarım aşamasında belki de tümüyle kullanıcının beklentisine uymayan biçimde karşılanmasına yol açabilir. ”

İsterler İçeriği :

Yazılım isterlerinin belirtilmesinde üç yöntem kullanılabilir ;

1. **Girdi/çıkıtı belirtimi,**
2. **Belirleyici örnekler,**
3. **Modeller.**

Girdi/çıkıtı belirtimi:

Olası girdilerin üreteceği tepkileri ya da istenen çıktıları almak için verilecek girdileri belirtmeye dayanan bu yöntem, tüm olası girdileri göz önüne almanın güçlüğü nedeniyle bazı durumlarda kullanılamayabilir.

Belirleyici örnekler:

Tüm girdiler ele alınmamakla birlikte bazı durumlarda iyi seçilmiş bazı örneklerle sistemin nasıl çalışmasının gerekliliği anlaşılmayan nokta bırakmayacak biçimde açıklanabilir.

Modeller:

Soyut (örneğin matematiksel ya da çizgesel) modellerin oluşturulabildiği durumlarda, diğer yöntemlerden çok daha kesin belirtilmelerin elde edilmesine yarayabilir.

Yazılım İsterler Belgesi İçindekiler Önerisi:

1. Giriş

- 1 Amaç
- 2 Kapsam
- 3 Tanım ve kısaltmalar
- 4 Göndermeler
- 5 Özet

2. Genel belirleme

- 1 Ürünün yeri
- 2 Ürünün işlevleri
- 3 Kullanıcı özellikleri
- 4 Genel kısıtlar
- 5 Varsayım ve bağımlılıklar

3. Özel isterler Ekler

- 1 ...

Yazılım İsterleri Belirlemede Biçimsel Yöntemler

- Biçimsel olmayan kullanıcı isterlerinin sistem belirtimine dönüştürülmesini kolaylaştırmalı,
- Belirtimin tutarlılık, bütünlük gibi niteliklerinin denetlenmesine olanak sağlamalı,
- Ortaya çıkan sistem belirtiminin gerçekten kullanıcı isterlerine karşı gelip gelmediğini görebilmek için kullanıcının biçimsel olmayan diline dönüştürülebilmeli.

Yazılım İsterlerinin Çözümlemesi

1. İşlevsel isterlerin anlaşılabilir, kesin ve kullanıcı açısından anlamlı biçimde ortaya konulabilmesi,
2. Belirtimin içsel tutarlılığı, kullanıcı örneklerini destekleyebilmesi, işlem sonuçlarının doğruluğu, v.b. özelliklerin doğrulanabilmesi,
3. Kullanıcının kaynak ve başarımlarına uyan bir gerçekleştirimin oluşturulabilmesi, işlevsel isterleri karşılayan, işlevsel-dışı olanları da kısıtlamayan bir belirtimin ortaya konulabilmesi,
4. Belirtimin oluşturulması, doğrulanması ve gerçekleştirilmesinin, kullanışlı biçimde yapılabilmesi,
5. Belirtim yönteminin kullanımının, ekonomik olarak olurlu olması.

Yazılım Niteliği Ölçütleri

Başarıma ilişkin etmenler:

Verimlilik : Kaynak kullanımı.

Güvenilirlik : Sonuçlara ne ölçüde güvenildiği.

Tutarlılık : Yazılım birimlerinin ne ölçüde çelişkisiz sonuç verdiği.

Kalımlılık : Uygunsuz koşullarda çalışabilme yeteneği.

Kullanışlılık : Kullanım ve öğrenim kolaylığı.

Tasarıma ilişkin etmenler:

Doğruluk : İsterlere uygunluk düzeyi.

Bakım kolaylığı : Onarım olanağı.

Doğrulanabilirlik: Sonuçların doğruluğunun ne ölçüde denetlenebildiği.

Genişletilebilirlik: Yeteneklerin artırılabilirle, yazılımın geliştirilebilme düzeyi.

Uyarlanabilirliğe ilişkin etmenler:

Esneklik : Değiştirilebilme, yeni uygulamaları destekleme ölçüsü. Uyumluluk: Çalışabilme, uyum düzeyi.

Taşınabilirlik: Farklı donanımlar ve destek yazılımları üzerinde çalışabilme düzeyi

“Bu etmenler aynı zamanda özellikle satın alınacak yazılımın değerlendirilmesinde kullanılmaya uygun nitelik ölçütlerininide oluşturmaktadırlar. ”

Nitelik etmenleri:

Yazılımın çalışmasına ilişkin etmenler

Doğruluk : isterlere uygunluk.

Güvenilirlik : Sonuçların doğruluk ve duyarlılığı.

Verimlilik : Yazılımın gerektirdiği bilgisayar kaynakları, zaman ve program uzunluğu.

Korunmuşluk: Yetkisiz kullanıma ve zarara karşı korunma düzeyi.

Kullanışlılık : Öğrenim, işletim ve genel olarak kullanıma uygunluk düzeyi.

Yazılımın gelişmesine ilişkin etmenler

Bakım kolaylığı: Görülen bir hatanın nedeninin bulunması ve giderilmesi için gereken emek (10. Bölümde bu konu daha ayrıntılı olarak incelenecektir.)

Esneklik: Çalışan bir programda değişiklik yapmak için gereken emek.

Sınanabilirilik: Yazılımın istenen işlevi gerçekleştirdiğini sınamak için gereken emek

Yazılımın uyumluluğuna ilişkin etmenler

Taşınabilirlik: Yazılımın bir yazılım/donanım ortamından diğerine aktarılması için gereken emek.

Desteleyicilik: Yazılımın ya da parçalarının yeni uygulamalarda kullanılabilirliği

Uyumluluk: Başka yazılım sistemleriyle işbirliği yapabilme düzeyi.

Yazılım İsterlerinin Çözümlemesi

Yazılım Niteliği Ölçütleri

Kurallara uyum : Standartlara ve kuruluş içindeki yazılım üretimi kurallarına uyum düzeyi.

Hatasızlık : işlemlerin ve denetimin hatasızlık duyarlılığı.

İletişim rahatlığı: Standart arabirimlerin, protokollerin ve iletişim yöntemlerinin kullanılma düzeyi.

Tamlık : İsterlerde belirtilen işlevin gerçekleştirilme düzeyi.

Karmaşıklık : Yazılımın yapısal karmaşıklığı. Çağırılan yordam sayısı, koşulların ve kararların, girdi çıktı birimlerinin fazlalığı v.b. ayrıntılara bağlı olarak yazılım karmaşıklığının tanımı için bkz. [McCabe, 1976]. Önerilen en basit karmaşıklık ölçüsü, yazılımın satır sayısı olarak uzunluğudur. Burada yine (0-10) aralığında bir değerlendirme söz konusudur.

Özlülük : Birçok araştırmacı, diğer ölçütlerin, özellikle de karmaşıklığın yazılım uzunluğuna bağlı olduğu görüşündedir. (Ör. [Li, 1987]) Özlülük, gerçekleştirilen işlev sayısının yazılım uzunluğuna oranı olarak tanımlanabilir.

Tutarlılık : Yazılımın tüm birimlerinde ve geliştirme sürecinin tümünde tutarlı bir tasarım ve belgeleme yönteminin kullanılması.

Veri rahatlığı : Tüm yazılımın standart ve uyumlu veri yapıları kullanması.

Hata hoşgörüsü: Hata durumunda işlemi sürdürme kolaylığı.

Yazılım İsterlerinin Çözümlemesi

Verimli çalışma :

Yazılımın başarımı; yapılan işin kısa zamanda ve az kaynakla gerçekleştirilmesi.

Genişletilebilirlik :

Yazılım mimarisinin, veri yapılarının ve süreç tasarımının genişletilebilme yeteneği.

Genellik :

Yazılımın kullanım alanının genişliği.

Donanımdan bağımsızlık:

Yazılımın işleyebilmesi ya da belli başarımlarına uyabilmesi için donanıma ne ölçüde bağımlı olduğu.

Denetimlilik :

Yazılımın kendi işleyişini denetleme, başarımlarını ve oluşan hataları izleme yeteneği.

Birimsellik :

Yazılım birimlerinin işlevsel tutarlılık ve birbirinden bağımsızlık düzeyi.

Yazılım İsterlerinin Çözümlemesi

İşletilebilirlik:

Yazılımın çalıştırılma kolaylığı.

Güvenlik:

Programları ve verileri koruyan düzeneklerin etkililik düzeyi.

Anlaşırılık:

Yazılımın, özellikle kaynak kodunun açıklayıcı belgeleme içerme düzeyi.

Basitlik:

Yazılımın kolay anlaşılabilme düzeyi.

Yazılımdan bağımsızlık:

Yazılımın standart olmayan yazılım desteklerine, işletim sistemi, v.b. yazılım sistemlerine bağımlılık

Açıklanabilirlik:

Yazılımın yapısal ya da işlevsel özelliklerinden her birinin, hangi isterlerden kaynaklandığının açıklığı.

Eğitim:

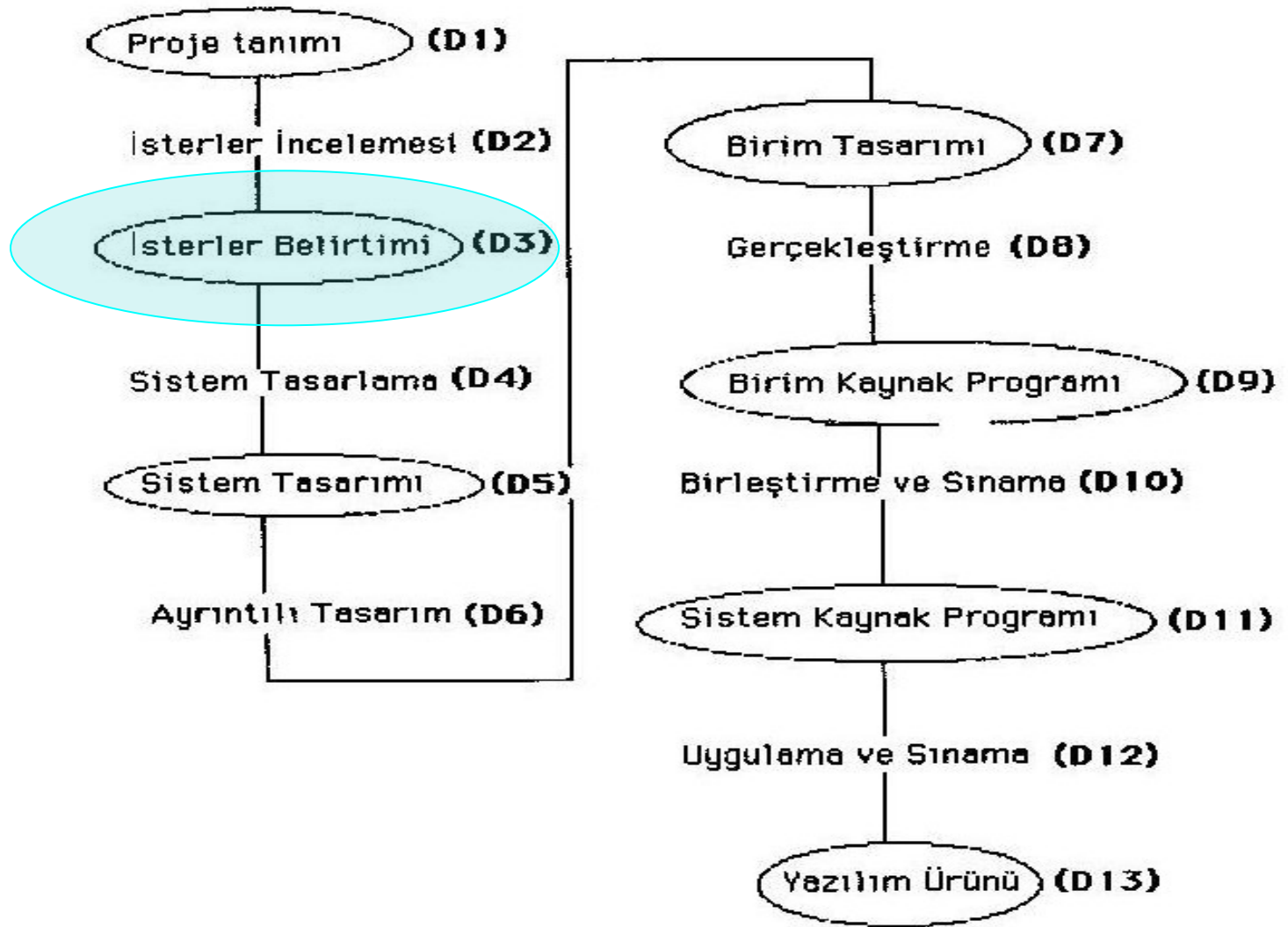
Yazılımın, yeni kullanıcılara ne ölçüde kolayca kullanım olanağı sunduğu ve eğitim gerektirdiği.

Yazılım Sürecinin Nitelik Üzerinde Etkisi

Yazılım niteliğine ilişkin olarak yapılan daha yeni çalışmalar, niteliğin, özellikle de güvenilirlik öğesinin, yalnızca son yazılım ürünü üzerinde belli bir zamanda yapılacak incelemeyle saptanamayacağı görüşüne ağırlık vermektedir.

Yalnızca son ürünün değil, yaşam çevriminin çeşitli aşamalarındaki ara ürünlerin de incelenmesi yazılım niteliğinin belirlenmesinde gerekli görülmektedir.

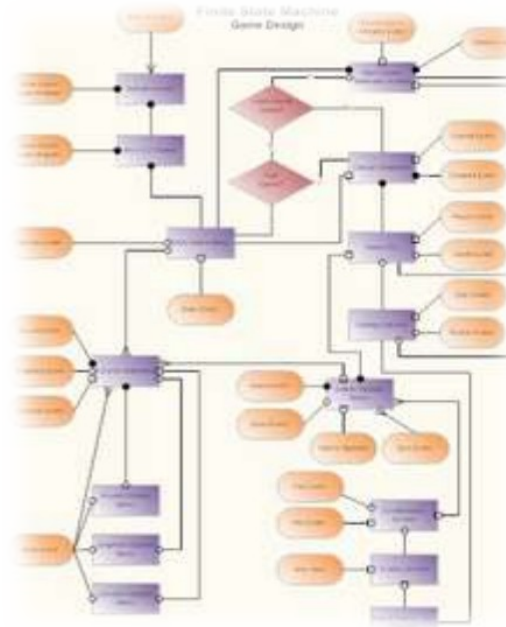
Yazılım İsterlerinin Çözümlemesi



Yazılım nitelik öğeleri dizisi.

VERİ AKIŞ DIYAGRAMI

GELİŞTİRME



Analist dört sorunun cevabını bilmek ister:

1. Sistemi hangi prosesler oluşturuyor?
2. Her bir proseste kullanılan veriler nelerdir?
3. Hangi veriler saklanıyor?
4. Sistemin hangi verileri girer ve hangi verileri çıkar?

Sistem analisti, mevcut verinin organizasyonundaki rolünü saptar.

“ Veri akış analizinin amacı olan, işletme akışı boyunca veri akışını izlemek analiste organizasyonun amacına nasıl ulaştığını belirtir. “

Tüm işlemler gerçekleşirken veri, girer, işlenir, saklanır, geri alınır, kullanılır, değiştirilir ve çıkar.

“ Veri akış analizi, her aktivitede verinin kullanımını inceler. Saptadıklarını veri oluş diyagramlarında dokümante eder. Bu diyagramlar proses ve veri arasındaki ilişkiyi grafik olarak gösterir. “

Veri sözlüğünde ise sistem verilerini ve nerede kullanıldığını açıklar.

Veri Akış Analizi Araçları

Veri Akış Diyagramı:

Sistem boyunca verinin hareketinde işlemleri, veri saklamayı ve gecikmeleri içerecek şekilde grafik olarak açıklar.

Fiziksel komponentlerden bağımsız olarak, mantıksal olarak veri akışını açıklar. Bu nedenle de “logical data flow diagram – mantıksal veri akış diyagramı” olarak adlandırılır.

Veri Sözlüğü:

Mevcut sistemin verilerinin, adları, açıklaması ve organizasyonunu belirtir.

Veri Yapısı Diyagramı :

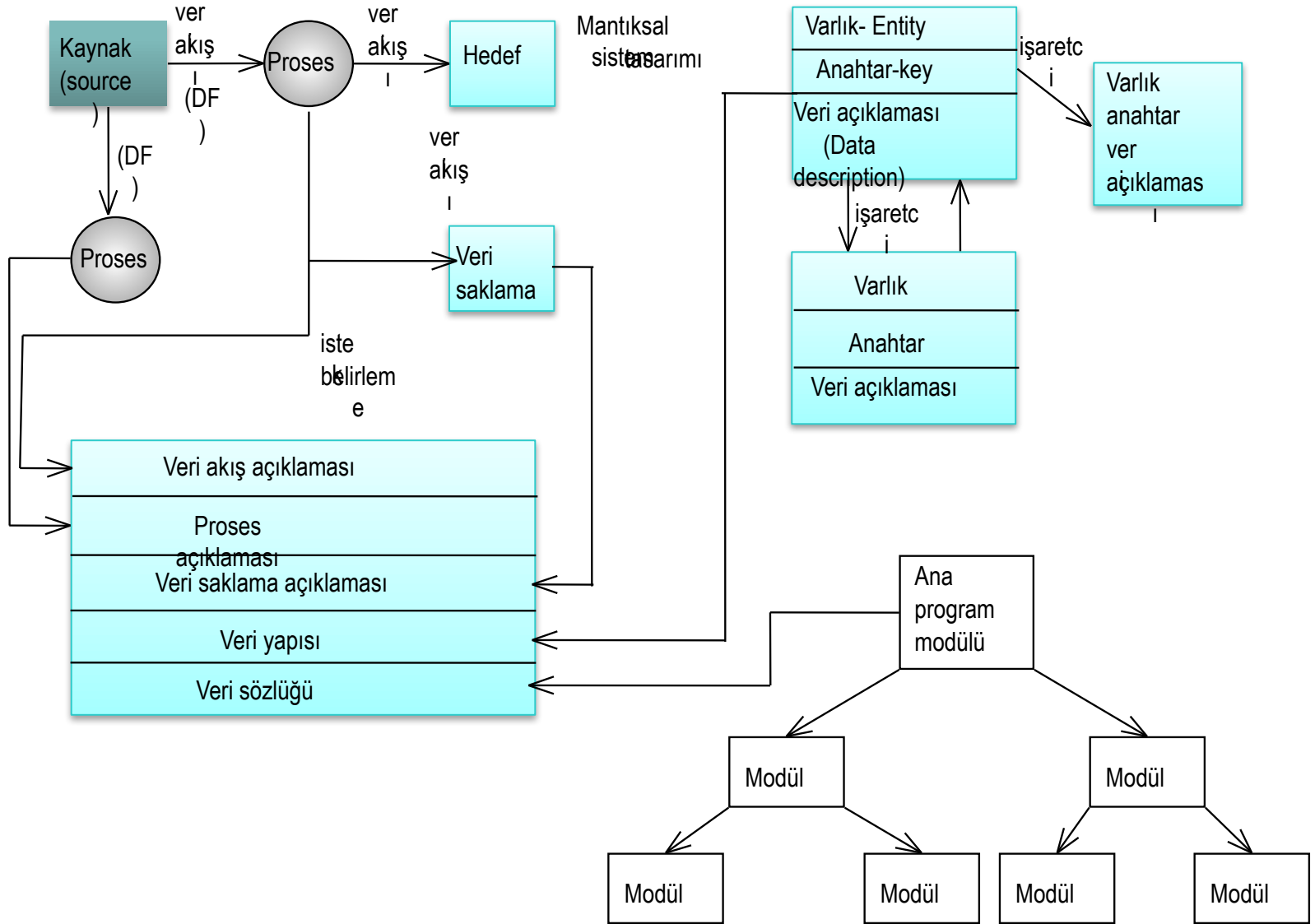
Varlıklar (insan, yer, olay ve nesneler) arasındaki ilişkilerin görsel açıklamasıdır. Varlıklar hakkında bilgi içerir.

Yapı Çizelgesi :

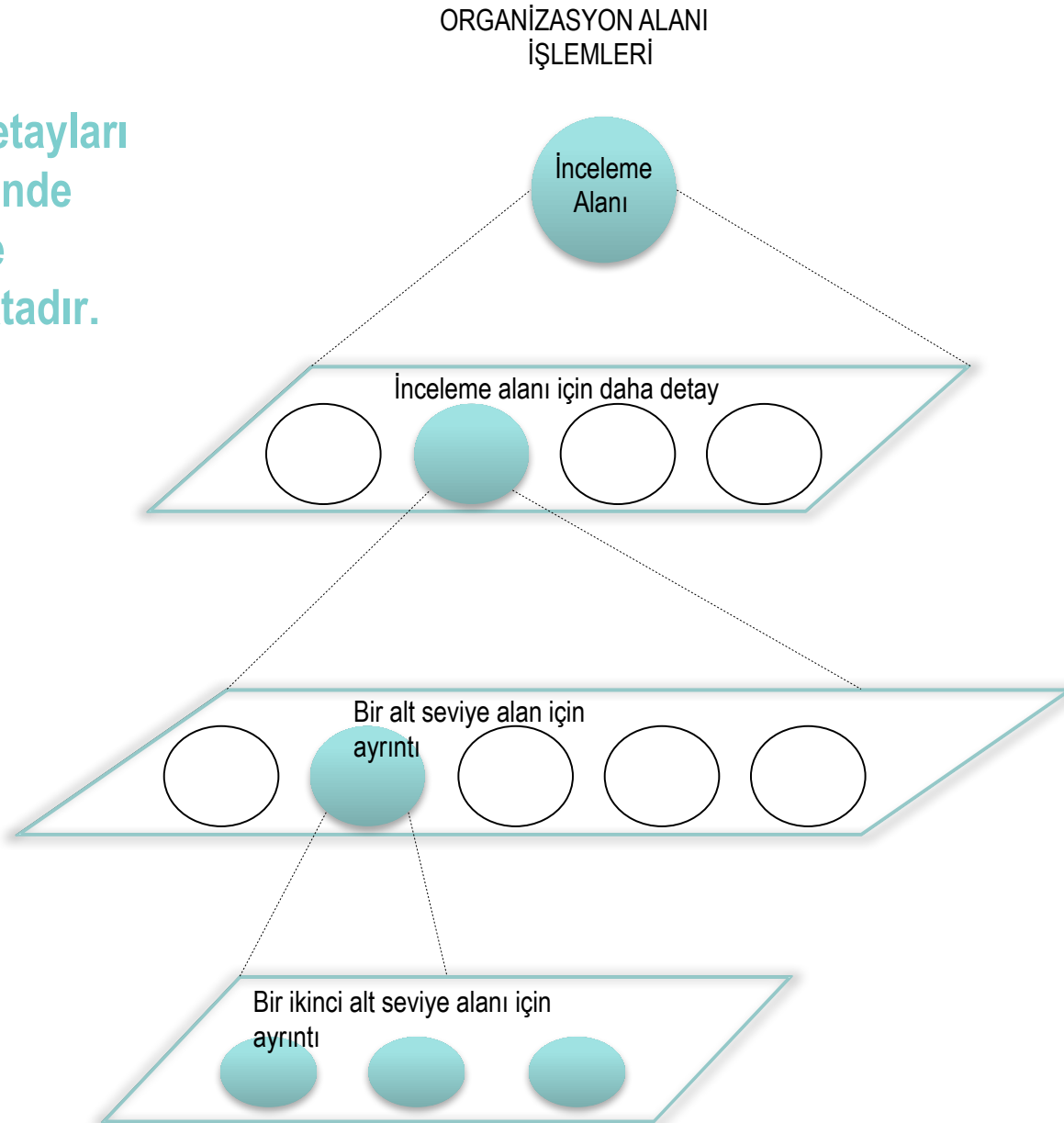
Yazılımın modüllerinin arasındaki ilişkiyi gösteren görsel bir tasarım aracıdır. Modüller arasındaki hiyerarşiyi açıklar.

Giriş – çıkış dönüşümlerini, işlemlerin analizini içerir.

Veri Yapısı Diyagramı



Sistem detayları
her seferinde
bir seviye
artırılmaktadır.



- Sistem analisti, mevcut sistemi inceler ve geçerli aktiviteleri ve prosesleri saptar.

Sistem analisti terminolojisinde bu, fiziksel sistemin (physical system) incelenmesidir.

- Fizik sistem, veri ve işlemlere odaklanan mantıksal açıklamaya dönüştürülür.



Veri akış analizi sırasında , ayrıntılar veri akışının mantıksal bileşenleri olan

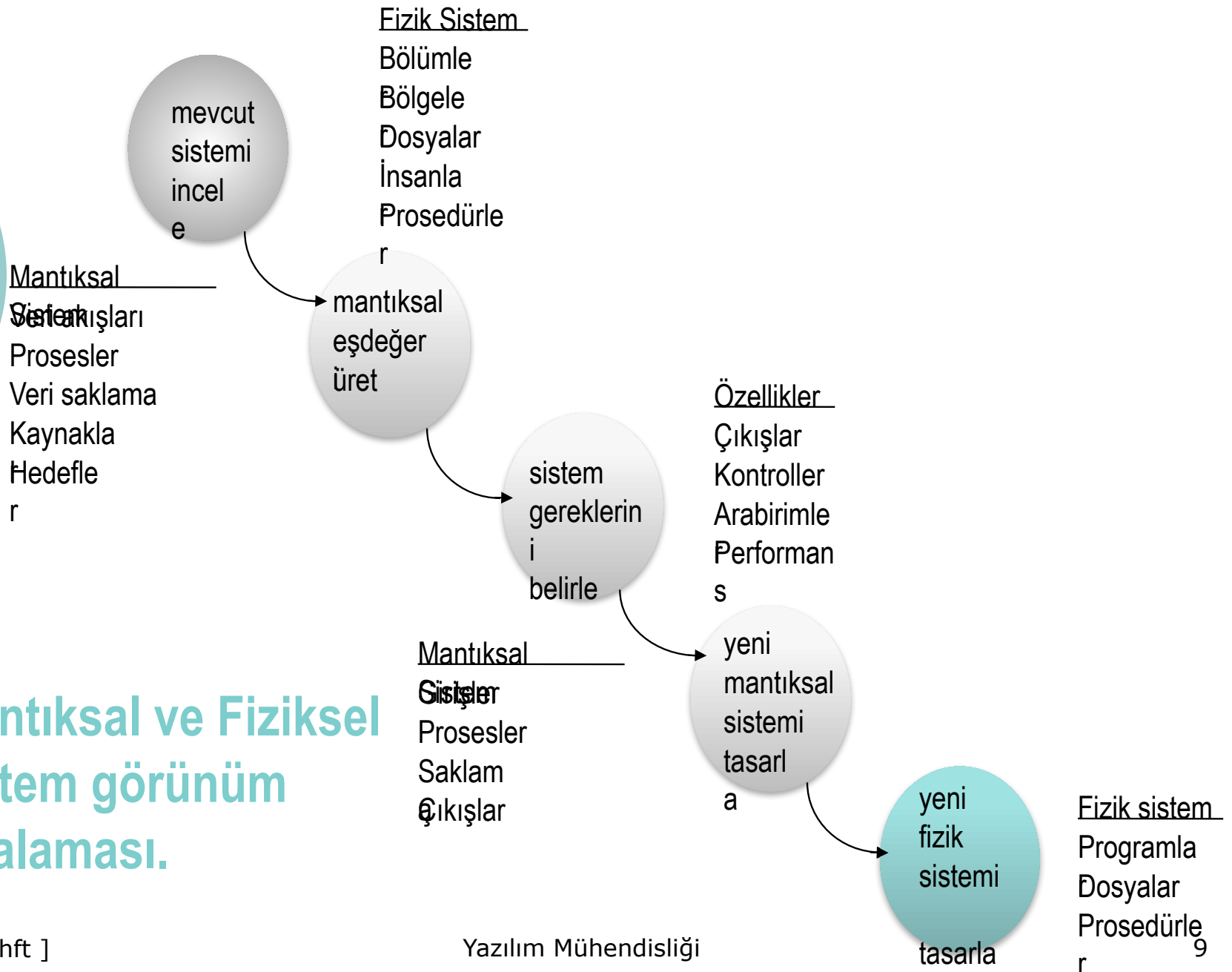
- veri akışları,
- prosesler,
- veri saklama,
- kaynaklar ,
- hedefler

anlamında değerlendirilir.

Veri akış diyagramları iki tiptir.

1. Fiziksel
2. Mantıksal

Mantıksal ve Fiziksel sistem görünüm sıralaması.



Fiziksel Veri Akış Diyagramları

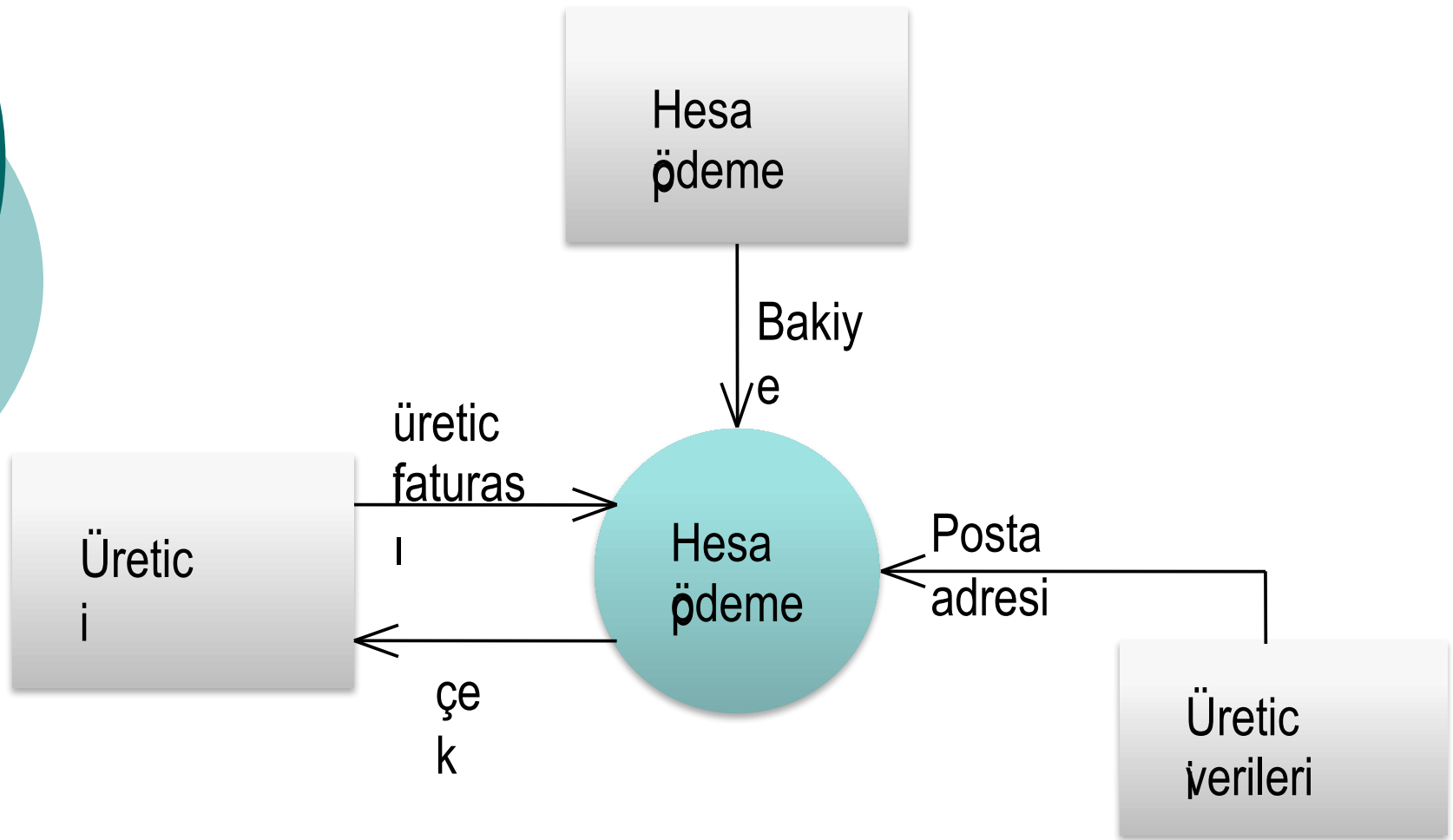
Mevcut sistemin, gerçeklemeye bağımlı bir görüntüsü olup, hangi işlerin yapıldığını, nasıl yapıldığını gösterir.

Fiziksel karakteristikler;

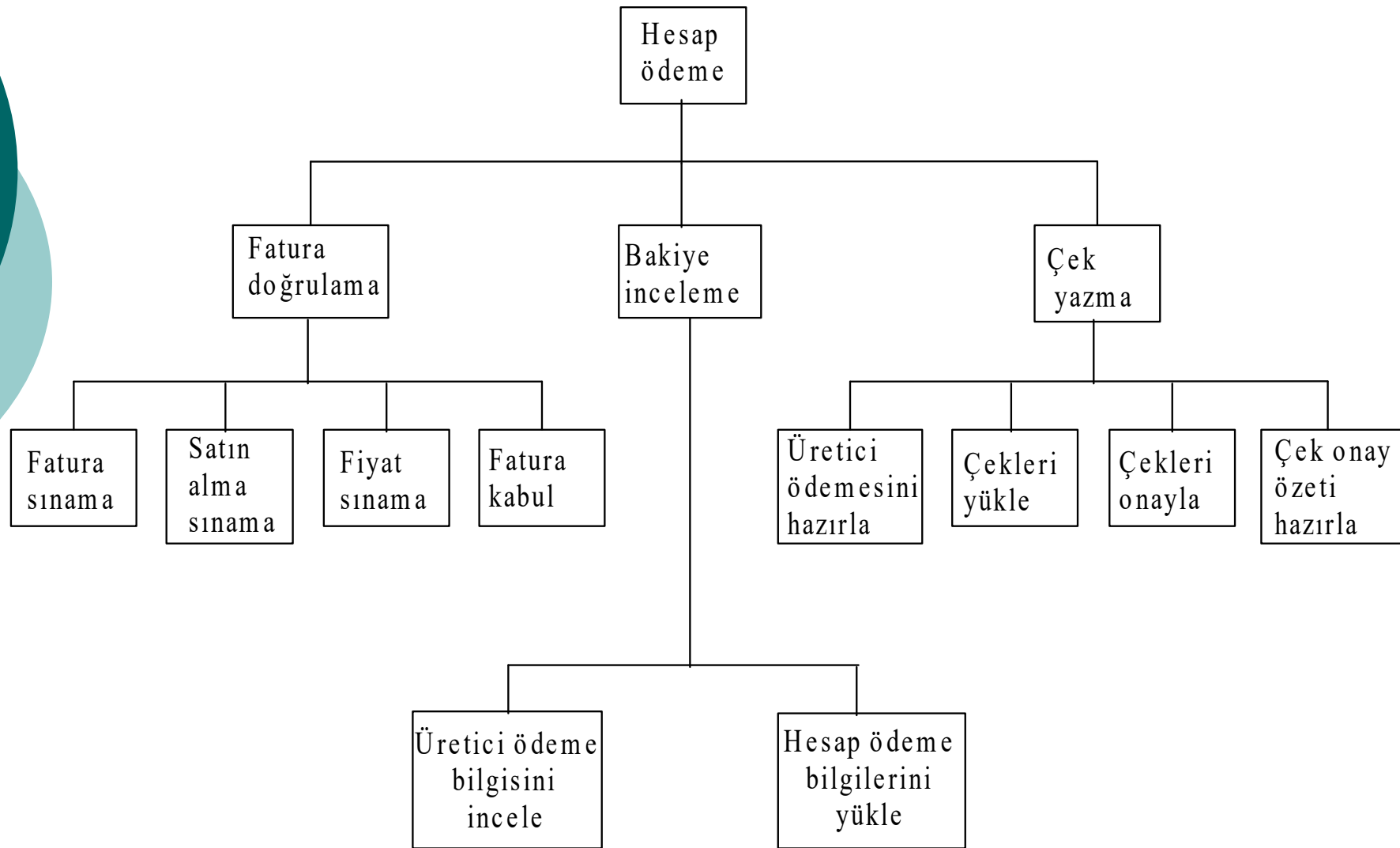
- İnsanların isimleri
- Form ve dokümanların isimleri/numaraları
- Bölüm adları
- Ana ve işlem (transaction) dosyaları
- Yerleşimler
- Prosedür isimleri

MANTIKSAL VERİ AKIŞ DİYAGRAMLARI

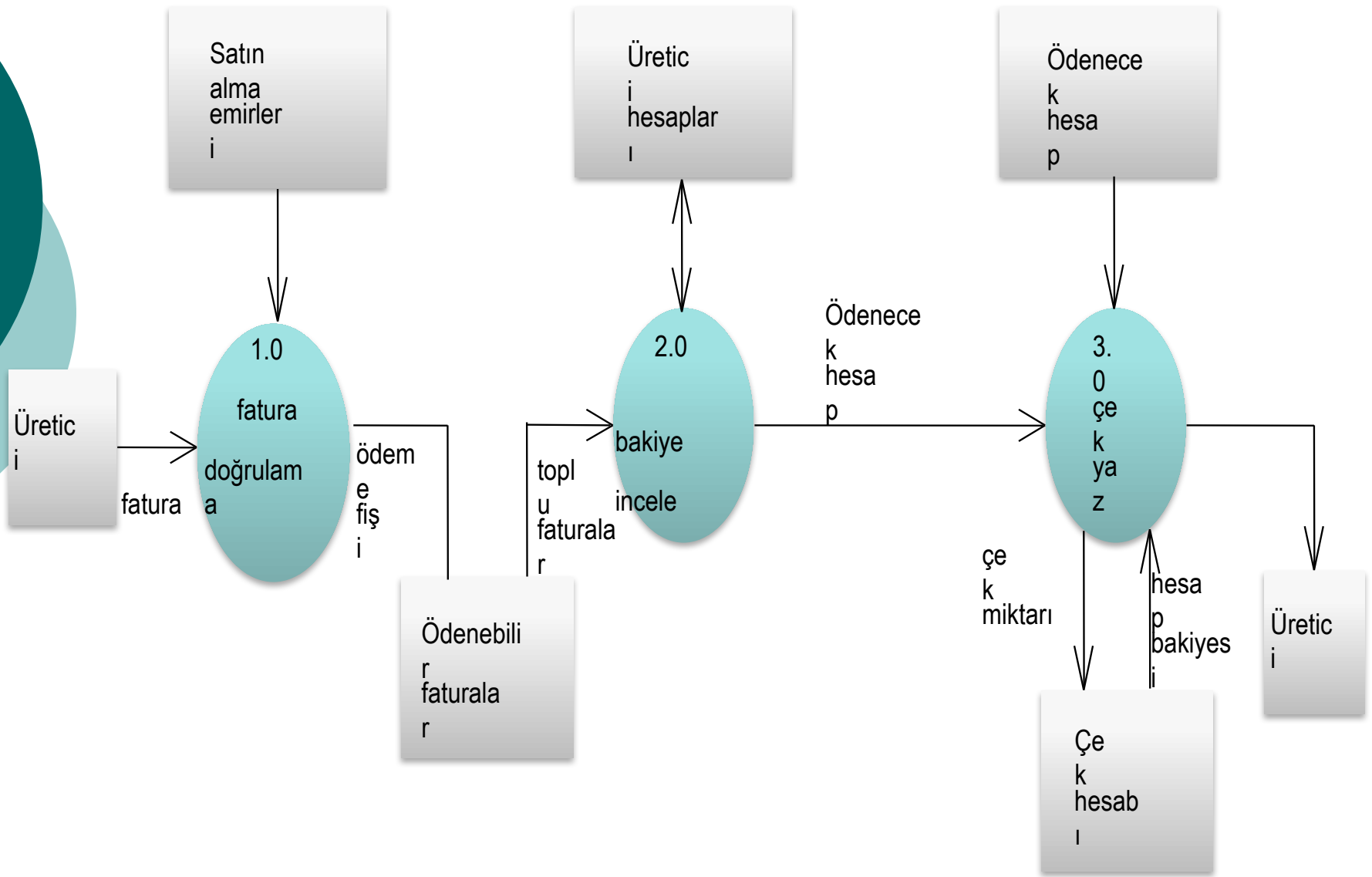
- Sistem çalışmasına fiziksel veri akış diyagramı ile başlanır.
- Sistemin fiziksel bileşenleri arasındaki etkileşim, insanların davranışları, dokümanlar ve bölümler arası bilgi akışının açıklanması için gereklidir.
- Ayrıca kullanıcılarla iletişimde fiziksel akış diyagramı kullanışlıdır.
- Sistemin o andaki gerçek çalışmasını saptamak, problemleri belirlemek için en iyi yoldur.



Hesap ödeme sistemi bağlam veri akış diyagramı

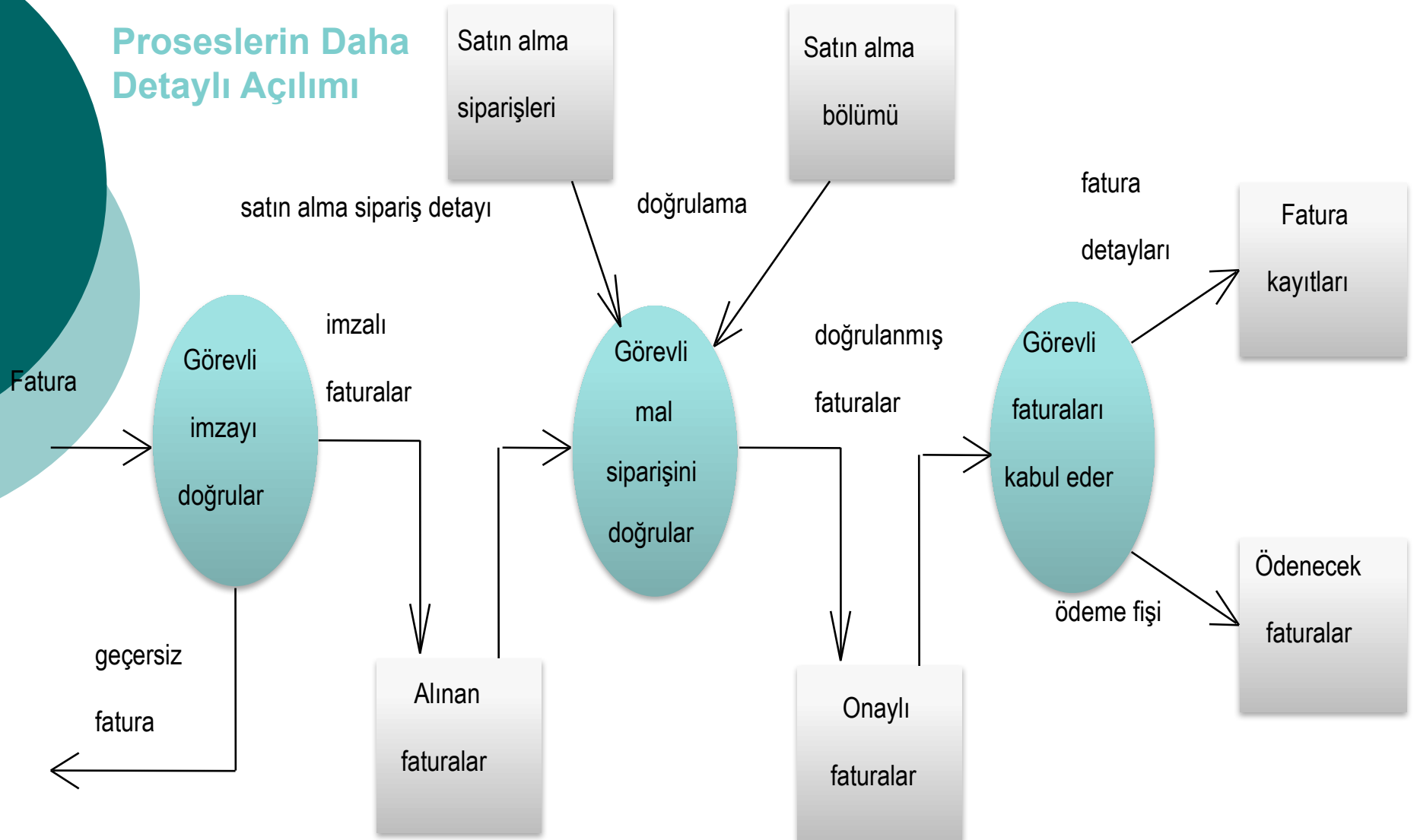


Hesap ödeme için proses hiyerarşi çizelgesi



1. seviye fiziksel veri akış diyagramı

Proseslerin Daha Detaylı Açılımı



Fatura doğrulama işleminin fiziksel veri akış diyagramı gösterilmiştir.

Genel olarak bir seviye aşağı diyagram şu özellikleri içermelidir.

- Öncelikle diyagramda prosesi açıklayan tüm veri akışları, alt seviye diyagramda yer almalıdır.
- Yeni veri akışları ve veri saklamaları prosesin içinde kullanılıyor ve iç prosesleri bağlama durumunda ise ilk defa olarak bu diyagramda tanımlanabilir.
- Prosesle kaynaklanan veri akışı ve saklamaları gösterilmelidir.
- Hiçbir varlık bir üst seviye diyagramla çelişmemelidir.
- Benzer biçimde, üretici hesabı (bakiye incele) ve çek yazma prosesleri de açılır.



- Burada karşımıza çıkan soru, kaç seviye daha gereklidir?
- Sistemin karmaşıklığına bağlı olarak herhangi bir ön değer verilemez.
- Sistemin tüm detayları cevap buluncaya kadar seviyelendirme sürecidir.

Genel olarak söylenirse, fiziksel veri akış diyagramlarında;

- Farklı insanlar yada yerler arasındaki veri akışı isteyen birçok görev içeren prosesler açılmalıdır.
- Tek bir kişi yada masa tarafından yapılan ancak veri akışı ve doküman olmayan çoklu işlerin açılmasına gerek yoktur.

Mantıksal Görünüm Türetilmesi

- Mantıksal veri akış diyagramı, fiziksel olanından bunlar yapılarak türetilir.
- Proseste gereken gerçek veriyi göster, onları içeren dokümanları değil.
- Yönlendirme bilgisini yani insanlar, ofisler yada yerler arasındaki akışları kaldır, prosesler arasındaki akışı göster.
- Araç ve cihazları kaldır (dosya dolapları, kutular v.s.).
- Kontrol bilgisini kaldır.
- Fazlalık veri saklamalarını düzenle.
- Veriyi yada veri akışını değiştirmeyen gereksiz prosesleri kaldır. (Örneğin yönlendirme, saklama, kopyalama)

Mantıksal Veri Akış Diyagramı Çiziminde Genel Kurallar

- Bir prosten çıkan herhangi veri akışı, proses girişindeki veriye dayanmalıdır.
- Tüm veri akışları adlandırılmalıdır. İsimler, prosesler, veri saklamaları, kaynaklar ve hedefler arasında akan veriyi yansıtmalıdır.
- Sadece prosesin çalışması için gereken veri, proses girişi olmalıdır.
- Bir proses sadece kendi giriş ve çıkışlarına bağlı olmalı, sistemdeki diğer proseslere bağlı olmamalıdır.
- Prosesler her zaman çalışmaktadırlar, durup çalışmazlar. Sistem her zaman dinamiktir.

Mantıksal Veri Akış Diyagramı Çiziminde Genel Kurallar

- Proseslerin çıkışları şu biçimlerden birini alabilir;
 - a) Proses tarafından eklenmiş bilgi ile birlikte giriş veri çıkışı
 - b) Veri biçiminin değişimi yada cevabı (örneğin kar miktarını, yüzdeli olarak değiştirme gibi)
 - c) Durum değiştirme (onaysız durumdan – onaylı duruma)
 - d) İçerik değiştirme, bir yada daha çok gelen giriş akışından bilgiyi birleştirme yada ayırma
 - e) Organizasyonda değişiklik (fiziksel ayırım yada verinin yeniden düzenlenmesi gibi)



Veri akış diyagramını değerlendirirken şu sorular kullanışlı olacaktır:

- Veri akış diyagramında adlandırılmamış component (giriş, çıkış, veri akışı, proses, saklama) var mı?
- Giriş olan ama refere edilmemiş veri saklama var mı?
- Giriş almamış proses var mı?
- Çıkış üretmeyen proses var mı?
- Birçok amaca hizmet eden proses var mı? (Eğer öyle ise bu prosesin, birçok alt prosese bir alt seviye diyagramda açılması uygundur)
- Hiç refere edilmemiş veri saklama var mı?



Veri akış diyagramını değerlendirirken şu sorular kullanışlı olacaktır:

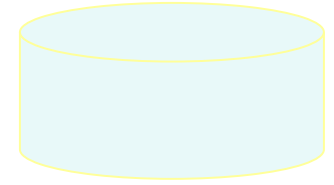
- Prosesi gerçeklemek için giriş veri akışı yeterlimi?
- Veri saklamada, fazla veri, gereksiz detay var mı?
- Prosesin girişine gelen veri, çıkışta üretilecek verinin üretilmesi için fazla mı? Lüzumsuz, fazlalık giriş var mı?
- Sistem açıklamasında (aliases) ek isimlendirmeler var mı?
- Veri sözlüğünde ele alınmışlar mı?
- Her bir proses, diğer proseslerden bağımsız mı? Sadece kendi girişinde aldığı veriye mi bağlı?



VERİ SÖZLÜĞÜ (Data Dictionary)

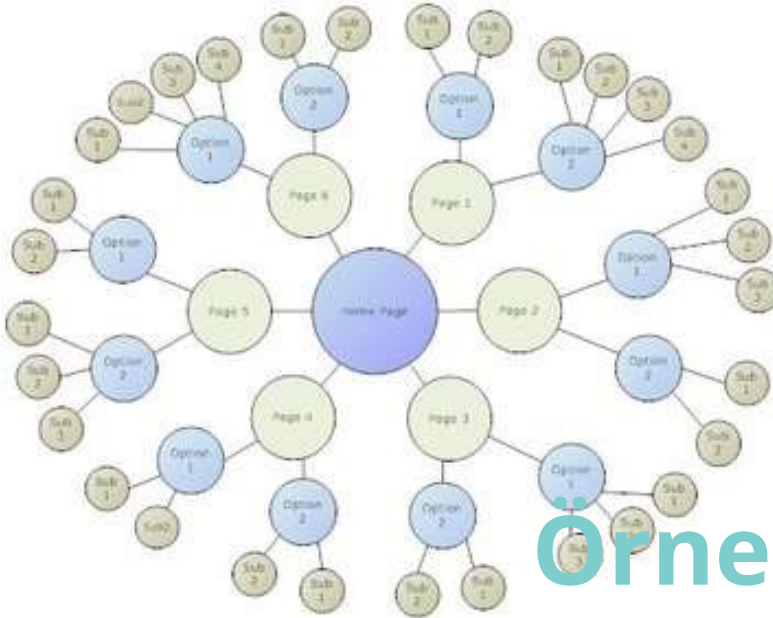
Kullanılma nedenleri olarak şunlar söylenebilir;

- Büyük sistemlerde ayrıntılarla başa çıkmak için,
- Tüm sistem elemanları için bir ortak anlamı sağlamak,
- Sistem özelliklerini dökümante etmek,
- Sistem değişiklikleri yapılması gerektiğinde karakteristiklikleri değerlendirmek ve belirlemek, ayrıntıların analizini kolaylaştırmak,
- Hataları ve unutulmaları belirlemek için.
- Bildiğimiz gibi bir bilgi sisteminin işlemleri, sorguları, raporları, çıkışları, dosyaları ve veri tabanları gibi tüm parçaları veriye bağlıdır. Sözlük, sistem üzerinde akan veri için iki tür açıklama içerir: veri elemanları ve veri yapıları. Veri elemanları, bir veri yapısı oluşturmak üzere gruplaşırlar.



VERİ UZUNLUĞU (length)

- Analiz sırasında saptanacak bir büyüklük olup, sistem tasarımı sırasında önem kazanır.
- Bazı proseslerde, sadece belirli veri değerlerine izin verilir. Veri değeri için, organizasyonda belli sınırlandırmalar yada tanımlar getirilmişse bunlar veri sözlüğünde de yer alır.



Örnek Uygulama

...

... Sistem Tasarımı

YAZILIM GELİŞTİRME METODOLOJİSİ VE YAŞAM DÖNGÜSÜ

METODOLOJILER



- **Metodoloji:** Bir BT projesi ya da yazılım yaşam döngüsü aşamaları boyunca kullanılacak ve birbirleriyle uyumlu yöntemler bütünü.
- Bir metodoloji,
 - bir süreç modelini ve
 - belirli sayıda belirtim yöntemini içerir
- Günümüzdeki metodolojiler genelde **Çağlayan** ya da **Helezonik model**i temel almaktadır

BİR METODOLOJIDE BULUNMASI GEREKEN TEMEL BİLEŞENLER (ÖZELLİKLER)

- Ayrıntılandırılmış bir süreç modeli
- Ayrıntılı süreç tanımları
- İyi tanımlı üretim yöntemleri
- Süreçlerarası arayüz tanımları
- Ayrıntılı girdi tanımları
- Ayrıntılı çıktı tanımları
- Proje yönetim modeli
- Konfigürasyon yönetim modeli
- Maliyet yönetim modeli
- Kalite yönetim modeli
- Risk yönetim modeli
- Değişiklik yönetim modeli
- Kullanıcı arayüz ve ilişki modeli
- Standartlar

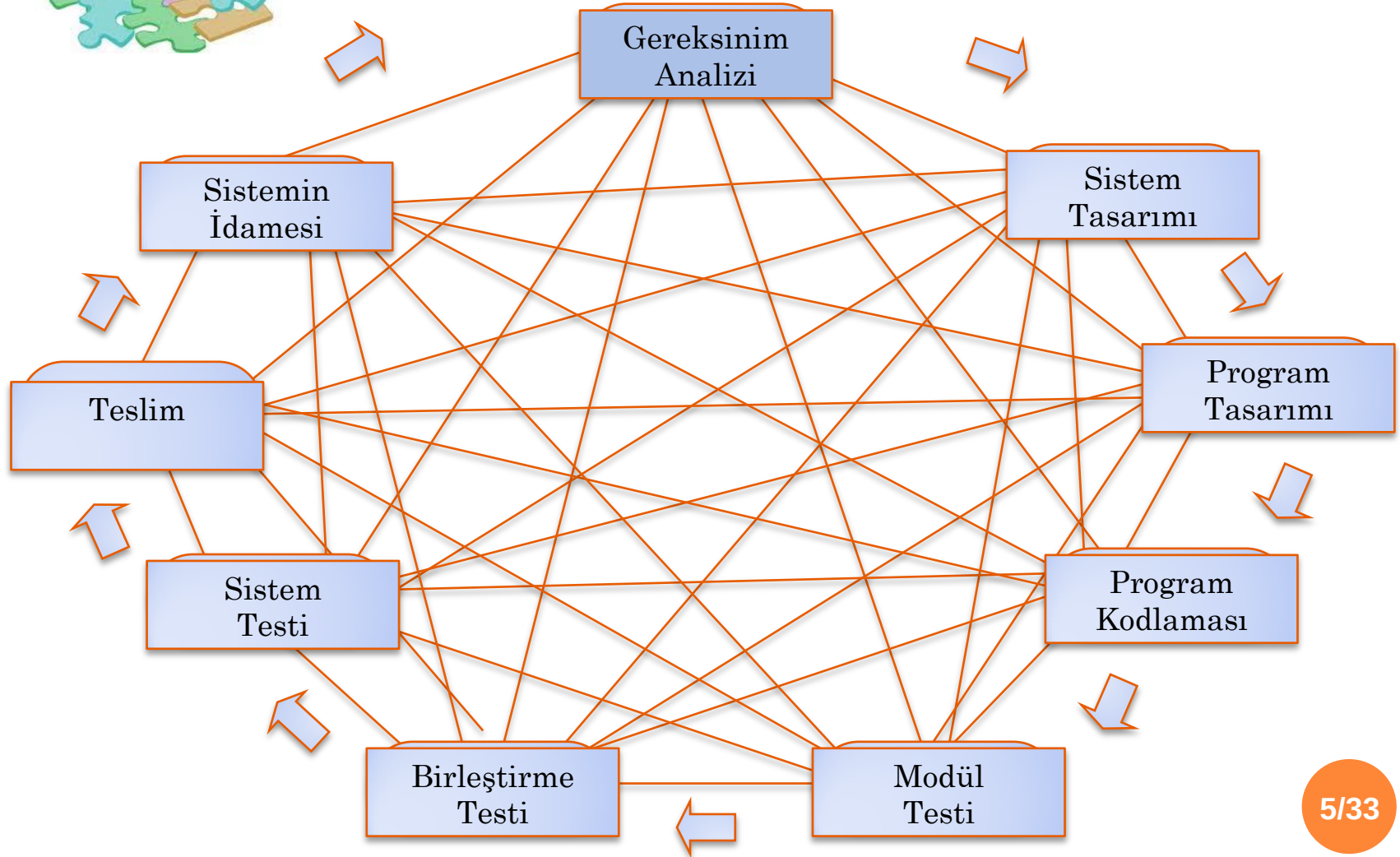
BİR METODOLOJIDE BULUNMASI GEREKEN

TEMEL BİLEŞENLER

- Metodoloji bileşenleri ile ilgili olarak bağımsız kuruluş (IEEE, ISO, vs.) ve kişiler tarafından geliştirilmiş çeşitli standartlar ve rehberler mevcuttur.
- Kullanılan süreç modelleri ve belirtim yöntemleri zaman içinde değiştiği için standart ve rehberler de sürekli güncellenmektedir.
- Bir kuruluşun kendi metodolojisini geliştirmesi oldukça kapsamlı, zaman alıcı ve uzmanlık gerektiren bir faaliyet olup, istatistikler yaklaşık 50 kişi/ay'lık bir iş gücü gerektirdiğini göstermektedir.



GERÇEK HAYATTA PROGRAM GELİŞTİRME



YAZILIM GELİŞTİRME YAŞAM DÖNGÜSÜ

- Yazılımın hem üretim, hem de kullanım süreci boyunca geçirdiği tüm aşamalar **yazılım geliştirme yaşam döngüsü** olarak tanımlanır.
- Yazılım işlevleri ile ilgili gereksinimler sürekli olarak değiştiği ve genişlediği için, söz konusu aşamalar sürekli bir döngü biçiminde ele alınır.
- Döngü içerisinde her hangi bir aşamada geriye dönmek ve tekrar ilerlemek söz konusudur.
- Yazılım yaşam döngüsü tek yönlü ve doğrusal değildir.

YAZILIM YAŞAM DÖNGÜSÜ TEMEL ADIMLARI

1. Analiz

Sistem gereksinimlerinin ve işlevlerinin ayrıntılı olarak çıkarıldığı aşama. Var olan işler incelenir, temel sorunlar ortaya çıkarılır.

2. Planlama

Personel ve donanım gereksinimlerinin çıkarıldığı, fizibilite çalışmasının yapıldığı ve proje planının oluşturulduğu aşamadır.

3. Tasarım

Belirlenen gereksinimlere yanıt verecek yazılım sisteminin temel yapısının oluşturulduğu aşamadır.

mantıksal; önerilen sistemin yapısı anlatılır,

fiziksel; yazılımı içeren bileşenler ve bunların ayrıntıları.

4. Gerçekleştirim

Kodlama, test etme ve kurulum çalışmalarının yapıldığı aşamadır.

5. Bakım

Hata giderme ve yeni eklentiler yapma aşaması (teslimden sonra).

YAZILIM YAŞAM DÖNGÜSÜ TEMEL ADIMLARI

- Yaşam döngüsünün temel adımları **çekirdek süreçler (core processes)** olarak da adlandırılır.
- Bu süreçlerin gerçekleştirilmesi amacıyla
 - **Belirtim (specification) yöntemleri** - bir çekirdek sürece ilişkin fonksiyonları yerine getirmek amacıyla kullanılan yöntemler
 - **Süreç (process) modelleri** - yazılım yaşam döngüsünde belirtilen süreçlerin geliştirme aşamasında, hangi düzen ya da sırada, nasıl uygulanacağını tanımlayan modeller kullanılır.

BELİRTİM YÖNTEMLERİ

○ Süreç Akışı İçin Kullanılan Belirtim Yöntemleri

Süreçler arası ilişkilerin ve iletişimin gösterildiği yöntemler
(Veri Akış Şemaları, Yapısal Şemalar, Nesne/Sınıf Şemaları).

○ Süreç Tanımlama Yöntemleri

Süreçlerin iç işleyişini göstermek için kullanılan yöntemler
(Düz Metin, Algoritma, Karar Tabloları, Karar Ağaçları, Anlatım Dili).

○ Veri Tanımlama Yöntemleri

Süreçler tarafından kullanılan verilerin tanımlanması için kullanılan yöntemler.

(Nesne İlişki Modeli, Veri Tabanı Tabloları, Veri Sözlüğü).

YAZILIM SÜRECİ MODELLERİ

- Yazılım üretim işinin genel yapılma düzenine ilişkin rehberlerdir.
 - Süreçlere ilişkin ayrıntılarla ya da süreçler arası ilişkilerle ilgilenmezler.
1. Gelişigüzel Model
 2. Barok Modeli
 3. Çağlayan (Şelale) Modeli
 4. V Modeli
 5. Helezonik (Spiral) Model
 6. Evrimsel Model
 7. Artırımsal Model
 8. Araştırma Tabanlı Model

GELİŞİGÜZEL MODEL

- Herhangi bir model ya da yöntem yok.
- Geliştiren kişiye bağımlı (belli bir süre sonra o kişi bile sistemi anlayamaz ve geliştirme güçlüğü yaşar).
- İzlenebilirliği ve bakımı oldukça zor.
- 60'lı yıllarda.
- Genellikle tek kişilik üretim ortamı.
- Basit programlama.

BAROK MODELİ

- İnceleme
- Analiz
- Tasarım
- Kodlama
- Modül Testleri
- Alt sistem Testleri
- Sistem Testi
- Belgeleme
- Kurulum

Yaşam döngüsü temel adımlarının doğrusal bir şekilde geliştirildiği model.

70'li yıllar.

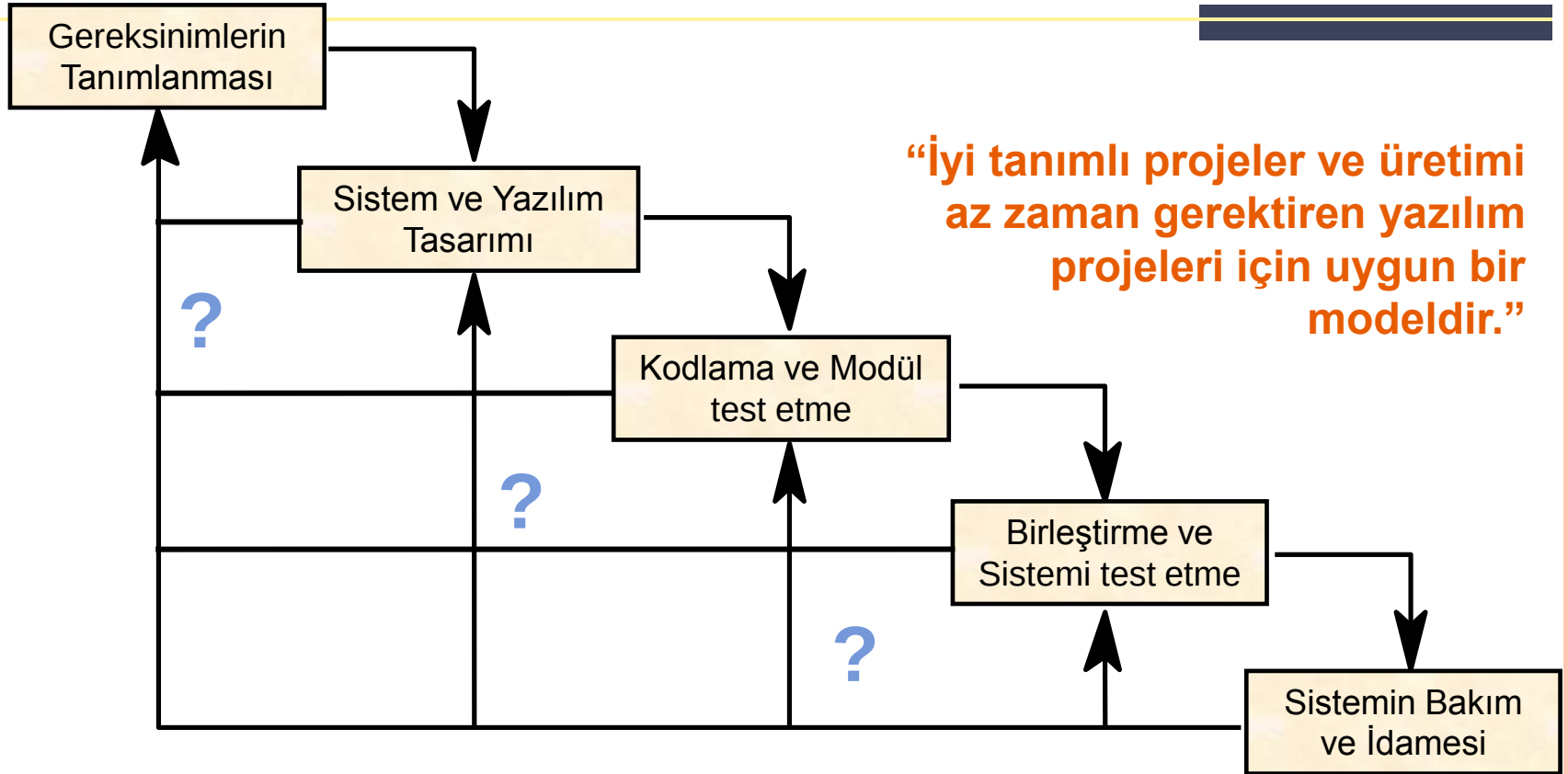
Belgelemeyi ayrı bir süreç olarak ele alır, ve yazılımın geliştirilmesi ve testinden hemen sonra yapılmasının öngörür.

Halbuki, günümüzde belgeleme yapılan işin doğal bir ürünü olarak görülmektedir.

Aşamalar arası geri dönüşlerin nasıl yapılacağı tanımlı değil.

“ Gerçekleştirim aşamasına daha fazla ağırlık veren bir model olup, günümüzde kullanımı önerilmemektedir. ”

ÇAĞLAYAN (ŞELELE) MODELİ



“Geleneksel model olarak da bilinen bu modelin kullanımı günümüzde giderek azalmaktadır.”

“Yazılım yaşam döngüsü adımları baştan sona en az bir kez izlenir.”

ÇAĞLAYAN (ŞELEALE) MODELİ

- Barok modelin aksine **belgeleme** işlevini ayrı bir aşama olarak ele almaz ve üretimin doğal bir parçası olarak görür.
- Barok modele göre geri dönüşler iyi tanımlanmıştır.
- Yazılım tanımlamada belirsizlik yok (ya da az) ise ve yazılım üretimi çok zaman almayacak ise uygun bir süreç modelidir.

ÇAĞLAYAN (ŞELELE) MODELİ

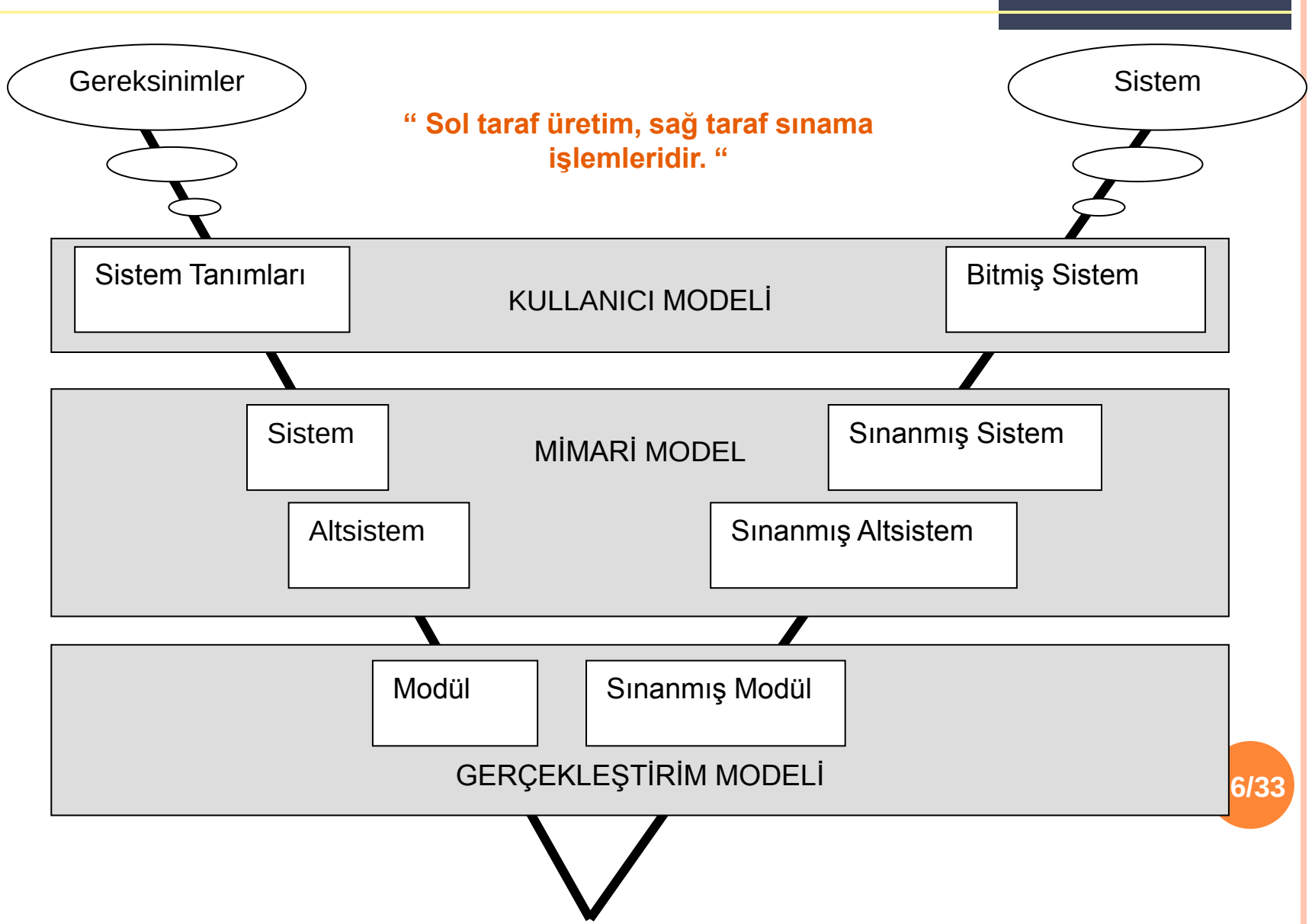
SORUNLARI



- Gerçek yaşamdaki projeler genelde yineleme gerektirir.
- Genelde yazılımın kullanıcıya ulaşma zamanı uzundur.
- Gereksinim tanımlamaları çoğu kez net bir şekilde yapılamadığından dolayı, yanlışların düzeltilme ve eksiklerin giderilme maliyetleri yüksektir.
- Yazılım üretim ekipleri bir an önce program yazma, çalıştırma ve sonucu görme eğiliminde olduklarından, bu model ile yapılan üretimlerde ekip mutsuzlaşmakta ve kod yazma dışında kalan (ve iş yükünün %80'ini içeren) kesime önem vermemektedirler.
- Üst düzey yönetimlerin ürünü görme süresinin uzun oluşu, projenin bitmeyeceği ve sürekli gider merkezi haline geldiği düşüncesini yaygınlaştırmaktadır.

V SÜREÇ MODELİ

“Belirsizliklerin az iş tanımlarının belirgin olduğu bilişim teknolojileri projeleri için uygun bir modeldir. “



V SÜREÇ MODELİ

V süreç modelinin temel çıktıları;

- **Kullanıcı Modeli**

Geliştirme sürecinin kullanıcı ile olan ilişkileri tanımlanmakta ve sistemin nasıl kabul edileceğine ilişkin sınama belirtilimleri ve planları ortaya çıkarılmaktadır.

- **Mimari Model**

Sistem tasarımı ve oluşacak alt sistem ile tüm sistemin sınama işlemlerine ilişkin işlevler.

- **Gerçekleştirim Modeli**

Yazılım modüllerinin kodlanması ve sınanmasına ilişkin fonksiyonlar.

Model, kullanıcının projeye katkısını arttırmaktadır.

HELEZONİK(SPIRAL) MODELİ

Planlama

Amaca, Alternatiflere ve Sınırlamalara karar verme

Risk Analizi

Alternatifleri değerlendirme ve risk analizi

Yazılım Mühendisliği

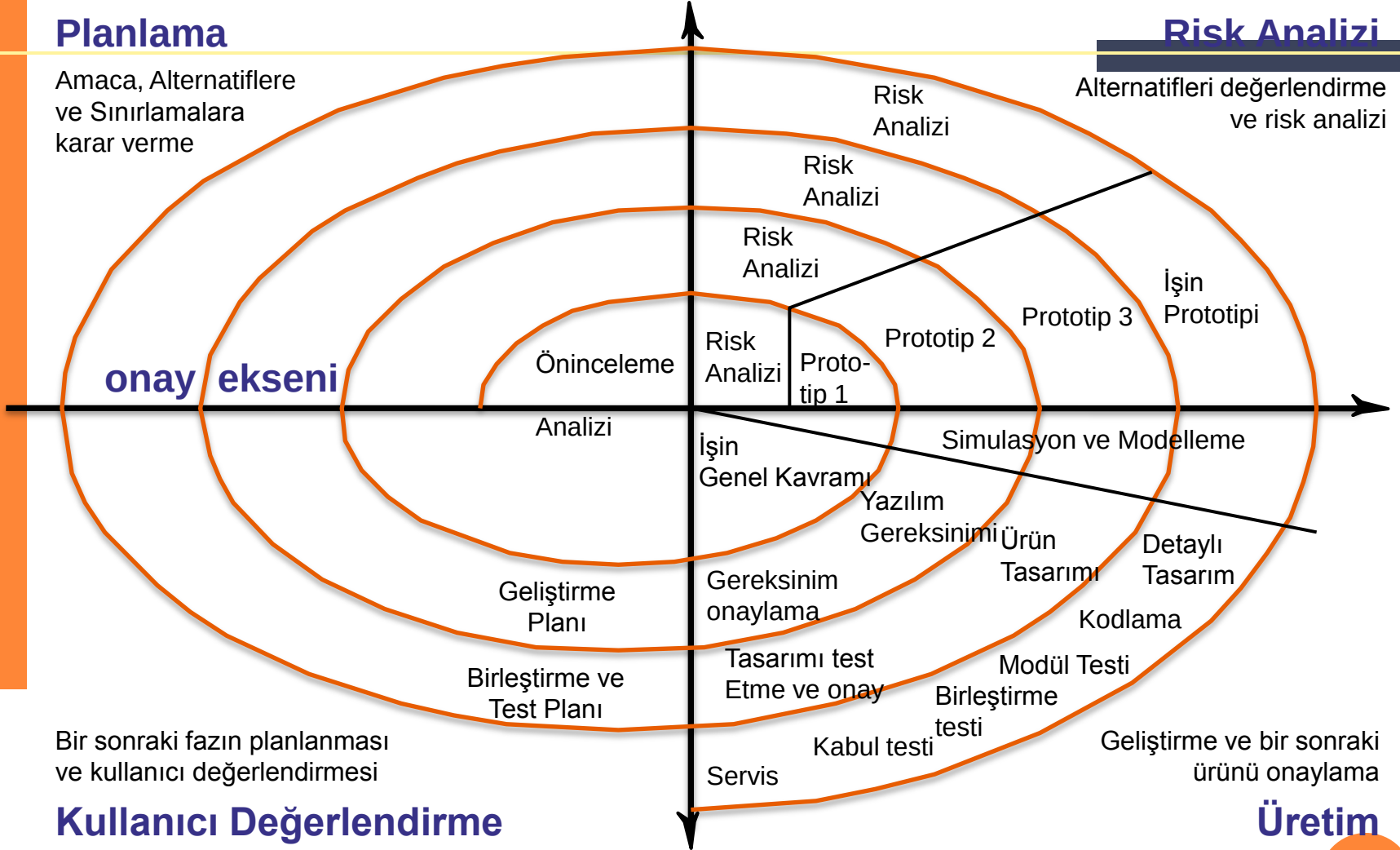
onay eksenini

Bir sonraki fazın planlanması ve kullanıcı değerlendirmesi

Kullanıcı Değerlendirme

Üretim

18/33



HELEZONIK MODEL

1. Planlama

Üretilcek ara ürün için planlama, amaç belirleme, bir önceki adımda üretilen ara ürün ile bütünleştirme

2. Risk Analizi

Risk seçeneklerinin araştırılması ve risklerin belirlenmesi

3. Üretim

Ara ürünün üretilmesi

4. Kullanıcı Değerlendirmesi

Ara ürün ile ilgili olarak kullanıcı tarafından yapılan sınama ve değerlendirmeler

HELEZONİK MODELİN AVANTAJLARI

1. Kullanıcı Katkısı

Üretim süreci boyunca ara ürün üretme ve üretilen ara ürünün kullanıcı tarafından sınanması temeline dayanır.

Yazılımı kullanacak personelin sürece erken katılması ileride oluşabilecek istenmeyen durumları engeller.

2. Yönetici Bakışı

Gerek proje sahibi, gerekse yüklenici tarafındaki yöneticiler, çalışan yazılımlarla proje boyunca karşılaştıkları için daha kolay izleme ve hak ediş planlaması yapılır.

3. Yazılım Geliştirici (Mühendis) Bakışı

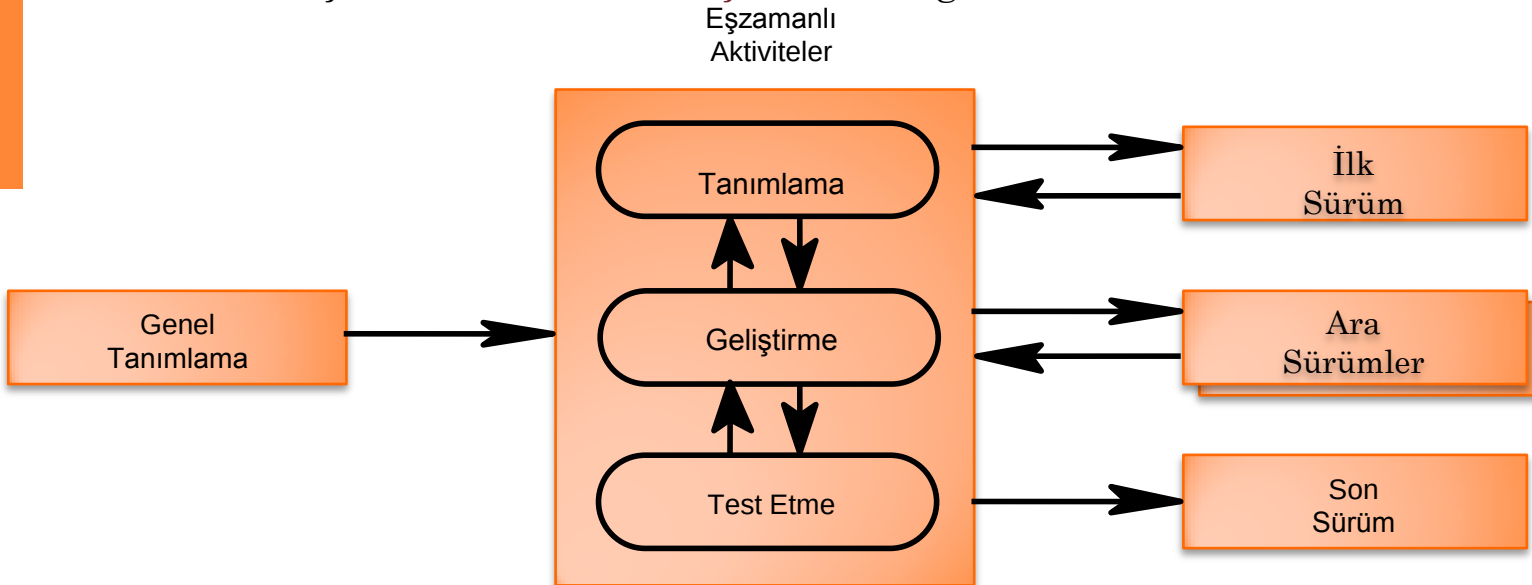
Yazılımın kodlanması ve sınanması daha erken başlar.

HELEZONIK MODEL

- Risk Analizi Olgusu ön plana çıkmıştır.
- Her döngü bir fazı ifade eder. Doğrudan tanımlama, tasarım,... vs gibi bir faz yoktur.
- Yinelemeli artımsal bir yaklaşım vardır.
- Prototip yaklaşımı vardır.

EVİRİMSEL GELİŞTİRME SÜREÇ MODELİ

- İlk tam ölçekli modeldir.
- Coğrafik olarak geniş alana yayılmış, çok birimli organizasyonlar için önerilmektedir (**banka uygulamaları**).
- Her aşamada üretilen ürünler, üretildikleri alan için tam işlevselliği içermektedirler.
- **Pilot uygulama** kullan, test et, güncelle diğer birimlere taşı.
- Modelin başarısı **ilk evrimin başarısına** bağlıdır.



Evrimsel Geliştirme Süreç Modeli

- Banka ve şubeleri olan işletme uygulamaları.
- Önce sistem geliştirilir ve adım adım dağıtılır.
- Her adımda geri besleme ile düzeltmeler yapılır.

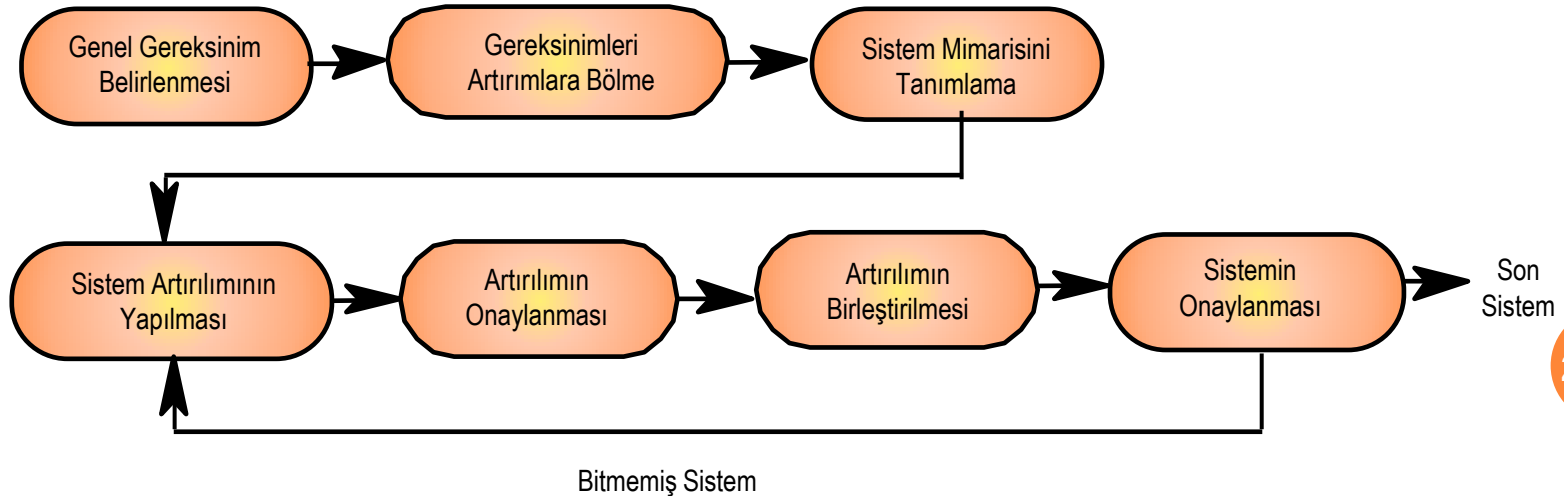
Sorunlar :

- Değişiklik denetimi
- Konfigürasyon Yönetimidir
 - Sürüm Yönetimi
 - Değişiklik Yönetimi
 - Kalite Yönetimi



ARTIRIMSAL GELİŞTİRME SÜREÇ MODELİ

- Üretilen **her yazılım sürümü birbirini kapsayacak** ve giderek artan sayıda işlev içerecek şekilde geliştirilir.
- Öğrencilerin bir dönem boyunca geliştirmeleri gereken bir programlama ödevinin 2 haftada bir gelişiminin izlenmesi (bitirme tezleri).
- Uzun zaman alabilecek ve sistemin eksik işlevlikle çalışabileceği türdeki projeler bu modele uygun olabilir.
- Bir taraftan kullanım, diğer taraftan üretim yapılır.



ARAŞTIRMA TABANLI SÜREÇ MODELİ

- Araştırma ortamları **bütünüyle belirsizlik** üzerine çalışan ortamlardır.
- Yap-at **prototipi** olarak ta bilinir.
- Yapılan işlerden edinilecek sonuçlar belirgin değildir.
- Geliştirilen **yazılımlar genellikle sınırlı sayıda kullanılır** ve kullanım bittikten sonra işe yaramaz hale gelir ve atılır.
- Model-zaman-fiyat kestirimi olmadığı için **sabit fiyat sözleşmelerinde uygun değildir.**



- En Hızlı Çalışan asal sayı test programı!
- En Büyük asal sayıyı bulma programı!
- Satranç programı!
- ...

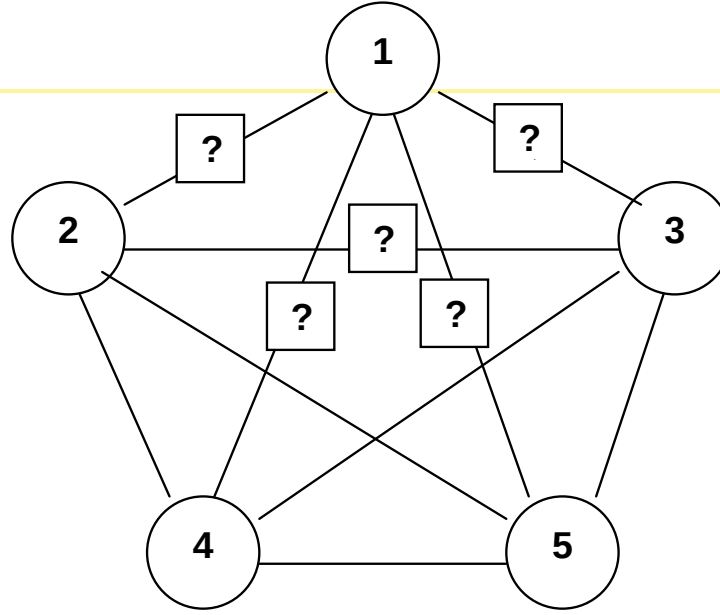
YOURDON YAPISAL SİSTEM TASARIM METODOLOJISI

Kolay uygulanabilir bir model olup, günümüzde oldukça yaygın olarak kullanılmaktadır.

Çağlayan modelini temel almaktadır.

Bir çok CASE aracı tarafından doğrudan desteklenmektedir.

Aşama	Kullanılan Yöntem ve Araçlar	Ne için Kullanıldığı	Çıktı
Planlama	Veri Akış Şemaları, Süreç Belirtilimleri, Görüşme, Maliyet Kestirim Yöntemi, Proje Yönetim Araçları	Süreç İnceleme Kaynak Kestirimi Proje Yönetimi	Proje Planı
Analiz	Veri Akış Şemaları, Süreç Belirtilimleri, Görüşme, Nesne ilişki şemaları Veri	Süreç Analizi Veri Analizi	Sistem Analiz Raporu
Analizden Tasarıma Geçiş	Akışa Dayalı Analiz, Süreç belirtilimlerinin program tasarım diline dönüştürülmesi, Nesne ilişki şemalarının veri tablosuna dönüştürülmesi	Başlangıç Tasarımı Ayrıntılı Tasarım Başlangıç Veri tasarımı	Başlangıç Tasarım Raporu
Tasarım	Yapısal Şemalar, Program Tasarım Dili, Veri Tabanı Tabloları	Genel Tasarım Ayrıntılı Tasarım Veri Tasarımı	Sistem Tasarım Raporu



- 1: Düşük Maliyetli Gerçekleştirim**
- 2: Bakım Kolaylığı**
- 3: Zamanında Teslim**
- 4: Güvenirliği Yüksek Yazılım**
- 5: Uzmanca Yazılım Geliştirme**

Uygulama Yazılımı Geliştirmede Çelişen Tercihler



Pilot uygulama yapmaya üç nedenle başvurulur:

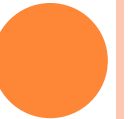


Kaynak :

www.mehmetduran.com

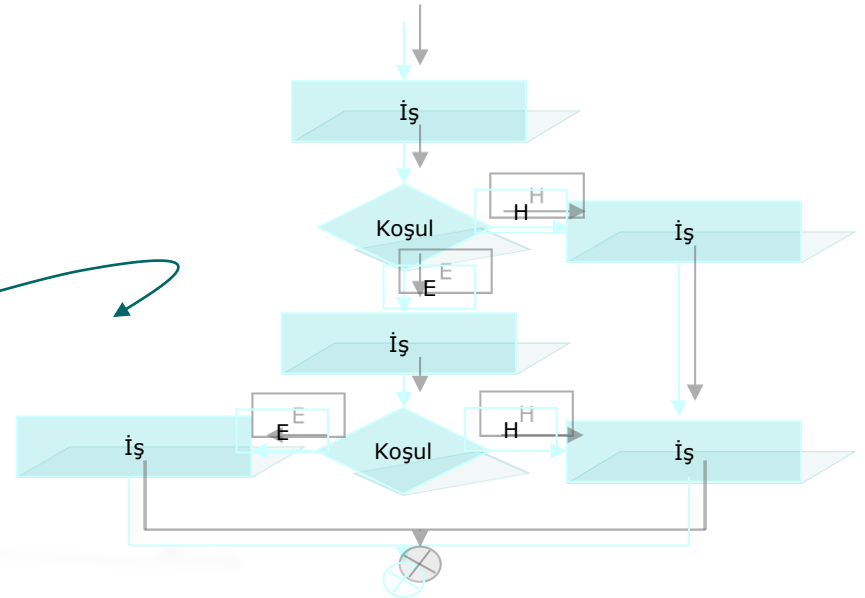
03/2007 - Yrd.Doç.Dr. G.YILMAZ Yazılım Geliştirme Yaşam Döngüsü

ÖRNEK ÇALIŞMA



Sistem

Tasarımı



Tasarım , herhangi bir mühendislik sürecindeki ilk adımdır.

Genel olarak deneyim bilgi birikimiyle desteklenen çeşitli kurallarla yapılır.

Çeşitli geliştirme teknikleri , tanımlama ve tasarım yöntemleri bulunsa da Yazılım mühendisliği hala bir “**sanat**” niteliğindedir.

En önemli adımlarından birisi **Veri Tasarımı** dır.

Çözümleme sırasında toplanan bilgilerin kullanılacak veri yapılarına dönüştürülmesini içerir.

Mimari Tasarım ;

Yazılım birimlerinin yapısal parçalarını , birbirleriyle ilişkilerini tanımlar.

Yordamsal Tasarım ;

Yazılılımı oluşturan yapısal birimler yordam ve fonksiyonlar haline dönüştülür.

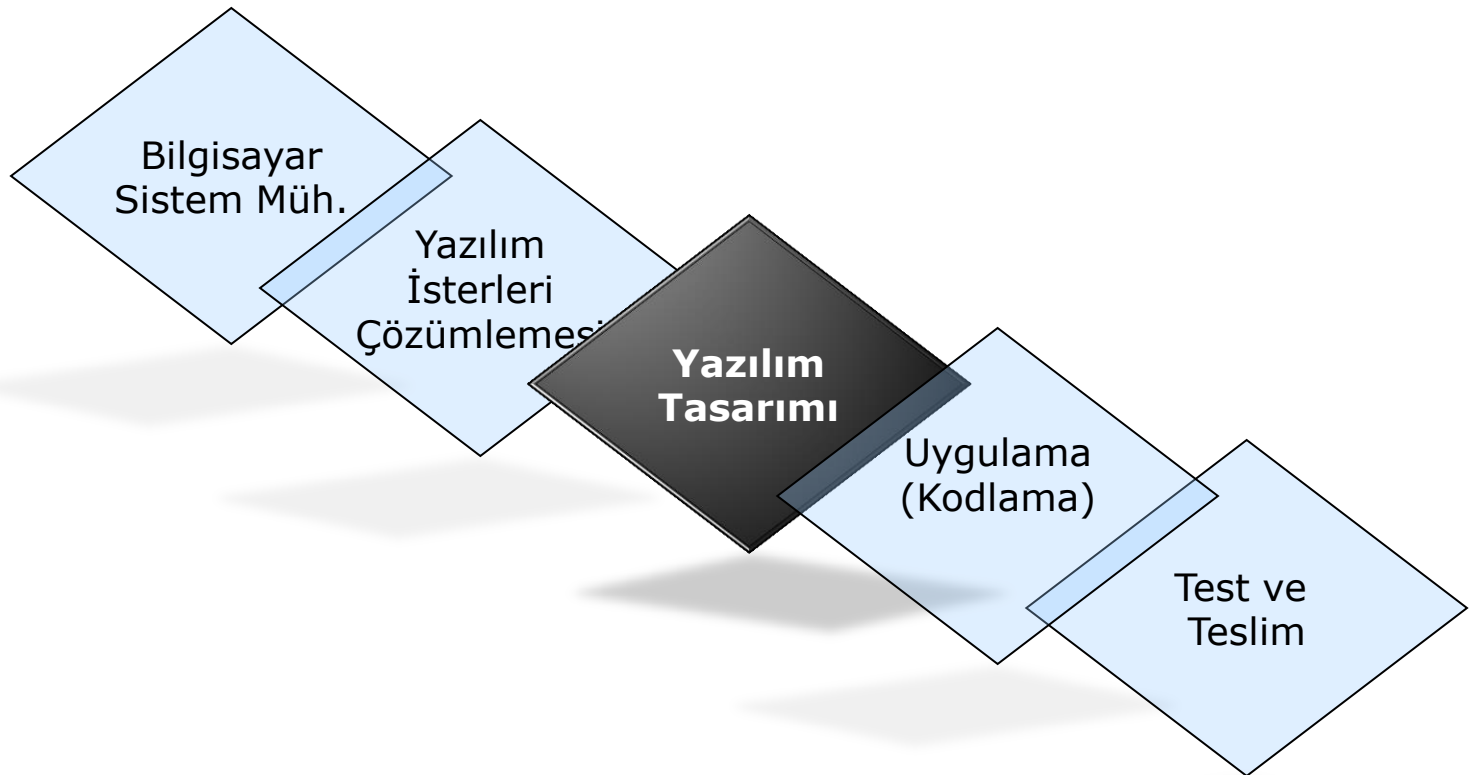
Arayüz Tasarımı ;

İnsan – Makine etkileşiminin şeklini alt sistemlerle olan arayüzlerin ayrıntılarını içerir.

Tüm detaylar belgede toplanır , değerlendirilir sonra da kodlama aşamasına geçilir.

Tasarım , yazılım testine kadar her şeyi etkilediğinden nitelik unsurunun öne çıktığı ilk aşama özelliğini taşımaktadır.

Yazılım geliştirme süreci içerisinde tasarım aşamasının yeri



Tasarımın ilk amacı Basitlik Olmalıdır.

“Sistem öyle tasarlanmalıdır ki, bir dizi değişiklik yapılsa bile sistem tasarımı hala basit kalabilmelidir.”

Yazılımın tasarımında gözden geçirilmesi gereken temel ilkelerden en önemlileri şunlardır ;

1. **Soyutlama** : Denetimi ve anlaşılabilirliği arttırmak üzere en az ayrıntı ve işlem yapmaktır.
2. **Bilgi Gizleme** : Modüllerin iç yapılarını diğerlerinden gizlemek, bu şekilde karmaşıklığı engellemek ve soyutlamayı arttırmaktır.
3. **Kapsama** : Tüm isterlerin eksiksiz olarak karşılanması amacıyla yordam ve verilerin denetim altına alınması.

Tasarım Nitelikleri :

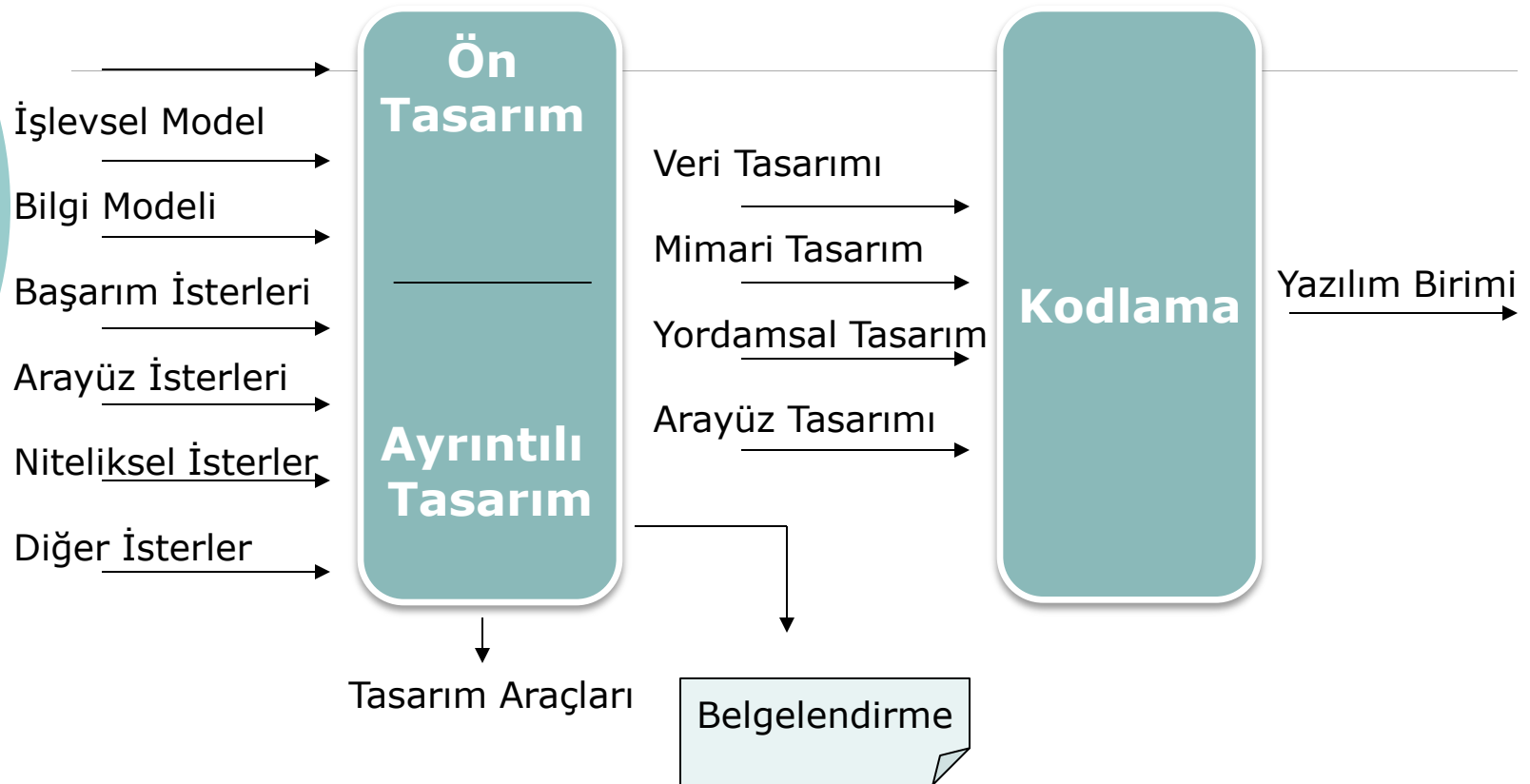
1. İsterler ile izlenebilirliği olmalıdır.
2. Geliştirilen birimin kodu ve testleri ile izlenebilirliği olmalıdır.
3. Programlama dilinden mümkün olduğunca bağımsız olmalıdır.
4. İşlevselliği , başarımı ve güvenilirliği yüksek bir ürün oluşturulmalıdır.
5. Yürütme sırasında oluşabilecek hataların ilgili iş sürecini aksatmayacak şekilde kotarılması sağlanmalıdır.
6. Öğrenmesi ve kullanımı kolay bir ürünü hedeflemelidir.
7. Tekrar kullanılabilir olmalıdır.
8. Bir ürün ailesine temel oluşturabilmelidir.
9. Kolay anlaşılmalıdır.
10. Gerektiğinde kolaylıkla değiştirilebilmelidir.
11. Kurumsal tasarım standartlarına uygun olmalıdır.
12. Diğer tasarımlarla birleştirilebilmesi mümkün olmalıdır.

Yazılım tasarımı sürecinde ve tanımlamalarında rehber olarak bazı standartlar kullanılabilir.

Yönetsel olarak süreç iki aşamada ele alınabilir :

- **Ön tasarım** : isterlerin veri ve mimari tasarımına dönüştürülmesidir.
- **Ayrıntılı tasarım** : Veri ve mimari tasarımın ayrıntılı veri yapıları ile algoritmik gösterime dönüştürülmesidir.

Yazılım geliştirme sürecinin tasarım aşaması sırasında kullanılan veri akışı :



Veri Tasarımı :

Veri yapısı ve veri modeli iç içe geçmiş iki kavramdır.

Birisi verinin bellekte tutulması veya saklanmasıyla ilgilenirken diğeri veriler arasındaki ilişki ve bağıntılar konusıyla ilgilenir.

Veriler üzerinde işlem yapacak olan algoritmalar da bu veri modellerine göre tasarlanırlar.

İyi bir veri tasarımı için neler gereklidir ;

- Veri yapıları/veri modelleri üzerinde yapılacak işlemlerin tanımlanması.
- Veri sözlüğünün oluşturulması.
- Döngüsel bir yol izlenmesi.
- Veri yapıları yalnızca kendilerini kullanan modüllere görünür olmalıdır.
- Sık kullanılan veri modelleri kütüphane haline getirilmelidir.
- Programlama dili özellikleri kullanılarak kodlama yapılmalıdır.

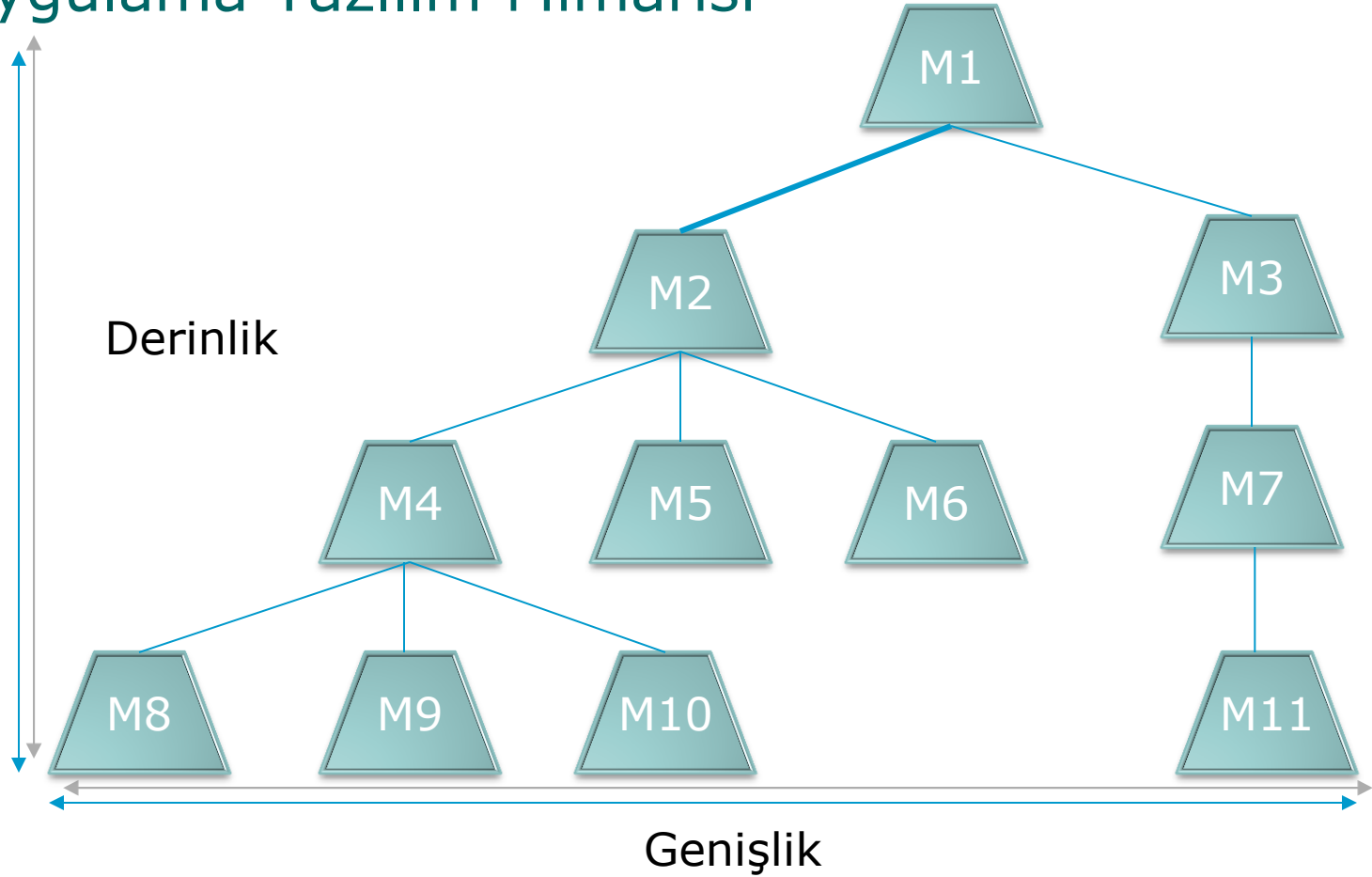
Mimari Tasarım ;

Uygulama yazılımı bir problemin çözümünü çeşitli parçalara bölerek sağlayabilir.

Parçaların yazılımdaki karşılığı **modüller** dir

Modüllerin sıradüzensel ilişkilerini gösteren yapıya **uygulama yazılım mimarisi** denir.

Uygulama Yazılım Mimarisi



- Uygulama alanının özellikleri :
Donanım özellikleri ...
- Uygulama yazılımının karmaşıklık derecesi :
Basit uygulamalar , tek program içinde , hertürlü arayüz ve bilgi işlemeyi kapsayacak şekilde geliştirilebilirler.Bölümlemek.
- Kullanıcı arayüzü kısıtlamaları :
Bilgi işleme birimleri ile kullanıcı arayüzünün farklı mimariye sahip işlemcilerde çalışması gereken durumlar olabilir.
- Taşınabilirlik :
Farklı işletim sistemi ve donanım özelliklerinde de çalışabilmesi gereklidir.

Yapısal Programlama Gösterimi :

Yazılım tarihinin en eski tasarım yöntemlerinden biri belirli yapıları kullanarak işlevleri metinsel bir şekilde anlatmaktır.

Tasarım dillerinin ortak özellikleri :

- Her türlü yapıyı destekleyebilen sabit bir anahtar sözcük listesi.
- Veri tipleri ve veri yapıları tanımlama yeteneği.
- Alt program tanımlama ve çağırma düzeneği.
- Bilgi işlemeyi serbest bir dille anlatabilme olanağı.
- Arayüz tanımlama yeteneği.
- Koşul ve çevrim yapıları.
- Giriş / Çıkış yapıları
- Zaman belirtimleri.

Grafiksel Gösterim :

Bazen bir resim bir çok satırdan oluşan bir anlatım yerine geçebilir.

Bu gerçekten hareketle grafiksel gösterim yöntemleri bulunmuş, bu yöntemleri kullanan yazılım araçları geliştirilmiştir.

Gösterim şekillerinin iyi bilinmemesi sonucu tasarımı yanlış anlaması, hatalı kodlamaya neden olabilir.

“Grafiksel gösterimlerin iyi öğrenilmesi ve anlaşılması gereklidir.”

Yapısal çözümleme ve tasarım :

Yapısal çözümleme ve tasarımda veri akış diyagramları ve durum geçiş diyagramları kullanılır.

UML :

Nesneye yönelik çözümleme ve tasarımın hem metinsel hem de grafiksel olarak yapılabilmesine yardımcı olan uluslar arası çevrelerce kabul edilmiş, standart ve yaygın bir tanımlama dilidir.

Akış diyagramları :

Çeşitli tasarım yöntemlerinde kullanılabilecek görsel anlatımları ve diyagramları ikiye ayırmak mümkündür.

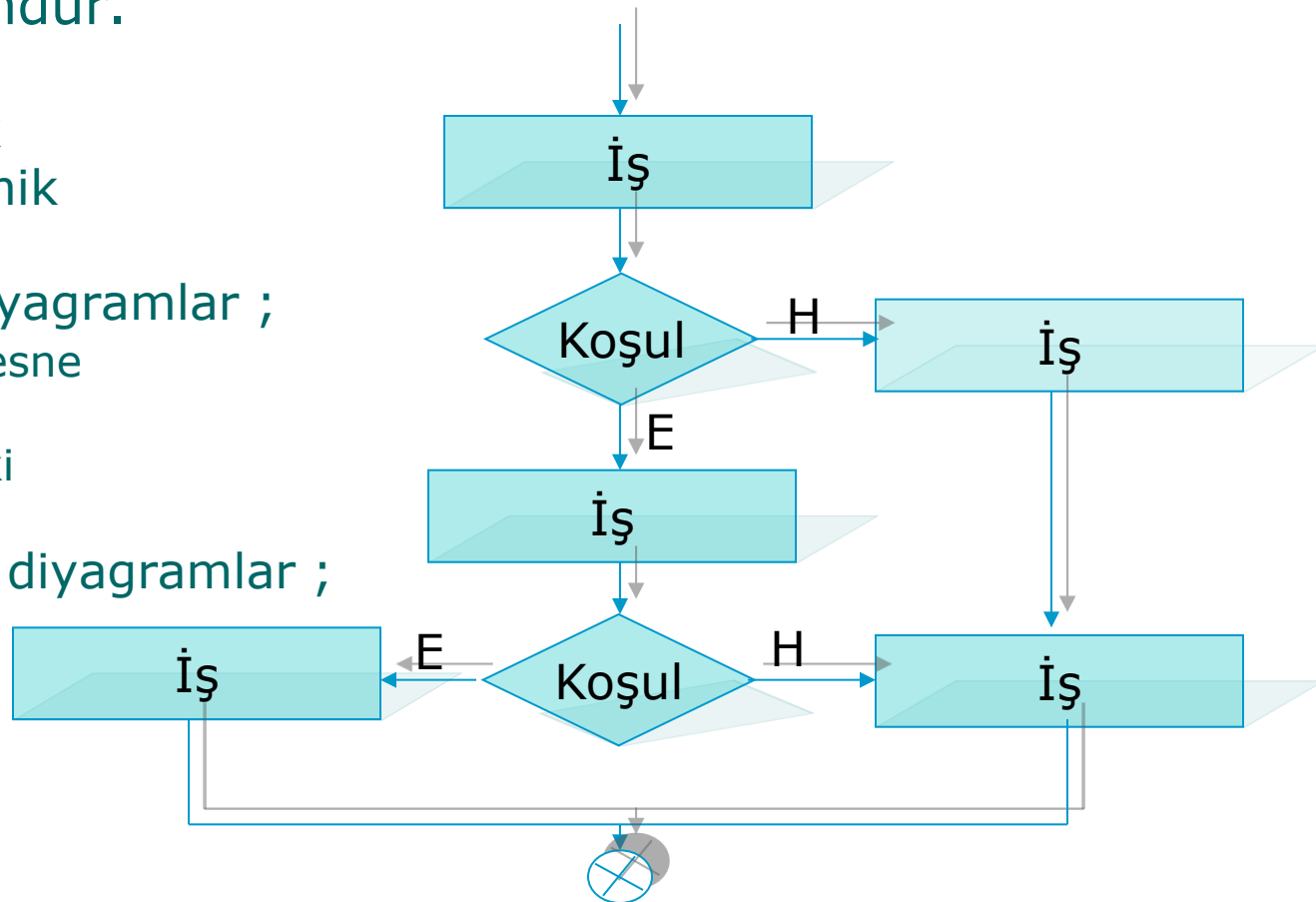
1. Statik
2. Dinamik

Statik akış diyagramları ;

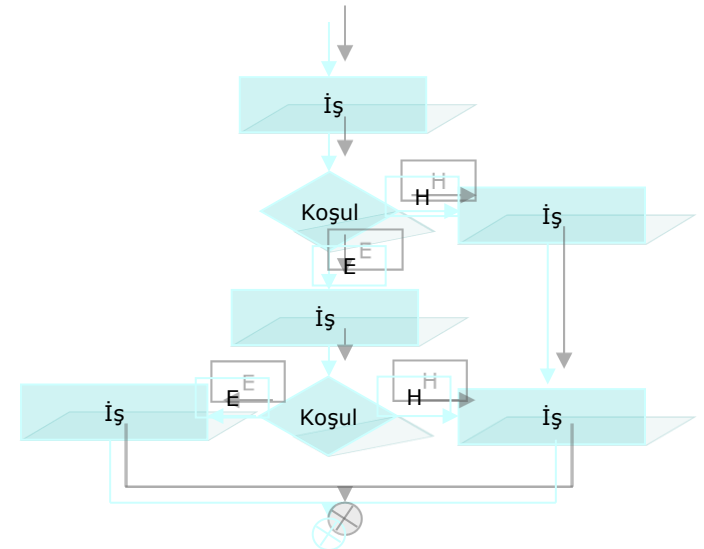
Sınıf ve nesne
Bileşen
Varlık-iliski
Yapı ...

Dinamik akış diyagramları ;

Veri akış
Etkileşim
Durum
Akış ...

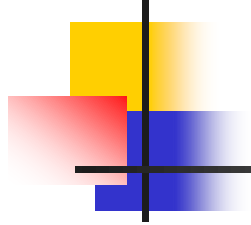


- <http://e-bergi.com/2008/Ekim/Cevik-Modelleme-ve-Cevik-Yazilim-Gelistirme>
- Sistem Analizi ve Tasarımı Prof.Dr. Oya Kalıpsız
- Yazılım Mühendisliği Dr.M.Erhan Sarıdoğan
- BT HABER dergisi, Sayı 259, 2000.
- DELPHI UNLEASHED, SAMS PUBLISHING, Charles Calvert, 1997.
- www.mehmetduran.com
- <http://jamshidhashimi.com/2010/08/23/agilecevik-modelleme-ve-cevik-yazilim-gelistirme>
- http://en.wikipedia.org/wiki/Agile_Modeling
- http://en.wikipedia.org/wiki/Agile_software_development
- <http://www.minepla.net/2008/10/agilecevik-yazylym-gelithirme>

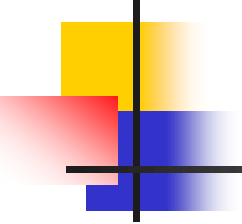




Minimum Viable Product



- MVP, ürününüzün bir aşamasıdır. Müşteriler ve tepkileri hakkında doğrulanmış veriler elde etmek için minimum efor sarfetmenize yarar.
- MVP sade bir ürün anlamına gelmez. Direkt olarak ürün geliştirmek ve satmak üzerine odaklanmış bir strateji ve süreçtir. Fikir geliştirme , prototip oluşturma, , bilgi toplama, analiz ve öğrenmeden oluşan tekrarlanan bir süreçtir. Tekrar işlere harcanan zamanı düşürmeyi amaçlar. Süreç makul pazar payını elde etmiş bir ürün ortaya çıkana veya uygulanamaz addedilene kadar kendini tekrar eder.
- Bir "Minimum Uygulanabilir Ürün" ürünün tamamını veya bir kısmını kapsayabilir.



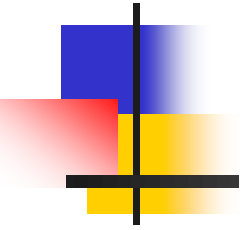
■ Teknikler

- **Ürün :** Web uygulamaları için kabul gören MVP stratejisi şu şekildedir. Model bir web site yapılır ve siteye direk trafik çekmek için internet reklamı yapılır. Maket web site pazarlamaya yönelik bir ana sayfadan ve daha fazla açıklama veya satın alma istendiğinde içeri girilmesini sağlayan bir linkden ibarettir. Link aslında satın alma sistemine bağlı değildir. Müşteri ilgisini ölçmeye yarar.
- **Özellik:** (önce konuşlandır, sonra kod yaz) Bir web uygulamasında yeni bir özelliğin linki kolayca görülen bir lokasyonda olmalıdır. Böylece tıklamaları sayarak müşterilerinizin bu özelliğe olan rağbetini öğrenebilirsiniz.

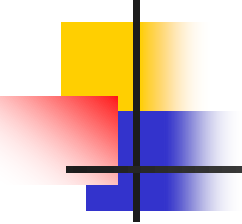


Ayrışmalar

- Bir MVP'nin etkisini değerlendirmek; devreye almadan önce ürün hakkındaki fikirleri elemek için kullanılan bir pazar testi stratejisidir. Yaygın web uygulaması dilleri ve hızlı uygulama geliştirme araçları ile kolaylaşmıştır.
- MVP, erken aşamada zaman ve para harcayan geleneksel market test stratejilerinden ayrılır. Aynı şekilde, erken ve sık sık sürüm çıkarmak ve kullanıcıların ürün özelliklerini tarif etmesini beklemek temeline dayanan açık kaynak metodolojisinden de ayrılır. **MVP, direk ve dolaylı kullanıcı tepkisini ölçmeye dayalı bir sistem olsa da, ürünün tüm yaşam döngüsü boyunca korunan bir ürün vizyonu ile başlar.**
- MVP, Steve Blank'ın müşteri geri beslemeleri ile sürekli ürün döngüsünü temel alan "müşteri geliştirme" isimli metodolojisinin bir kısmı olarak bilinir. Ayrıca, henüz var olmayan bir ürünün veya özelliğin sunumu A/B testi gibi istatistiksel hipotez testlerinden daha düzgün olabilir.
- Önce konuşlandır sonra kod yaz metodu test güdümlü geliştirme olarak anılan çevik kod testi metodolojisine yakındır



İş Kurmadan Önce Fikrinizin Potansiyelini Ölçün



Öncelikle bu sorulara cevap verin

- Sunmayı planladığım bu yazılım/ürün pazar ihtiyaçlarını karşılayacak mı?
- Potansiyel / hedef müşterilerim kim ve neredeler?
- Pazarda rekabet durumu nasıl? Yoğun bir rekabet mi var yoksa pazarda hiç bir rakip / ikame ürün yok mu? (Pazarda rekabetin çok olması kadar hiç olmamasının da bir sorun olduğunu unutmayın. Rakip olmaması, herhangi bir pazar olmadığına da işaret edebilir)
- Eğer rekabet varsa, benim ürünüm pazardaki ürünlerden nasıl farklılaşacak?
- Yazılımım/Ürünüm değişen trendlere uyum sağlayabilecek mi yoksa saman alevi gibi parlayıp sönecek mi?
- Kanunen yazılımı/ürünü pazara sunmamda herhangi bir engel var mı (ülke yasal mevzuatı, patent / telif hakları vb.)
- Karlılığımı korumak için hangi fiyattan satış yapmalıyım? Potansiyel müşteriler yazılıma/ürüne ne kadar ödemeye hazırlar?

Pivot etme konusundaki tanımım: Yolda değişiklik yapmak, kararlarından vazgeçmek, iş fikrinde yükselişe geçebilmek için uygun süreçleri tekrardan düşünmek.

Belki de müşterilerinden vazgeçmek.

Pivot edebilmek için sistemin ana sorununu bulmanız gerekir. Eksik, yavaş ya da yanlış bir şeyler varsa düzeltmeye hazır olun.

İşler doğru gitmeyince iş fikri tutmadı mı sorusuna kapılmayın. **Instagram, Facebook uygulaması olsaydı ve tutmasaydı fikir mi kötü derdik yoksa platform mu?**

Tekrardan gözden geçirelim;

Fikriniz çok iyi ama müşteri segmentasyonunuz yanlış olabilir.

Fikriniz çok iyi fakat müşterinin beklentileri farklı olabilir.

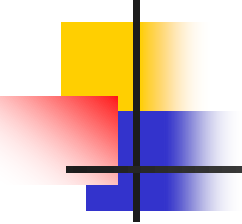
Fikriniz çok iyi fakat yanlış platformda olabilirsiniz.

Fikriniz çok iyi fakat iş mimarisini yanlış kurmuş olabilirsiniz.

Fikriniz çok iyi fakat para kazanma şekliniz doğru olmayabilir.

Fikriniz çok iyi olabilir fakat kullandığınız teknoloji doğru olmayabilir.

Birçok fikir bence MVP (Minumum Viable Product) olmadan yayına girdiği için yüz binlerce lira maliyet nedeniyle pivot etmesi zorlanıyor. Siz MVP'yi önemseyin ve Pivotunuzu MVP boyutunda yapmaya özen gösterin.



Geri Bildirim Alın

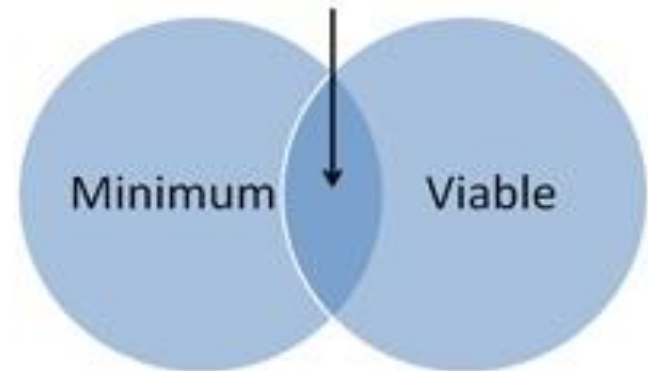
- Ürününüzü / fikrinizi pazara sürmeden önce potansiyel müşterilerinizin bulunduğu yerlerde ufak test sunumları, anketler yaparak insanların ne düşündüğünü öğrenin. Aynı zamanda iş forumlarında, sosyal medyada fikrinizi paylaşarak insanların ne düşündüğünü, neyi beğenip beğenmediğini de öğrenebilirsiniz.

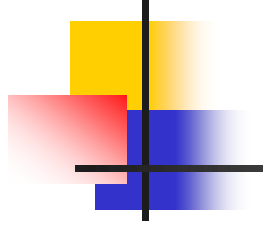
Prototip Üretin ve Pazara Sunun (Minimum Viable Product)

Bir iş fikri (ürün / servis) ile ilgili en doğru geri bildirimi ancak insanlara gerçekten satarak alabilirsiniz.

Ürüne para ödeyip test eden insanlar çok daha gerçekçi geri dönüşler yapacak ve ürününüzü eksileriyle ve artılarıyla değerlendireceklerdir.

Minimum + Viable: Good products for startups to build



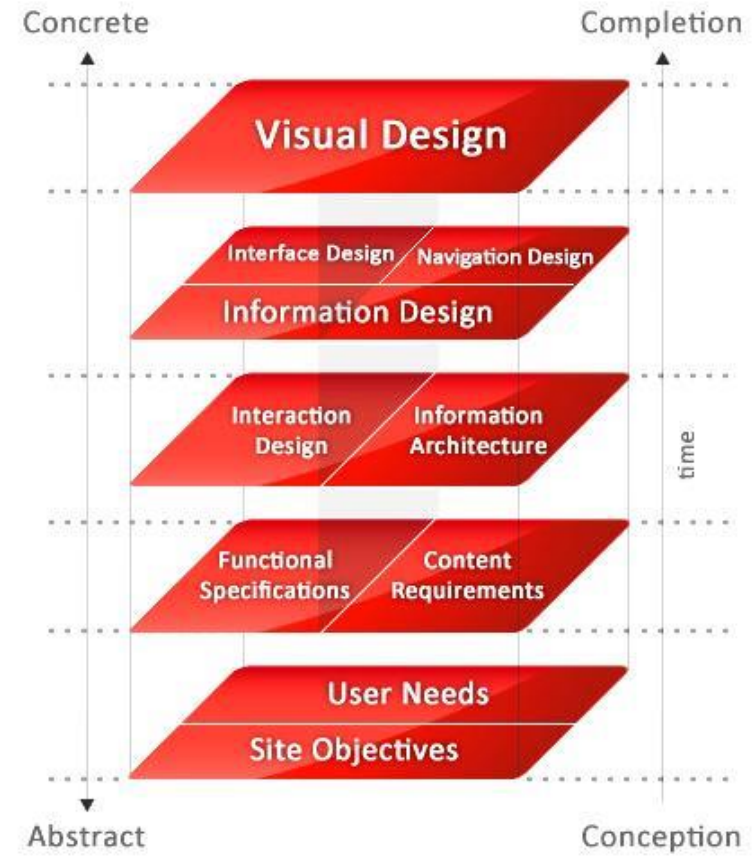


Bu sebeple ürününüzden numune bazlı bir üretim yaparak ufak miktarda satışa çıkarıp geri dönüşü görün. Tabii kafanızda yer alan fikir otomobil vs gibi birim maliyeti çok yüksek olan bir ürün değilse Minimum Viable Product, Türkçe çevirisi ile “Temel Kullanılabilir Ürün” olarak adlandırılan prototip ürünler ile gerçek Pazar potansiyelini ölçebilirsiniz.

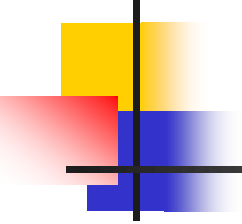
Oluşturduğunuz prototiplere düşündüğünüz fiyat etiketini vurun ve satmaya çalışın. İnsanlar ne kadar fiyattan satınca almaya hevesli, ne kadar fiyat koyunca pahalı geliyor öğrenerek en doğru satış fiyatlarını da çıkarabilirsiniz.

Tabii burada hedef müşterilerinizin ödemeye hazır olduğu ücretin maliyetinizden kesinlikle daha yüksek ve karlılığınızı korumaya (işletmenizi yaşatmaya) elverişli olması en önemli nokta.

Arayüz Tasarımı



İyi tasarlanmış bir kullanıcı-yazılım arayüzünün,
yapılan işin kalitesini artırma,
kullanıcının tatmin düzeyini yükseltme,
işgücünün verimliliğini artırma,
yazılımın kontrol ettiği sistemin güvenliğini sağlama
gibi çok önemli avantajları vardır.

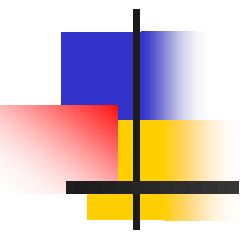


İçindekiler

- Yazılım tasarımı
- **Arayüz tasarımı**
 - Kullanılabilirlik
 - M.V.P.
 - Temel Kurallar

Yazılımın belirli yöntemlere göre uygulanması çeşitli nedenlerden ötürü sıkıntılı olabilir , buna rağmen kurallar dahilinde yazılıma çaba harcanmalıdır.

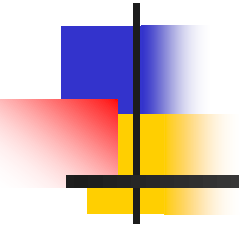
Yazılım tasarımının da kullanılabilecek yöntemler

- 
- Böl ve Yönet
 - Tümevarım
 - Tümdengelim
 - Aşamalı ayrınılandırma
 - Buluşsal yöntemler
 - Artımlı yaklaşım
 - İşleve yönelik yaklaşım
 - Yapısal tasarım
 - Veri akışına yönelik tasarım
 - Nesneye yönelik tasarım
 - Veriye yönelik tasarım
 - ...

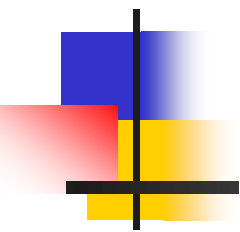
Kullanılabilirlik, sistem ve kullanıcıların arayüz aracılığı ile açık ve hızlı bir biçimde iletişim kurabilmesidir.

“**Kullanılabilirlik**, yazılımların geliştirilmesi ve bu yazılımların başarısında en önemli faktördür.”

- ❖ Kullanıcı merkezli tasarım, ürünlerin tasarlanmasında ve prototip ürünlerin değerlendirilmesinde kullanılabilirlik konusu üzerinde odaklanır.
- ❖ Üretici işletmeler de bu konuyu rekabet avantajı sağlayıcı bir husus olarak görürler.
- ❖ Özellikle ev ve işyerinde kullanılan ürünlerin gittikçe kompleks hale gelmesiyle kullanılabilirlik konusu daha da önem kazanmaya başlamıştır.



- ❖ Bilişim alanındaki gelişmeler ve bilgisayarın insan yaşamının ayrılmaz bir parçası haline gelmesi, kullanılabilirlik konusunda yapılan araştırma ve uygulamaların büyük oranda yazılım-kullanıcı arayüzü (software-user interface) alanında boy göstermesine neden olmuştur.
- ❖ Teknik olarak birçok üstün özelliği olduğu halde kullanım kolaylığı açısından oldukça kötü tasarlanmış bir ürünün piyasada tutulabilir olduğunu söylemek mümkün değildir.



❖ Zira bir ürünün kalitesi ve tüketici tarafından kabul edilebilirliği sadece teknik özelliklerine değil, aynı zamanda ve daha da önemlisi ürünün, kullanım kolaylığı ve kullanıcının fiziksel, zihinsel ve psikolojik özellikleri ile uyumlu olmasına bağlıdır.

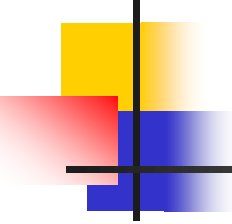
❖ **Kullanıcıların, kullanım kolaylığını ürün kalitesinin vazgeçilmez ve en önemli unsuru olarak görmeleri, üreticileri ürün tasarım sürecine insan faktörleri /ergonomi uzmanlarını dahil etmeye sevk etmiştir .**

- ❖ Ürünler belli bir amacı gerçekleştirmeye yönelik olarak tasarlanırlar. Bu amaca ulaşmak için ürünler, genellikle bir veya birden fazla kullanıcı tarafından kullanılır.
- ❖ Burada önemli olan, kullanıcıların, kendilerine sunulan ürün ile kısa sürede, hata yapmadan ve üründen memnun kalarak amaçlarına ulaşmalarını sağlamaktır.
- ❖ Etkin ve kaliteli bir kullanıcı-ürün arayüzü tasarımının önemi bu noktada başlar. Kullanıcı-ürün arayüzü, kullanıcıların ürünü kullanmalarını sağlayan tasarım kararlarının toplamıdır.
- ❖ Arayüz tasarımı yapılırken amaç, kullanıcı-ürün entegrasyonunu sağlayarak yüksek performans elde etmektir.

- ❖ Sağlıklı bir arayüz tasarımı disiplinler arası bir çalışmayı gerektirir. Bu disiplinler arasında ergonomist / insan faktörleri uzmanı merkezi bir işlev görür.
- ❖ Ergonomist tasarım grubuna tasarım alternatifleri için kullanıcı performansı ile ilgili bilgileri sağlar.
- ❖ Kullanıcı performansı ile ilgili bilgiler, genellikle bir model veya prototip üretilerek, bu prototip veya modeli belli bir kullanıcı kitlesinin kullanması neticesinde yapılan gözlem ve ölçümler neticesinde elde edilir.
- ❖ Bu şekilde yapılan kullanıcı testleri oldukça pahalı ve zaman alıcıdır. Bundan dolayı tasarımcılar, genellikle kendi bilgi ve deneyimlerine, hayal güçlerine ve kendilerini kullanıcı yerine koyarak ürün geliştirmektedirler.

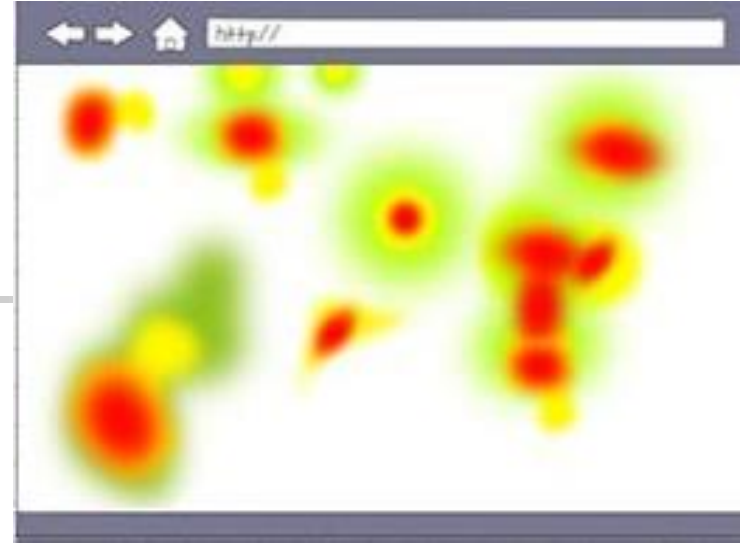
“Arayüz tasarımlarının kullanılabilirliğinin değerlendirilmesi genellikle **heuristik değerlendirme** ve **kullanıcı testleri** olmak üzere iki şekilde yapılır.”

1. **Heuristik değerlendirme** bir tasarımın özellikleri ile önceden belirlenmiş kullanılabilirlik prensipleri karşılaştırılarak uzman görüşüne dayalı olarak yapılan bir değerlendirmedir.
2. **Kullanıcı testleri ile yapılan değerlendirme** ise gerçek kullanıcılar ile yapılan, kullanıcı-ürün etkileşiminin gerçek ortamda gözlenebildiği ve ürünün kullanımı ile ilgili bilgilerin doğrudan kullanıcılardan elde edilebildiği bir yöntemdir.



Günümüzde Kullanılan

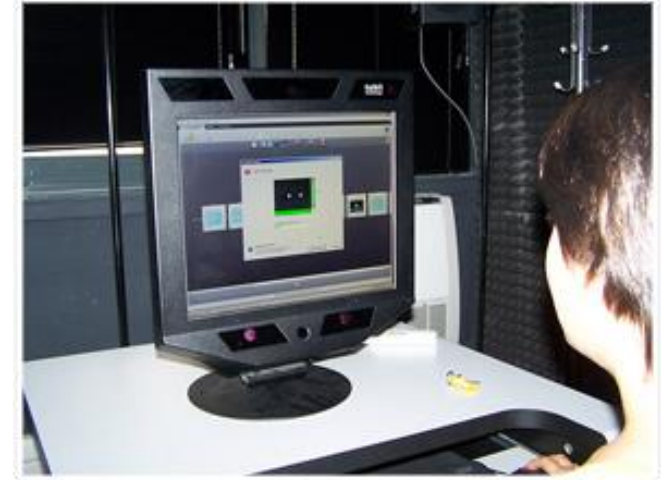
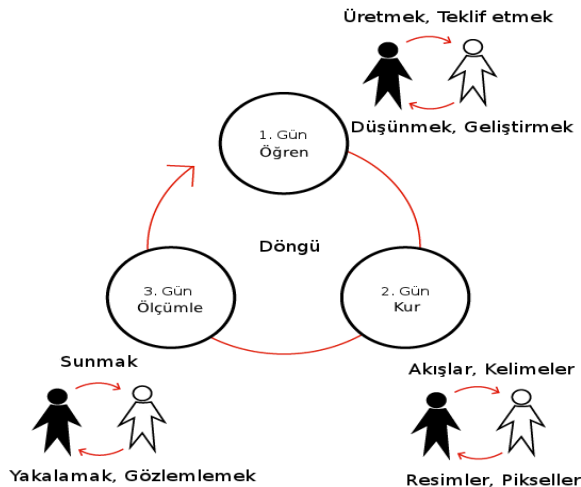
KULLANILABİLİRLİK TESTLERİ



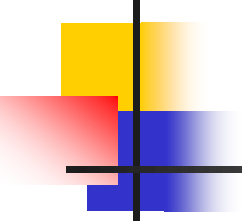
1. Göz İzleme Cihazı ile Kullanılabilirlik Testi
2. Gerilla ile Kullanılabilirlik Testi
3. Online ile Kullanılabilirlik Testi
4. Mobil Test
5. Analitik analiz
6. Reklam analizi
7. Uzman analiz

Göz İzleme Cihazı ile Kullanılabilirlik Testi Göz İzleme Nedir?

Göz izleme, kullanıcının nereye, ne kadar süre ve kaç kere baktığına, anlık ve geçmiş dikkatinin nerede yoğunlaştığına, niyetine, zihinsel durumuna ilişkin bilgi sağlamakta kullanılan bir yöntemdir. Göz izleme teknolojisinin kullanım alanları yalnızca kullanılabilirlik testleriyle sınırlı olmayıp, market araştırmaları ve psikolojik incelemeler gibi pek çok alanda kendisine yer bulmaktadır.



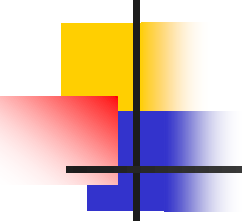
Neden Göz İzleme?



Göz İzleme ile geleneksel kullanılabilirlik metotları kullanarak ulaşabileceğiniz verilerin çok daha fazlasına ulaşabilirsiniz.

Örneğin;

- Ürününüzü kullanıcı gözüyle görebilirsiniz.
- Kullanıcıların neleri önemli ve ilginç bulduklarını ve neleri görmezden geldiklerini belirleyebilirsiniz.
- Kullanıcıların karar verme mekanizmaları hakkında bilgi sahibi olursunuz.
- Arayüzdeki verimsiz ve etkisiz alanları belirleyebilirsiniz.
- Arama yollarını ve stratejilerini tespit edebilirsiniz.
- Görsel tasarım ile hedeflerin ne kadar uyuştuğunu ortaya çıkarabilirsiniz.
- Kullanıcı deneyimine nelerin etki ettiğini gözlemleyebilirsiniz.



Göz İzleme ile Kullanılabilirlik Testi Nasıl Yapılır?

- **Hedef Kitle Belirleme & Senaryo Oluşturma** : Hedef kitleniz ve gerçekleştirmelerini istediğiniz görevler belirlenir.
- **Test** : Kullanıcılar verilen görevleri gerçekleştirmeye çalışırken, mouse hareketleri, göz hareketleri, sesleri ve görüntüleri kayıt altına alınır.
- **Analiz ve Raporlama** : Test sonucunda elde edilen kullanıcıya ait bütün göz ve mouse hareketleri, sıcaklık haritaları, ses ve mimikleri yardımıyla analiz sürecine geçilir. Analiz sonrasında ortaya çıkan başarı oranları, anket sonuçları ve diğer önemli bulgular kapsamlı bir rapor haline getirilir.
- **Uygulama** : Raporda göze çarpan maddeler için tasarım önerileri ve çözüm yolları oluşturulur.

Göz İzleme Testi Sonucunda Elde Edilen Veriler Nelerdir?



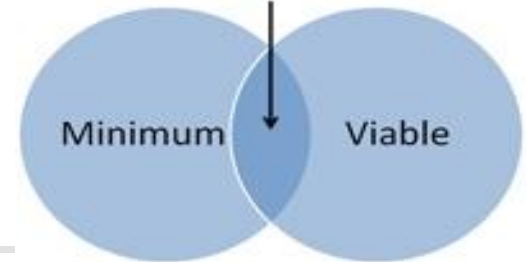
- **Sıcaklık Haritaları** : Her sayfa için kullanıcıların hangi noktalara ve ne kadar süre baktıklarını gösteren haritalar.
- **Kullanıcı Videoları** : Kullanıcıların görevleri gerçekleştirirken çekilmiş, sesli düşünce ve mimiklerini içeren videolar.
- **Yol Haritaları** : Her bir görev için kullanıcıların ne kadar kısmının, hangi yolları izlediğini gösteren haritalar.
- **Mouse Hareketleri** : Kullanıcının hangi anda, nereye, kaç kere tıkladığını belirleyen istatistikler.
- **Zaman İstatistikleri** : Kullanıcıların görev bitirme, sayfada kalma, link arama süreleri gibi zaman bazlı istatistiklerini içeren veriler.



Minimum Viable Product

Minimum Viable Product

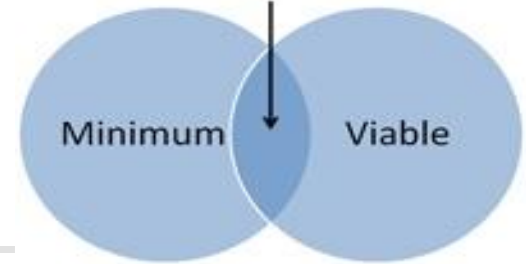
Minimum + Viable: Good products for startups to build



- MVP, ürününüzün bir aşamasıdır. Müşteriler ve tepkileri hakkında doğrulanmış veriler elde etmek için minimum efor sarfetmenize yarar.
- MVP sade bir ürün anlamına gelmez. Direkt olarak ürün geliştirmek ve satmak üzerine odaklanmış bir strateji ve süreçtir. Fikir geliştirme , prototip oluşturma, bilgi toplama, analiz ve öğrenmeden oluşan tekrarlanan bir süreçtir. Tekrar işlere harcanan zamanı düşürmeyi amaçlar. Süreç makul pazar payını elde etmiş bir ürün ortaya çıkana veya uygulanamaz addedilene kadar kendini tekrar eder.
- Bir "Minimum Uygulanabilir Ürün" ürünün tamamını veya bir kısmını kapsayabilir.

Minimum Viable Product

Minimum + Viable: Good products for startups to build

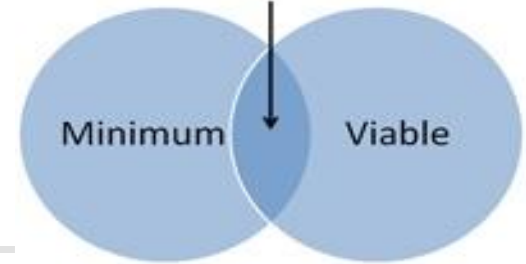


■ Teknikler

- **Ürün :** Web uygulamaları için kabul gören MVP stratejisi şu şekildedir. Model bir web site yapılır ve siteye direk trafik çekmek için internet reklamı yapılır. İçi doldurulmamış bir arayüzden oluşan bir web site pazarlamaya yönelik bir ana sayfadan ve daha fazla açıklama veya satın alma istendiğinde içeri girilmesini sağlayan bir linkden ibarettir. Link aslında satın alma sistemine bağlı değildir. Müşteri ilgisini ölçmeye yarar.
- **Özellik:** (önce konuşlandır, sonra kod yaz) Bir web uygulamasında yeni bir özelliğin linki kolayca görülen bir lokasyonda olmalıdır. Böylece tıklamaları sayarak müşterilerinizin bu özelliğe olan rağbetini öğrenebilirsiniz.

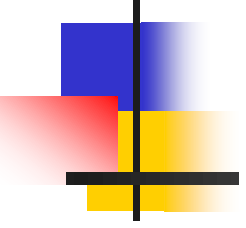
Minimum Viable Product

Minimum + Viable: Good products for startups to build



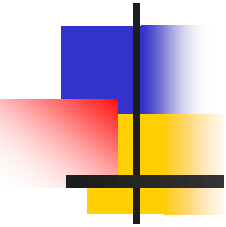
- **Ayrışmalar**

- Bir MVP'nin etkisini değerlendirmek; devreye almadan önce ürün hakkındaki fikirleri elemek için kullanılan bir pazar testi stratejisidir. Yaygın web uygulaması dilleri ve hızlı uygulama geliştirme araçları ile kolaylaşmıştır.
- MVP, erken aşamada zaman ve para harcayan geleneksel market test stratejilerinden ayrılır. Aynı şekilde, erken ve sık sık sürüm çıkarmak ve kullanıcıların ürün özelliklerini tarif etmesini beklemek temeline dayanan açık kaynak metodolojisinden de ayrılır. MVP, direk ve dolaylı kullanıcı tepkisini ölçmeye dayalı bir sistem olsa da, ürünün tüm yaşam döngüsü boyunca korunan bir ürün vizyonu ile başlar.
- MVP, Steve Blank'ın müşteri geri beslemeleri ile sürekli ürün döngüsünü temel alan "müşteri geliştirme" isimli metodolojisinin bir kısmı olarak bilinir. Ayrıca, henüz var olmayan bir ürünün veya özelliğin sunumu A/B testi gibi istatistiksel hipotez testlerinden daha düzgün olabilir.
- Önce konuşlandır sonra kod yaz metodu test güdümlü geliştirme olarak anılan çevik kod testi metodolojisine yakındır



Nedir Kullanılabilirlik Kriterleri ?

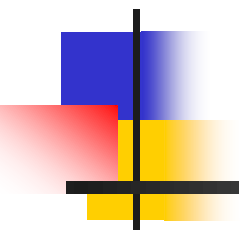
Kullanılabilirlik Kriterleri :



1-İşlevsellik: Sistem, kullanıcılar görevlerini yerine getirirken, yapılan görevin gerektirdiği ihtiyaç ve gereksinimleri karşılamalıdır.

2-Kontrol Edilebilirlik: Sistem mümkün olduğu kadar, kullanıcının kontrol edebilmesine olanak tanınmalıdır.

3-Esneklik: Kullanıcı arayüzü, yapısı, bilginin sunulması ve değişik potansiyel kullanıcıların ihtiyaç ve gereksinimlerine uygunluk bakımından yeterli esnekliğe sahip olmalıdır.

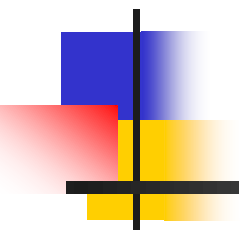


4-Hata Yönetimi: Sistem, hataların önlenmesi, hata olasılığının azaltılması, hataların tolere edilmesi ve hata oluştuğunda giderilmesi amacıyla kullanıcı ile interaktif ilişki kurabilecek şekilde tasarlanmış olmalıdır.

5-Kullanıcıya Uygunluk: Sistemin yapısı ve çalışma şekli kullanıcının fiziksel, zihinsel ve psikolojik özelliklerine uygun olmalıdır.

6-Kendi Kendini Betimleme: Sistem, kullanıcıya geri-besleme, kılavuzluk ve destek sağlayacak şekilde tasarlanmış olmalıdır.

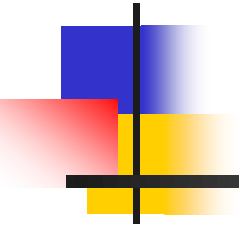
7-Tutarlılık: Sistemin çalışma şekli, yer, biçim ve format olarak kendi içinde tutarlılık arz etmelidir.



8-İş Yüğü: Sistem, kullanıcının, fiziksel ve zihinsel iş yükünü kabul edilebilir sınırlar içinde tutmalı ve etkileşim hızını artırmak için mesajlar kısa, öz ve anlaşılır olmalıdır.

9-Öğrenilebilirlik: Kullanıcının sistemi kullanırken öğrenme süreci hızlı olmalı ve zaman içinde benzer uygulama adımlarını rahatlıkla hatırlayabilmelidir.

Modüler bir şekilde geliştirilen yazılımın çeşitli arayüzleri bulunur.



- İçsel arayüzler

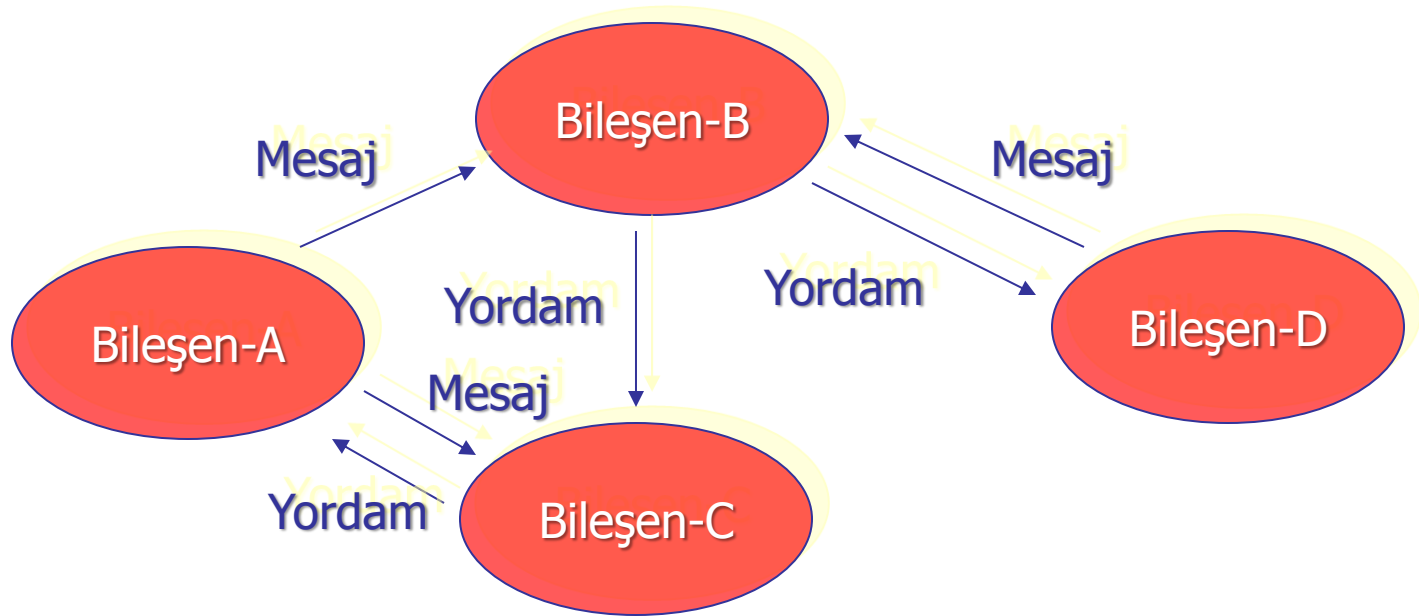
Yazılımın kendi iç öğeleri, bileşenleri ve birbirleri arasındadır.

- Dışsal arayüzler

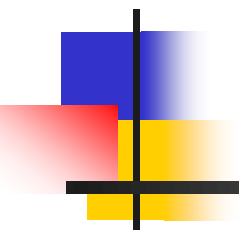
Yazılımın dış dünya ile arayüzü ...

Büyük yazılımlar birkaç ana öğeden , her bir öge birkaç bileşenden yada birimden oluşabilir.

Bileşenler arasında mutlaka tanımlı bir arayüz vardır.



Bileşenler arasında arayüz tasarlarken dikkat edilmesi gereken unsurlar ...

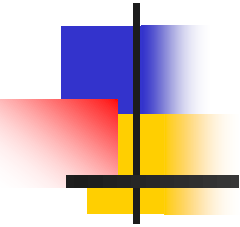
- 
1. Anlaşılabilir olmalıdır.
 2. İleti boyları uygun şekilde ayarlanmalıdır.
 3. Büyük veri aktarımları için ortak veri deposu ve verinin adresi kullanılmalıdır.
 4. Belirli veri türlerine bağımlı olmamalıdır.

"Kullanımı kolay, etkili ve açık bir arayüze sahip olmalıdır."

Sistemin insanlarla olan arayüzünün çok etkin ve kullanışlı olarak , verimliliği arttıracak şekilde, kullanıcı dostu olarak tasarlanması gerekir.

Bir sistemin arayüzünü öğrenmek ve etkin bir şekilde kullanmak ne kadar kolay olursa o sistem yetenek ve işlevlerinden yarar sağlamak da o kadar artar.

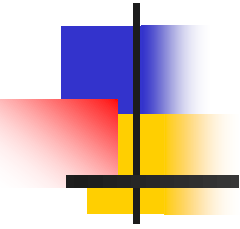
Kullanıcının test edilmesi istenen arayüzü kullanacağı bilgisayar bulunmaktadır.



Göz izleme özelliğine sahip bu bilgisayar sayesinde kullanıcının testi gerçekleştirdiği süre içerisinde arayüzün neresine, ne kadar süre ile ve kaç kere baktığı gibi kullanıcının görsel davranış biçimini belirten veriler, görsel ve sayısal olarak çeşitli şekillerde belirlenmelidir.

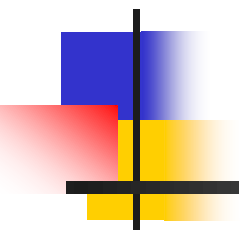
Bu sayede arayüzün tasarımı ve kullanıcı tarafından nasıl algılandığı konusunda bilgi edinilmektedir.

Bununla beraber, test odasında bulunan iki kamera sayesinde kullanıcının klavye-fare hareketleri ve yüz hareketleri kaydedilmelidir.



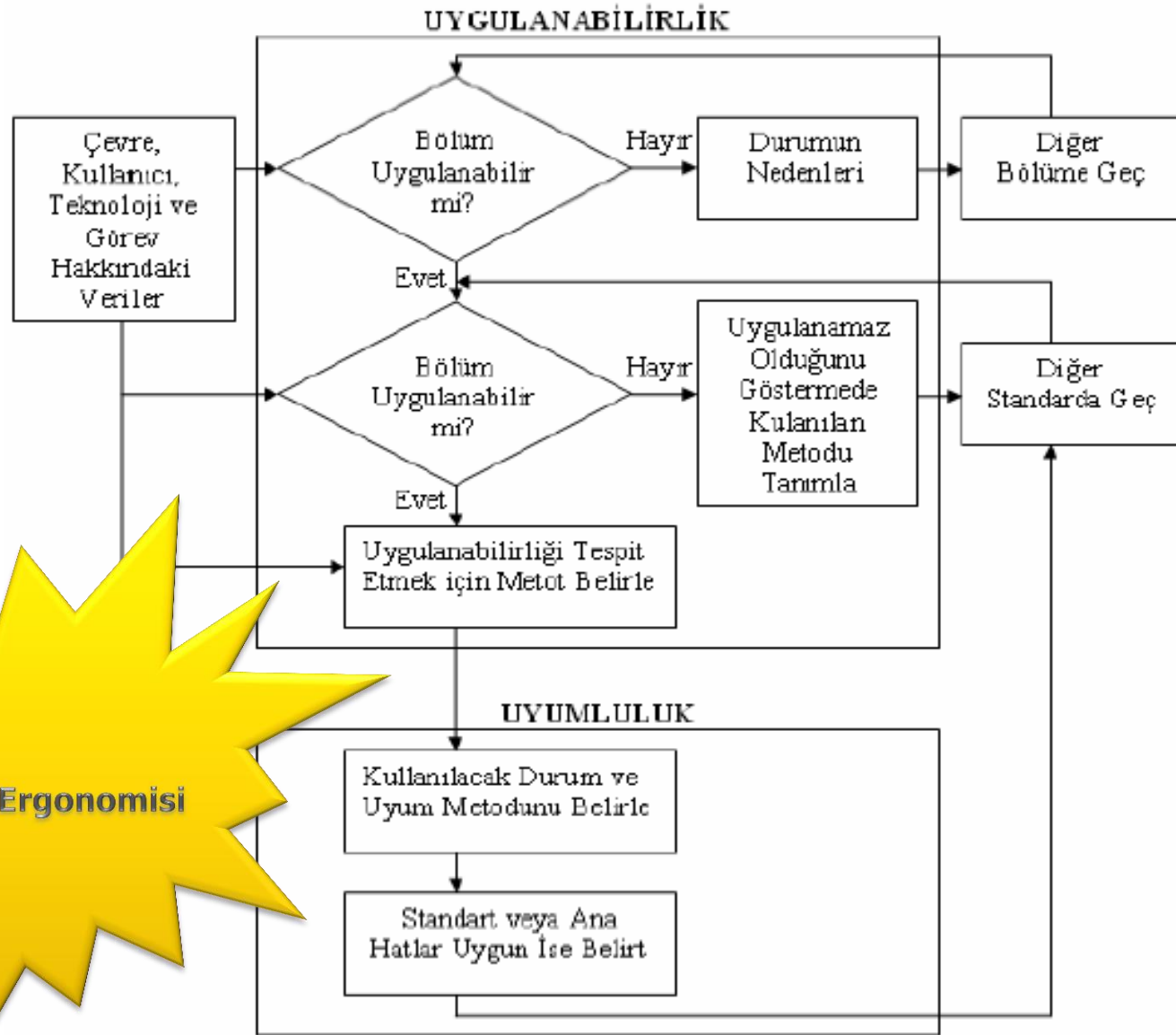
Kullanıcıların yüz hareketlerinin ve konuşmalarının kaydı ve incelenmesi, kullanıcının arayüze yaklaşımı hakkında bilgi sağlamalıdır.

Bu kayıt işlemi sayesinde veri kaybı olmadan, istenilen zamanda testlerin analizi gerçekleştirilebilmelidir.



“Yazılımların değerlendirilmesinde önemli bir kriter olan **kullanılabilirlik kavramı**, bir arayüzün verimli ve etkili bir şekilde kullanılması olarak tanımlanabilir.”

Geleneksel yöntemler olarak kabul edilen, test sırasında sesli düşünme, başarılan görev sayısı, yapılan hata sayısı, test öncesinde ve/veya sonrasında yapılan memnuniyet anketlerinin değerlendirilmesi ile kullanılabilirlik ölçülebilmelidir.



Bilişim Ergonomisi

Uygunluğun değerlendirilmesi, sürecin akış şeması

Fac: Order# 99004234 New Report Selection OCI SSF View Dupe Load View Routing Print Call C Cancelled

0 99004234 99031927 From To Fax Site

Call Joe Quote 0 Mode From SC To SC Find CAXX 100670861 Charges: 761.50
 Ptn [Unknown Shipper] Air ADT ADT CAXX
 Terms Prepaid Collect 3rd Party STD Tariff Service 20 0194 CAXX
 Cust: Hi Fo Holdings, Ltd. HFO Ship Ref R/L
 Inv: Hi Fo Holdings, Ltd. HFO PO# GBL Nam Cons Plat Billing Plat
 At: Hi Fo Holdings, Ltd. HFO GBL Nam Cons Plat Billing Plat
 Add: 1125 STREET SUITE 1200 Deliver By 06-12-02 17:00 Clock Stop
 C/P: VANCOUVER BC V6Z2X8 C/P MasterID: MAWB
 Ph: [Redacted] Fax: [Redacted] P/O Mkt Del Mkt
 Cont: [Redacted] Est P/L 0 0 0
 Appointment: D 06-10-02 F T Stelname: Hold P/L
 C/P: CANADIAN HARDWARE & H Add: AVENUE SUITE 101 Broker / Customs Agent
 C/P: SCARBOROUGH ON M1B5M4 C/P Value: 0.00 USI
 Ph: [Redacted] x Fax: (416) [Redacted] Modified
 Cont: [Redacted] Verbal Pod
 Appointment: D F T Nullify on POD
 COD \$0.00 [Redacted] [Redacted] [Redacted]
 Fee \$0.00 [Redacted] [Redacted] [Redacted]

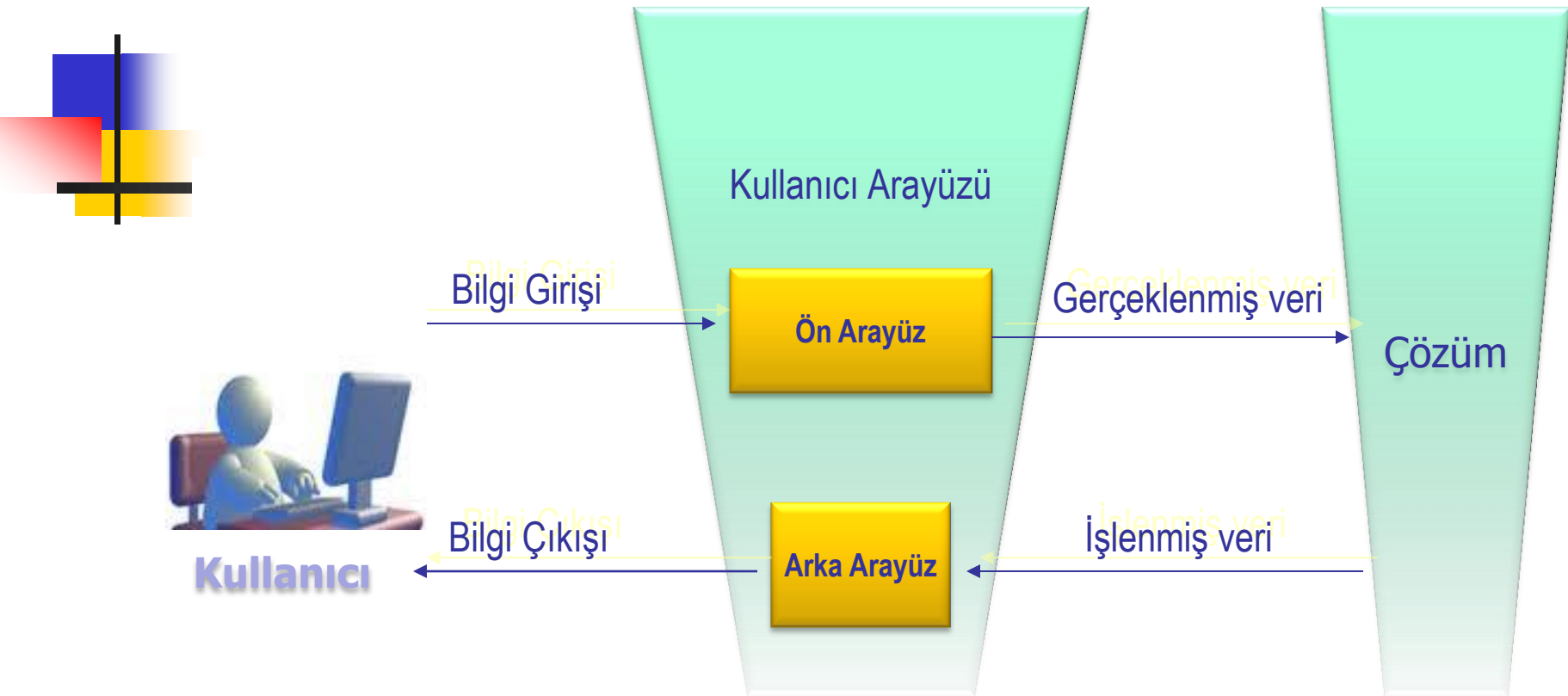
Units	Type	H Description	Stated
1	CRATE	CRATE	91
1	2MAN	2 MAN P&D	
2	CRATE	CRATE	500
3	Accel		

Accel \$40.00 DV 0 \$0.00 591

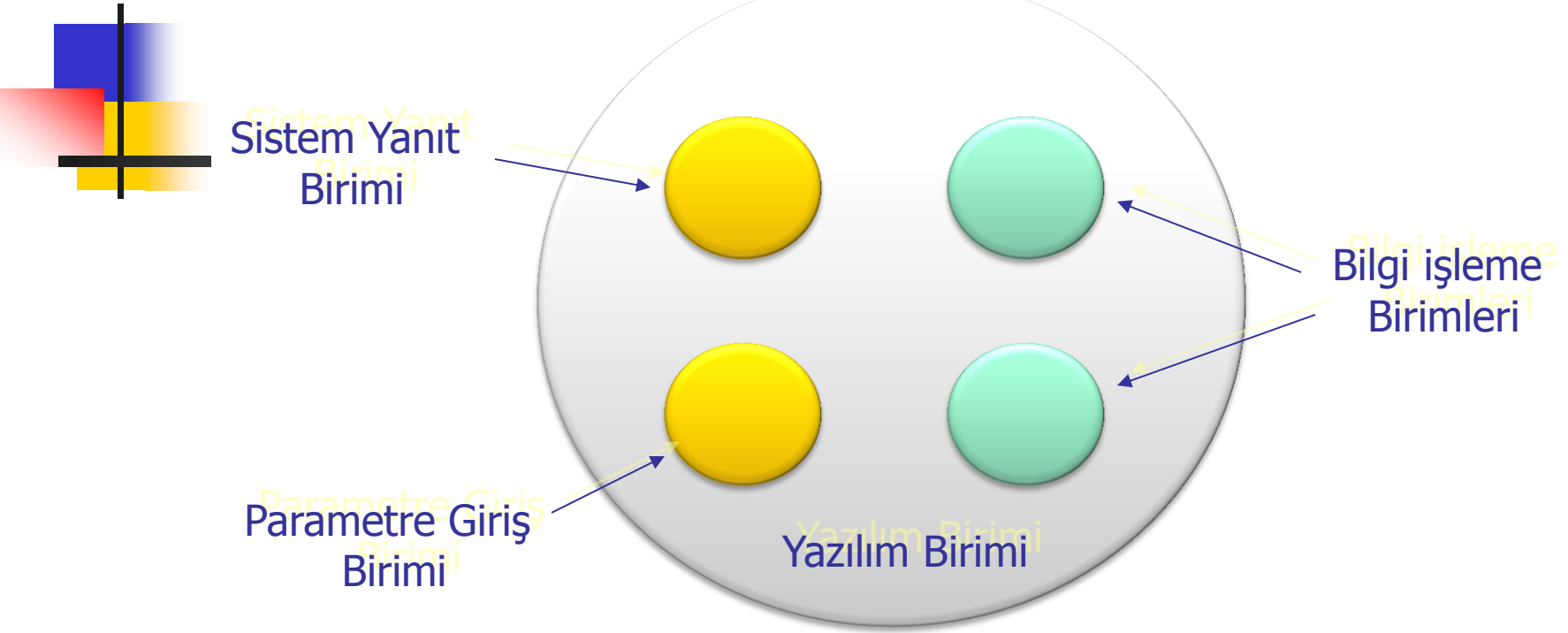
Araüz Tasarımı

"Kullanmak ne kadar kolay olursa o sistem yetenek ve işlevlerinden yarar sağlamak da o kadar artar."

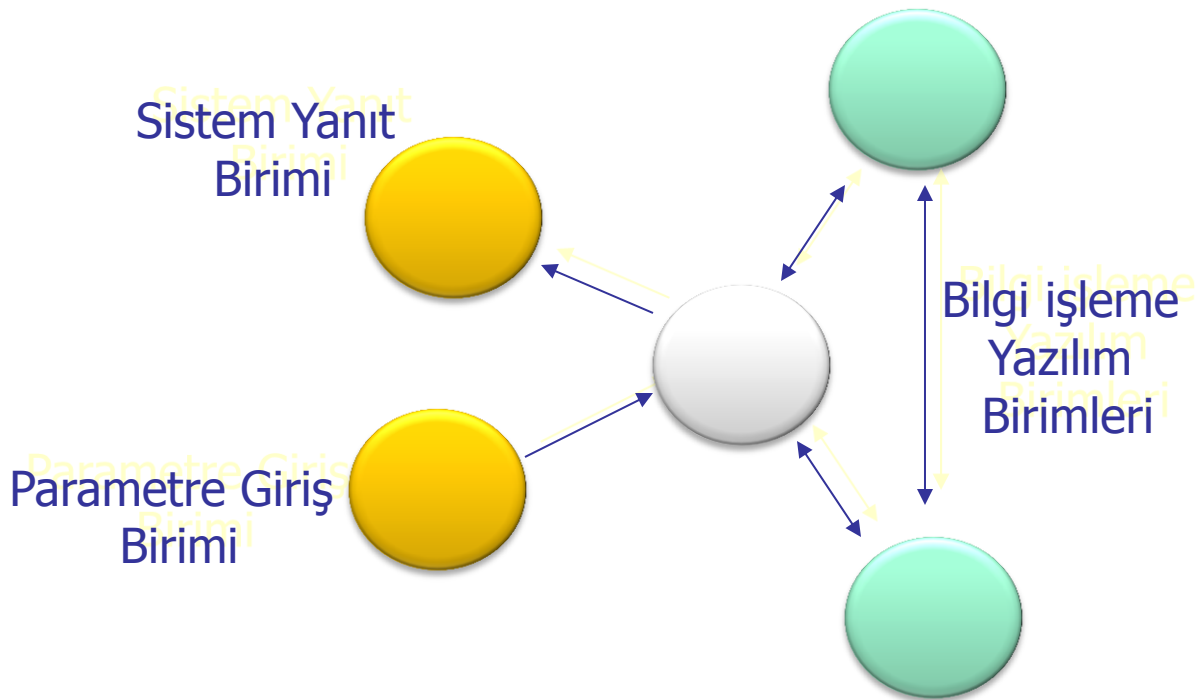




Bileşik Mimari

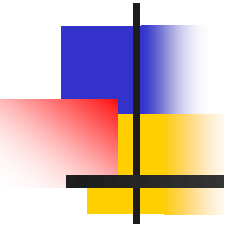


Ayrık Mimari



Yüksek Nitelik :

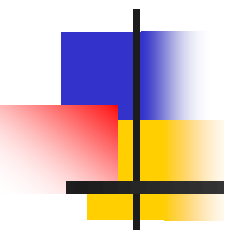
Temel Kurallar



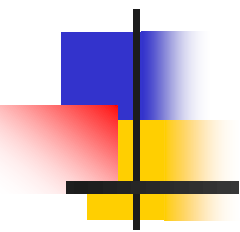
1. Anlaşılabilir Dil
2. Teknik terimler veya kısaltmalar
3. Mantıksal Sıralama
4. Hızlı bilgi girişi
5. Gerekli ve yeterli bilgi
6. Tekdüzelik
7. Fonksiyonellik (içinde bulunduğu durum çalışma kipi,yürütülmekte olan işlem, hata durumları vs.)
8. Çelişki oluşturacak etkinlik olmamalı
9. İşlem yapma ile ilgili klavuz bilgi verilebilmelidir
10. Önceki İşlem(arayüz) bilgisi verilebilmelidir

Yüksek Nitelik :

Temel Kurallar

- 
11. Etkinlikler doğrudan başlatılabilmelidir
 12. İşletilmekte olan etkinlik kolayca terkedilebilmelidir
 13. İşlemin doğruluğu/işlendiği kullanıcıya kısa sürede bildirilmelidir
 14. Yapılan iş ile ilgili süreç mesajları bildirilmelidir
 15. Az bir eğitimle öğrenim gerçekleştirilebilmelidir
 16. Gerekli olduğunda sesli ve görüntülü uyarıları destekleyebilmelidir
 17. Arayüz üzerinde kullanılan nesneler/rengler anlaşılabilir olmalıdır
 18. Sistem mesajları tarafsız olmalıdır
 19. İşlem adımları(menüler) yaygın standartlarda ve alışılmış görünümüyle olmalıdır

Kullanıcı dostu : Kullanıcı dostu alan arayüz verimlilik ve iletişim güvenilirliğini artırır.



Avantajları :

1. Klavuz/elkitabı olmadan kullanılabilir olmalıdır.
2. Gerekli detayları görebildiği için anımsamak zorunda kalmamalıdır
3. Alan terimleri uygun olmalıdır
4. Farklı bilgisayar birimleri bilgi girişi/çıkışı için yardımcı olabilmelidir.
5. Kullanıcı yazılım ve sistem araçlarını kullanırken pencere/menü seçeneklerini kullanabilmelidir.
6. Hızlı ve doğru giriş/çıkış yapabilmelidir
7. Giriş ve Çıkış için güvenlik/yetki sağlanmalıdır

Güvenilirlik : Sistem başarımını etkileyen en büyük etmenlerden biridir.

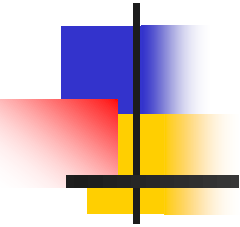
Dikkat edilmesi gerekenler :

1. Tüm girdiler görüntülenmeli, gerekli tip ve sınır kontrolleri yapabilmelidir.
2. Kilitlenme ve çökmelere karşı önlem alınmalıdır.
3. Çıkış ara yüzü tanımlanmış sınırlar içindeki değerler gösterilmelidir.
4. Çıkışlar beklenenleri karşılamalı
5. Uygulama alanının özelliğine göre , işlevi geri almak mümkün olabilmelidir.
6. Geri dönüşü olmayan işlemlerde , işlemler kullanıcıya doğrulatılmalıdır.(kritik yerlerde kullanılmayabilir)

Yardımlar : Arayüzün vazgeçilmez özelliklerinden biride etkileşim aşamasında kullanıcıya yardım verebilmesidir.

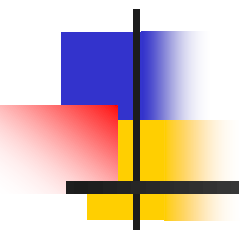
Dikkat edilmesi gerekenler :

1. Genel olabileceği gibi, o anki işleve bağlı yardım tasarlanabilir.
2. Yardıma ulaşmak tek tuşa basmak kadar kolay olacak şekilde tasarlanmalıdır.
3. Yardıma ulaşmak sistemin her yerinden aynı şekilde olmalı ve görüntülenmelidir.
4. Yardımdan çıkış giriş kadar kolay olmalı , askıya alma işlevi de tasarlanmalıdır.
5. Yardım metni basit bir düzyazı şeklinde olabileceği gibi konudan konuya geçme sağlayan yapıli metin şeklinde de tasarlanabilir.
6. Yardım penceresi içinde zenginleştirici eklemeler de yapılabilmelidir.
7. Yardım metni açık ve basit bir dille olmalı , sadece sitenenleri vermelidir.
8. Yardımlar aşamalı olarak planlanmalı , yönlendirici ve uygulatici bir yapıya sahip olmalıdır.



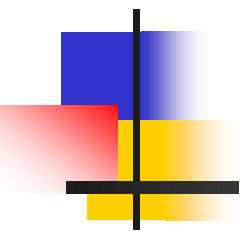
Hatalar ve Uyarılar : Kullanıcı arayüzünün bir özelliği de bilgi işleyici kısmı kullanıcı hatalarından korumak ve hata durumlarında kullanıcıyı uyarmaktır.

Dikkat edilmesi gerekenler :

- 
1. Eksik ve yanlış giriş işlemlerinde arayüz gerekli uyarıyı verebilmelidir.
 2. Görsel uyarılar gerçekten dikkat çekecek şekilde , ekranın ortasında ve başka bir iş yapılmasına izin vermeyecek şekilde yapılandırılmalıdır.
 3. Sesli uyarılarla desteklenmelidir.
 4. Sistem hatası ile ilgili açıklamalar açık ve sade bir dille verilmelidir.
 5. Hata sorunu açıklıkla ifade edilmeli , suçlayıcı ,yargılayıcı, itici ifadeler yer almamalıdır.

Yapısal Özellikler :

Bulunması gereken özellikler :

- 
1. Hata düzeltimi ve kullanıcı istekleri ile ilgili değişiklikler kolay ve hızlı yapılabilmelidir.
 2. Mümkünse bir kütüphane oluşturularak standart geliştirme yapılmalıdır.
 3. Estetik görünümün sağlanmalı , belirli bir yöntem geliştirilerek renk ve hizalama unsurlarına dikkat edilmelidir.
 4. Arayüzler seviyelendirilmelidir. (Uzmanlar,yeni başlayanlar gibi)
 5. Mümkünse arayüz kişiselleştirilebilmeli, özellikler sakalanabilmeli , kısayolların tanımları değiştirilebilir olması yarar sağlayıcı özelliklerdir.

Tüm sistem yazılımı belirli bir disiplinin uygulandığı geliştirme süreci ile gerçekleştirilir.

Kullanıcı arayüz yazılımı da ana sistem yazılımının temel öğelerinden biridir.

Kendi içinde bir süreç uygulanarak geliştirilir.

Süreç standart yazılım geliştirme süreçlerine benzer biçimde **dört** aşamadan oluşur.

1. **Çözümleme**
2. **Tasarım**
3. **Gerçekleştirim**
4. **Test**

Çözümleme :

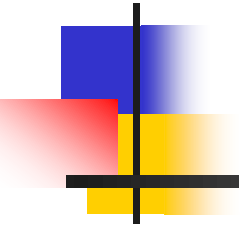
Bilgisayar tabanlı bir sistem genellikle elle yapılan işleri otomatik hale getirmek üzere tasarlanır ve geliştirilir.

Otomasyonun ne derece olacağı , işletmenin ne zaman devreye gireceği, sistemin girdi ve çıktıları sistem çözümlemesi sırasında belirlenir.

Çözümleyici bu amaçla sistemin amaçlarını , temel işlevlerini tanımlar ve çözümler.

Bu çözümleme sonunda sistem belirtiminin arayüz tanımlaması ortaya çıkar.

Arayüz şekillerinin karmaşıklığı aynı zamanda sistemin karmaşıklık derecesinide ortaya koyar.



“Arayüz şekillerinin karmaşıklığı aynı zamanda sistemin karmaşıklık derecesinide ortaya koyar.”

Tasarım :

Kullanıcıya ne tür arayüzler sağlayacağı, giriş ve çıkışların hangi aygıtlarla , ne şekilde yapılacağı , hata ve uyarı iletilerinin nasıl verileceği belirlenir.

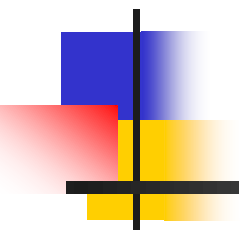
Özel donanım için geliştirme yapılacaksa donanım özelliklerine göre tasarım yapılmalıdır.

Standart donanım için yapılacak çalışmalar için ön çalışma yapıp , kullanıcı ile birlikte gözden geçirilmesi fayda sağlayacaktır.

Tüm görünümünün standart şablonlar üzerine oturtulması sağlanmalıdır.

Bilinen temel kullanım özellikleri tasarlanan tüm pencere, araç çubuğu gibi nesnelerde kullanılmalıdır.

Gerçekleştirim :



Çeşitli çizim araçlarıyla yapılabildiği gibi çeşitli geliştirme araçlarıylada yapılabilir.

Mümkünse kütüphane oluşturulmalıdır.

Arayüz geliştirme ile ilgili çıkan araçları öğrenip kullanmak bu aşamanın sürecinin ve güvenliğinin sağlanması için önemlidir.

Test :



Arayüz ile beraber yapılan işlevsel testler aslında tüm sistemin testi demektir.

Giriş ve çıkışların doğruluğu kullanıcı için bir kabul testi sayılır.

Sistemi çökertmeye yönelik girişimlerde bulunulması aslında sistemin sağlamlığının sınanması için çok önemlidir.

Sonuç :



Kullanılabilirlik değerlendirmeleri, bilhassa kullanıcı testleri pahalı ve zaman alıcı olduğundan tasarımcılar ve ürün geliştiriciler bu yöntemi kullanmaktan imtina etmektedirler.

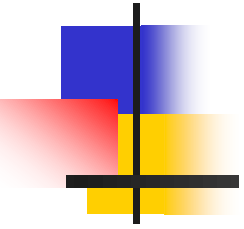
Bu durumda tüketici kuruluşları bünyesi içerisinde araştırma geliştirme merkezleri (AR-GE) kurularak değerlendirmelerin bu çatı altında yapılması oldukça faydalı olacaktır.

Sonuç :



Böylelikle hem tüketiciye ürün seçimi konusunda yardımcı olunmuş hem de üreticilere sağlıklı bir geri besleme imkanı sağlanmış olacaktır.

Bu şekilde tüketicilerin yanı sıra üreticilerde de kullanılabilirlik bilincinin oluşturulması ve geliştirilmesi ve üreticilerin daha kullanılabilir ürünleri üretmesi yönünde teşvik edilmesi sağlanmış olacaktır.



... Örnek Çalışma

CRM



CUSTOMER RELATIONSHIP
MANAGEMENT

*Müşteri İlişkileri
Yönetimi*

CRM Nedir

- Müşteri İlişkisi?



- Müşteri olma potansiyeli olan (prospect) / pazarlama tarafından müşteri olarak kazanılmaya çalışan
- Müşteri olup alışveriş yapan / satış yapılan
- Ürün ve/veya hizmet satın almış olup bununla ilgili satış sonrasında müşteri hizmetlerinin ilgi alanında olantüm kişilerle her temas anında kurulan etkileşimlerin tümünden oluşan ilişki

CRM Nedir

- **CRM**

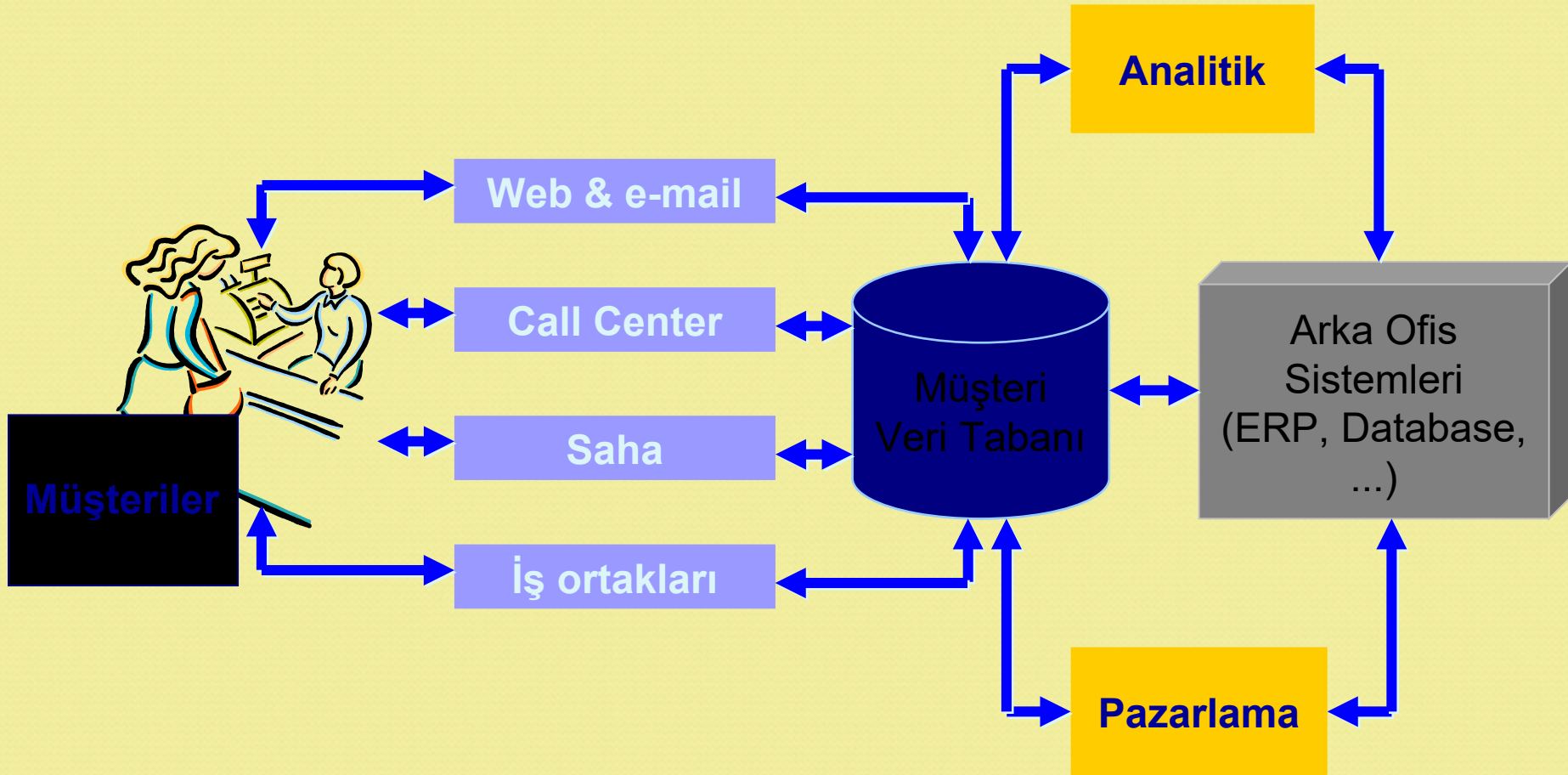
- En değerli “iş ilişkilerini” seçmeye ve yönetmeye yönelik **iş stratejisidir.**
- Etkin pazarlama, satış ve hizmet süreçlerini destekleyecek **müşteri odaklı bir iş felsefesi** ve **kültürü** ile mümkündür.
- **Teknoloji**, kurumda doğru bir liderlik, strateji ve kültür bulunması koşuluyla etkin bir müşteri ilişkileri yönetiminin (CRM) yürütülmesini sağlar.

Müşteri?

- Tüketici
- Kurum
- Bayi / satıcı
- İş ortakları: PRM

CRM Nedir

CRM Süreçleri?



CRM'in hedefi

- Müşterilerin tam istediği ürün ve hizmetleri sağlamak
- Müşteriye daha iyi hizmet sunmak
- Daha efektif çapraz satış
- Satış ekibinin daha hızlı satış kapatması
- Eski ve değerli müşterileri tutmak ve yenilerini kazanmak

CRM Yapı Taşları

- Müşteri
- Kanallar
- Müşteri Veri Tabanı
- Kurumsal Strateji
- Süreçler
- Araçlar

CRM Süreçleri

- Veri tabanının güncellenmesi
- Satış / Satış gücü otomasyonu
- Veri Analizi
- Pazarlama otomasyonu
 - Veri tabanına dayalı pazarlama
 - Kampanya Yönetimi
- Alternatif Kanallar
 - Call Center
 - Web
 - e-posta / sms

CRM Kime Gerekli?

- Herkes uyguluyor
- Doğru soru kim CRM projesi yapmalı
- CRM Projelerinin başarı oranı: herkes CRM projesi yapmalı mı?

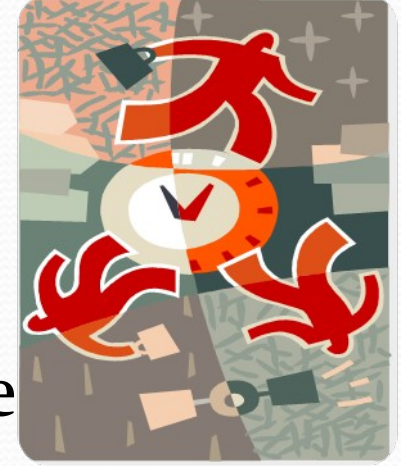
“Müşterinizin kim olduğu, ihtiyaç ve beklentileri ile ilgili doğru bir bakışı yakalayamıyorsanız CRM uygulamalısınız. Rekabete müşteri kaptırıyorsanız müşterilerinizi daha iyi anlamanız gerekir.”

CRM Kime Gerekli?

- Müşteri sayınız kaçtır?
- Ürünlerinizin tüketicisi kimdir?
- Rakipleriniz kimdir?
- Hangi kanalları kullanıyorsunuz?
- Hangi kanalları kullanabilirsiniz?
- Global pazarda yer alma hedefiniz?
- İşiniz için Interneti nasıl kullanabilirsiniz?
- Satış süreciniz ne kadar sürüyor?
- Fırsatların ne kadarı satışa dönüşüyor?
- Pazarlamaya ne kadar harcıyorsunuz?
- Müşterinizin kaçış oranı nedir?
- Müşterinizle doğrudan iletişiminiz var mı?

Müşterileriniz

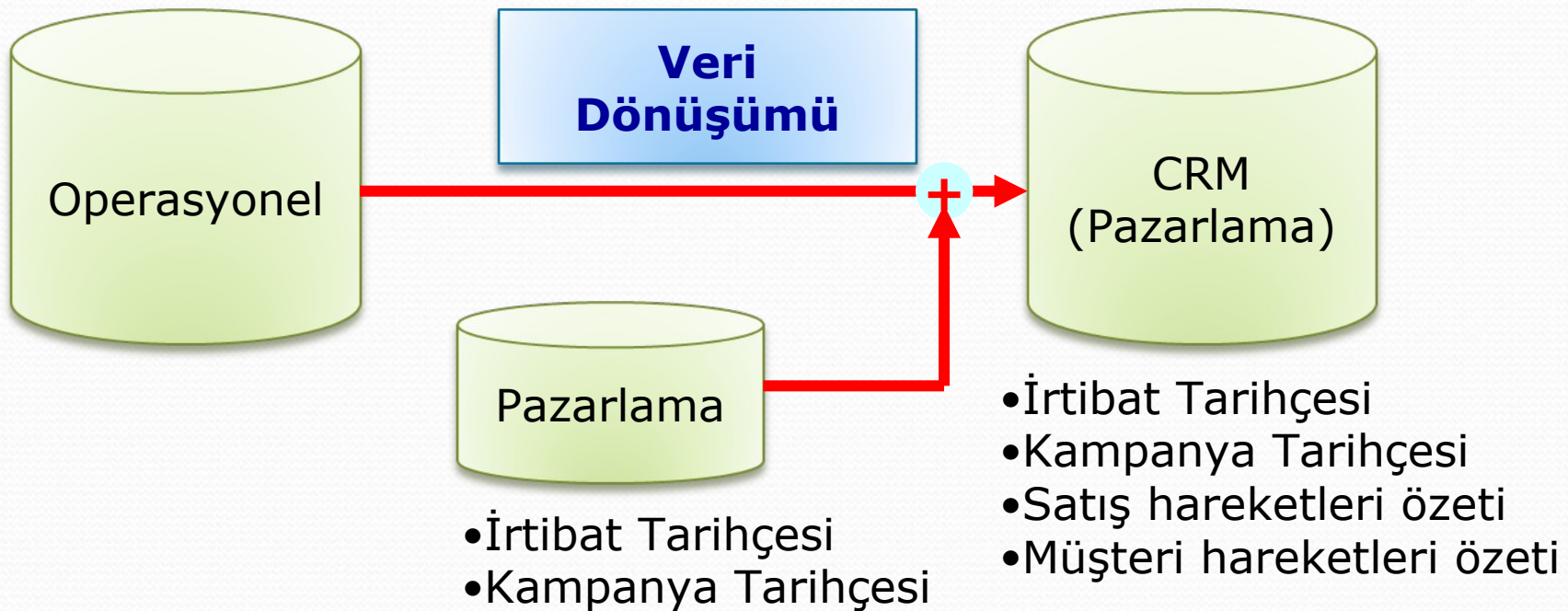
- Sayıca çok!
- Müşteri değeri geniş bir yelpazeye yayılmış
- Hizmet seviyesindeki beklentileri farklı
- Çok kanallı ilişkiler içeren
- Müşteri yaşam döngüsünün farklı noktalarında



Müşteri Veritabanı



Müşteri Veri Tabanları



Operasyonel Müşteri Veri Tabanı

- Müşteri İrtibat Bilgileri
- Satış hareketleri
(ürün + finansal)
- Ödeme bilgileri
- Müşteri cari bilgileri
(Borç, alacak durumu)



Pazarlamaya yönelik Müşteri Verisi

- Ailesel
- Demografik
- Tercihler
- Satınalma alışkanlıkları
- Finansal durum
- Karakter



CRM Verisi

- Operasyonel veri (bir kısmı)
- Pazarlama verisi
- İletişim Tarihçesi
- Kampanya Tarihçesi
- Satış döngüleri tarihçesi
- Analitik veri (skor, segment)
 - Örnek: Satınalma Eğilimi
 - Örnek: yeni ürün hedef kitlesi

CRM Verisi: Özellikler

- Verinin eksiksiz olması
- Verinin geçerli / güncel olması
- Bazı özel verilerin “gerçek zamanlı” olması
- Verinin “temiz” olması
- Verinin duplike olmaması
- Müşterilerin tek (unique) olarak saptanabilmesi
- Müşteriler arası ilişkilendirme
- Müşteri üzeri ilişkilendirme
- Verinin olabildiğince “kategorize” olması

Müşteri Veri Modeli

Workshop

- Veri Modeli Çalışması
- Örnek veri modeli
- Veri özellikleri
- Örnek verinin temizlenmesi

Olay ile tetiklenen iletişim ve veri tabanı

Örnek Olay: Doğum günü

- Uygun hediye: Kadın kıyafetleri
 - İş takımı
 - Spor kıyafet
- Karar Algoritması
 - Kalite seçici
 - Fiyat seçici
 - Moda seçici
- Demografi
 - Cinsiyet, Doğum tarihi, eğitim
 - Gelir, Finansal durum
 - Çocuklar
- Önemli tarihler
 - Doğum günleri
 - Yıldönümleri
 - Mezuniyetler
 - Planlı seyahatler

Olay ile tetiklenen iletişim ve veri tabanı

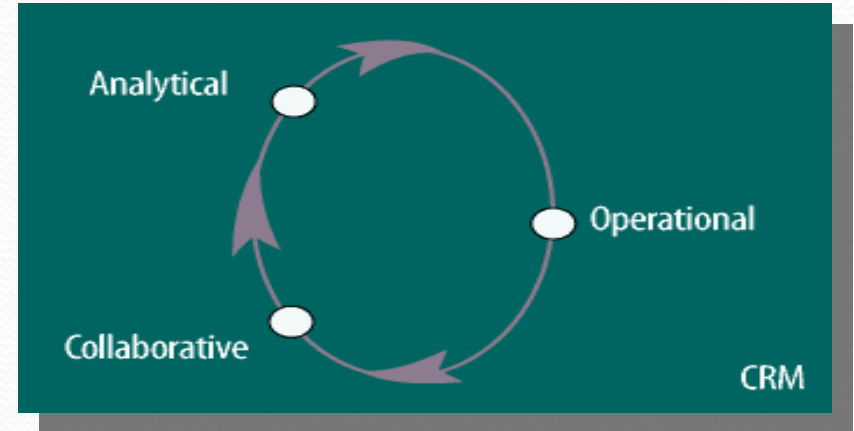
- Kişisel özellikler
 - Beden, ayakkabı no
 - Renk tercihleri
 - Tarz ve üreticinin tercihleri
 - Kişisel renklendirme
 - Ölçüler
 - Yüz ve vücut şekli
- Sosyografi
 - Alışveriş rolü
 - Meslek
 - Medeni durum
- Alışveriş Tarihçesi
 - Alışveriş davranışı
 - Toplam harcama /departman bazında
 - Mağaza / Marka tercihi
 - Hobi, ilgi, aktivite
 - Yaşam tarzı

"Veri tabanı periyodik olarak taranır ve bu tarihte "özel olay" denk gelen Müşterilere iletişim için hedef müşteriler tespit edilir."

CRM Kavramları

- CRM yaklaşımları
- Yapı taşları
- Süreçler
- Aşamalara göre CRM
- Teknolojiler

CRM Yaklaşımları

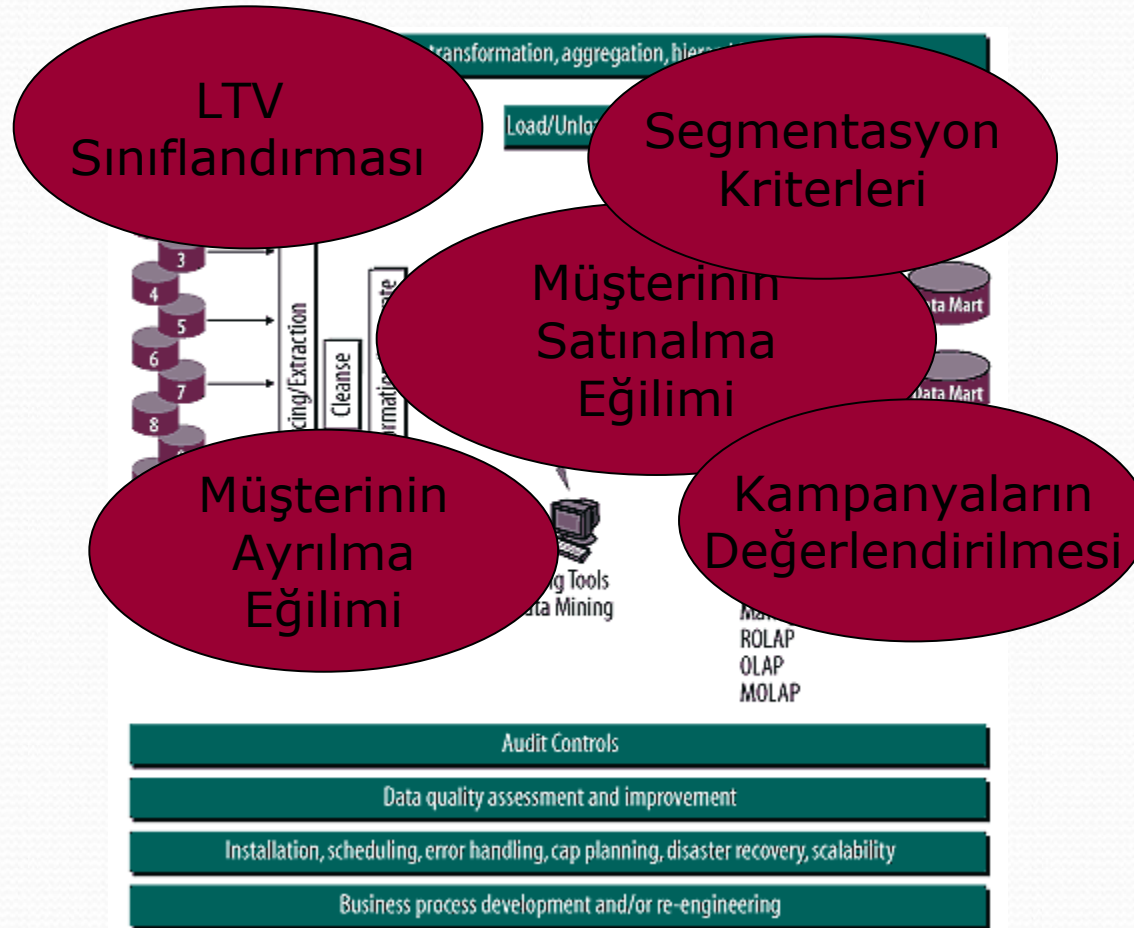


- Analitik
 - Veri analizi yoluyla müşteri tanıma
- Operasyonel
 - Müşteriye dönük süreçlerin otomasyonu
- İşbirlikçi

Operasyonel CRM

- Satış Gücü Otomasyonu
 - Satış sürecinin planlanması
 - Satış projelerinin takibi ve satış ekibinin performans değerlendirmesi
- Müşteri Hizmetleri
 - Satış sonrası hizmetler
- Pazarlama Otomasyonu
 - Kampanya yönetimi

Analitik CRM



İşbirlikçi CRM

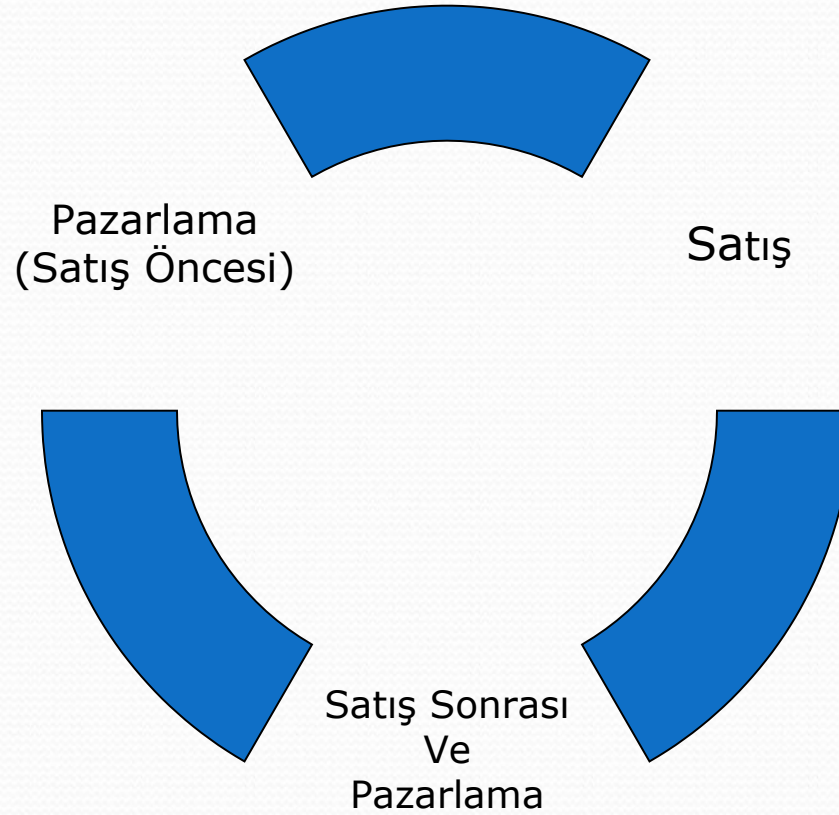
Tam olarak oturmamış bir kavram

- Kurum ile kanallar arasındaki ilişkinin kurulması – kanal yönetimi
- Müşteriye tüm kanallarda aynı görüntüyü sunabilecek altyapı ve hizmetler
- Kurumun çeşitli birimlerinin müşteri etkileşimlerinden toplanan bilgiyi paylaşımı
- Amaç: Müşteriye sunulan hizmet kalitesini arttırarak müşteri memnuniyeti ve bağlılığını arttırmak

CRM Yapı Taşları

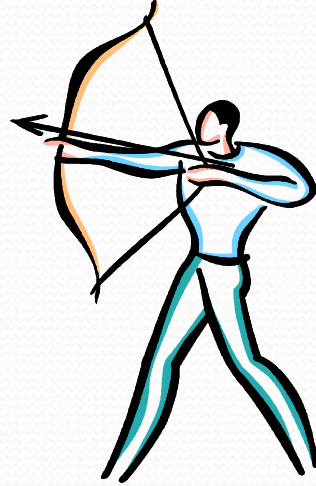


Müşteri İlişkileri Yaşam Döngüsü



Pazarlama Otomasyonu

- Kitlesel Pazarlama



- Veri Tabanına dayalı pazarlama

Pazarlama otomasyonu = CRM odaklı pazarlama süreçlerinin otomasyonu

Pazarlama Otomasyonu

- Pazarlama Veri Tabanı
- Pazarlama Analizi
- Kanal Yönetimi
- Segmentasyon
- Kampanya Yönetimi
- Tele Pazarlama
- Sadakat Programları

Kanallar

Klasik kanallar

- Satış Noktası / Mağaza
- Satış personeli (Saha satış)
- Arıza / Bakım Servisi
- Kitlel Medya
 - TV, Radyo
 - Gazete, Dergi
 - Broşür, doğrudan posta
- Stand, tanıtım noktası, showroom
- Saha anket
- Bayi ağı / partner ağı

Kanallar

CRM ve kanallar

- Call Center, Contact Center
- Help Desk
- Internet (Web) Cep Telefonu (WAP)
- Faks
- E-mail
- SMS
- Kiosklar
- ...
- Sorun: Kanal Yönetimi
 - Entegrasyon
 - Tek görüntü (Single view of customer)
 - Senkronizasyon

Satış Otomasyonu

- Satış Döngüsünün Yönetimi
- Müşteri ve potansiyaller
- Müşteri Temsilcileri (İlişki Yöneticileri)
- Satış aktivite planlama ve takvim
- Müşteri Satış Veritabanı
- Fırsatlar, Fırsat yönetimi
- Satış sürecinin aşamaları
- Satış aktiviteleri tarihçesi
- Rakipler, rakip analizleri
- Kayıp / Kazanç Analizleri
- MT Hedefleri ve Performansları
- Tele-Satış

Satış Sonrası Müşteri İlişkileri

- Müşteri Hizmetleri Yönetimi
- Servis ve Bakım hizmetleri
- Destek hizmetleri
- Tamamlayıcı Hizmetler
- Bilgilendirme
- Şikâyet Yönetimi
- Müşteri Memnuniyeti

Satış Sonrası Müşteri İlişkileri

- Örnekler
 - Yol Yardımı
 - Satış sonrası memnuniyet aramaları
 - Memnuniyet anketleri
 - Peryodik bakıma davet
 - Garanti kapsamındaki hizmetler
 - Help Desk, Web üzerinde knowledge base, elektronik el kitapları vb
 - Diğer müşteriler ile paylaşım ortamları
- Örnek: Toplu Konut Firması (Sweethome)

CRM ve Teknoloji



Teknoloji Nereelerde?

- Müşteri Veri Tabanı
- Analiz Araçları
- Satış / Kontak yönetimi
- Kanallar:
 - Call Center, Web, email/sms
- Kanal Entegrasyonu
- 360 derece Müşteri görüntüsü
- Kampanya Yönetimi

Başarılı CRM projeleri için...

- Teknolojiden önce müşteri odaklı stratejinizi oluşturun.
- CRM projesini yönetilebilir fazlara bölün.
- Küçük ama her departmanı içine alan bir pilot ile başlayın
- Mimariyi ve ölçeklenebilirliği planlayın
- Ne kadar veri oluşacağını hesaba katın
- Hangi veriyi toplamamanız gerektiğini iyi değerlendirin. Herşeyi biriktirmeyin!

CRM Uygulaması

Müşteri Bağlılığı

Müşteri Ne İster?

- Tanınmak
- Hizmet
- Kolaylık
- Yardımseverlik
- Bilgi
- İlişkilendirme

Müşteri Bağlılığı

- Churning: Müşterinin kaçması
 - Örnek: Telko.
- Retention:
 - Müşteriyi tutma oranı

CRM Teknikleri

- Çapraz Satış
- Yukarı Satış

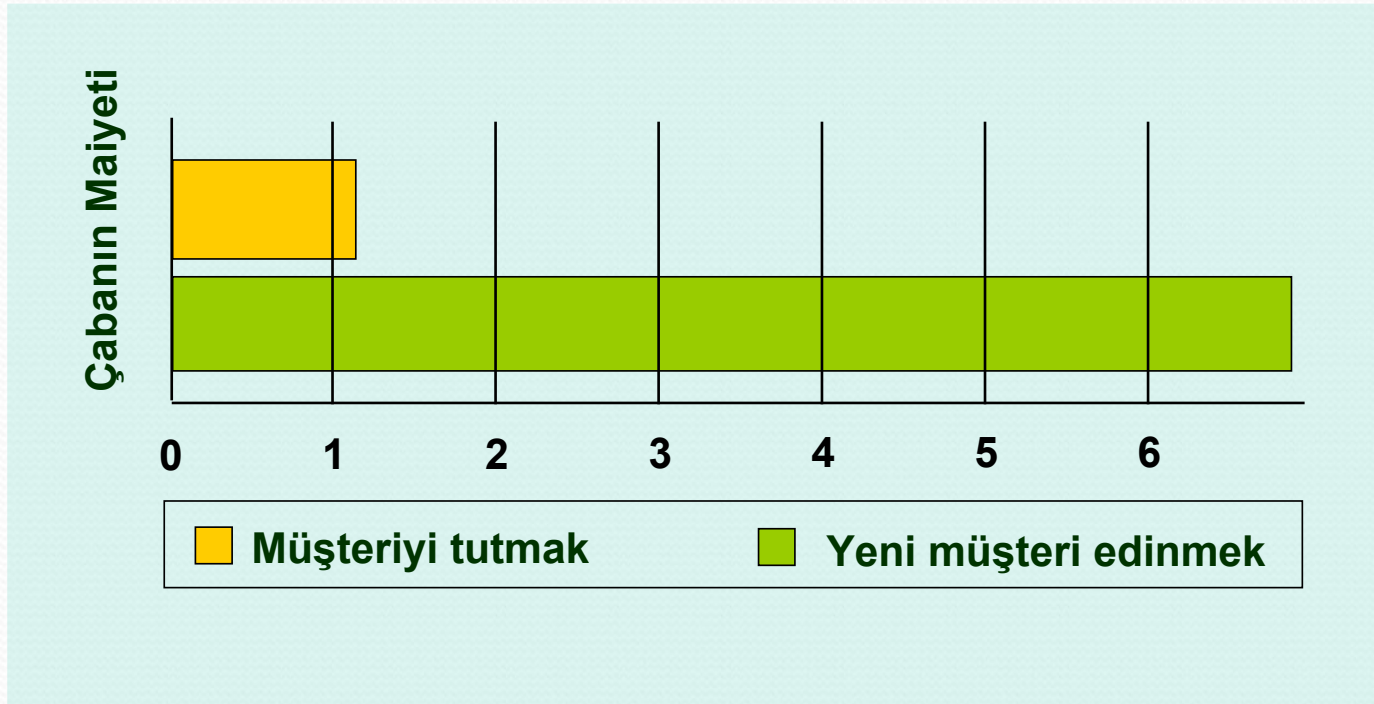
Müşteriler neden kaçar?

- 1% ölür
- 3% taşınır
- 5% yeni arkadaşlıklar kurar
- 9% rekabetçi sebeplerle gider
- 14% memnun kalmadığı için gider
- 68% kendisine karşı kayıtsız bir tavır hissettiği için gider

Genel İstatistikler

- Şikâyetini bildiren müşterilerden %70'i eğer sorunu efektif bir şekilde çözümlenirse müşteriniz olarak kalır.
 - Bu oran, çözüm hızla sağlanırsa %95'e çıkar!
- Şikâyetlerin %40'ı müşteri hatalarından ya da yersiz beklentilerden kaynaklanır
- Başarılı bir şekilde çözümlenen / yönetilen bir şikâyet, size şikâyet bildirilmemesinden çok daha iyidir
 - Şikâyet bildirip sonuçtan tatmin olan müşterilerin sadakat oranı hiç şikâyet etmeyenlere göre %8 daha fazladır.

Sadık Müşterinin Değeri



Dikkat: dahil değildir!!...

Mevcut müşteriler hakkındaki bilginin değeri
(ör. Pazarlama tarihçesi, çapraz satış ve yukarı satış olanakları)

Sadakat Programları

- Sadakat Kartları
 - Ödüllendirme
 - → Sadakati arttırma
 - Retention oranını arttırma
 - Veri toplama
 - Müşteriler arası ilişkilendirme
 - Satınalma trendlerinin analizi
- Örnekler
 - Çarşı anahtar
 - Shell smartcard
 - Migros kart
 - Banka kartı

Yatırımın Geri Dönüşü

- Yeni Müşteri: Hesaplamak Kolay
- Eski Müşteri: Kontrol Grubu kurmak gerekir.
- LTV ile hesaplamak
- Tarihçe karşılaştırması ile hesaplamak

Kampanya Yönetimi

- Doğru müşteriye doğru kanaldan doğru zamanda doğru mesajı vermek!
- CRM Veri Tabanı
- Analiz
- Segmentasyon
- Hangi Mesaj, Hangi şekilde hangi kanaldan kime?
- Ne zaman?
- Sonraki adımlar ne olacak?

e-posta kanalının kullanımı

e-posta kanalı özellikleri

- “Bir kanal daha”, ama önemli özellikler var:
 - İletişim başına maliyet açısından son derece efektif
 - Daha küçük segmentlere göre ayrı içerik dağıtılabilir
 - Bu kanal üzerinden yeni kampanyaların planlanması tasarlanması ve uygulanması çok daha hızlı
 - Yanıtların takibi ve ölçümü kolay
 - Müşteri dönüşü hızlı
 - Pazarlama ortamı olarak daha az rahatsız edici olabilir
 - SPAM uyarısı! Spam listelerine düşmeyin

Önemli!

- Hedefleme & alaka
- Kişiselleştirme
- İsteyene / izin verene gitmesi
- E-postalar, açık ve kolay “istemiyorum” linki içermeli
- Müsaade isteyen dil / yaklaşım
- Kişilere giden ileti hacim ve yoğunluğu dikkatli belirlenmeli
- e-posta adreslerinin toplanması, validasyonu

e-posta Pazarlama Özeti

HIZLI + UCUZ

≠ KOLAY

iyi çalışmalar ...

SWOT ANALİZİ

Nedir?
Nasıl Çalışılır?
Nasıl Kullanılır?

Tanımı

“Bir organizasyonu veya sistemi incelerken, veya bunlarla ilgili politika oluşturmak için kullanılan analitik bir yöntemdir.”

Anlamı

Strengths

Weaknesses

Opportunities

Threats

Güçlü yönler,

Zayıf yönler,

Fırsatlar,

Tehditler.

Nerede Kullanılır

- Planlama yaparken,
- Sorun tanımlamada ve çözümlemede,
- Strateji oluştururken,
- Analitik kararlarda.

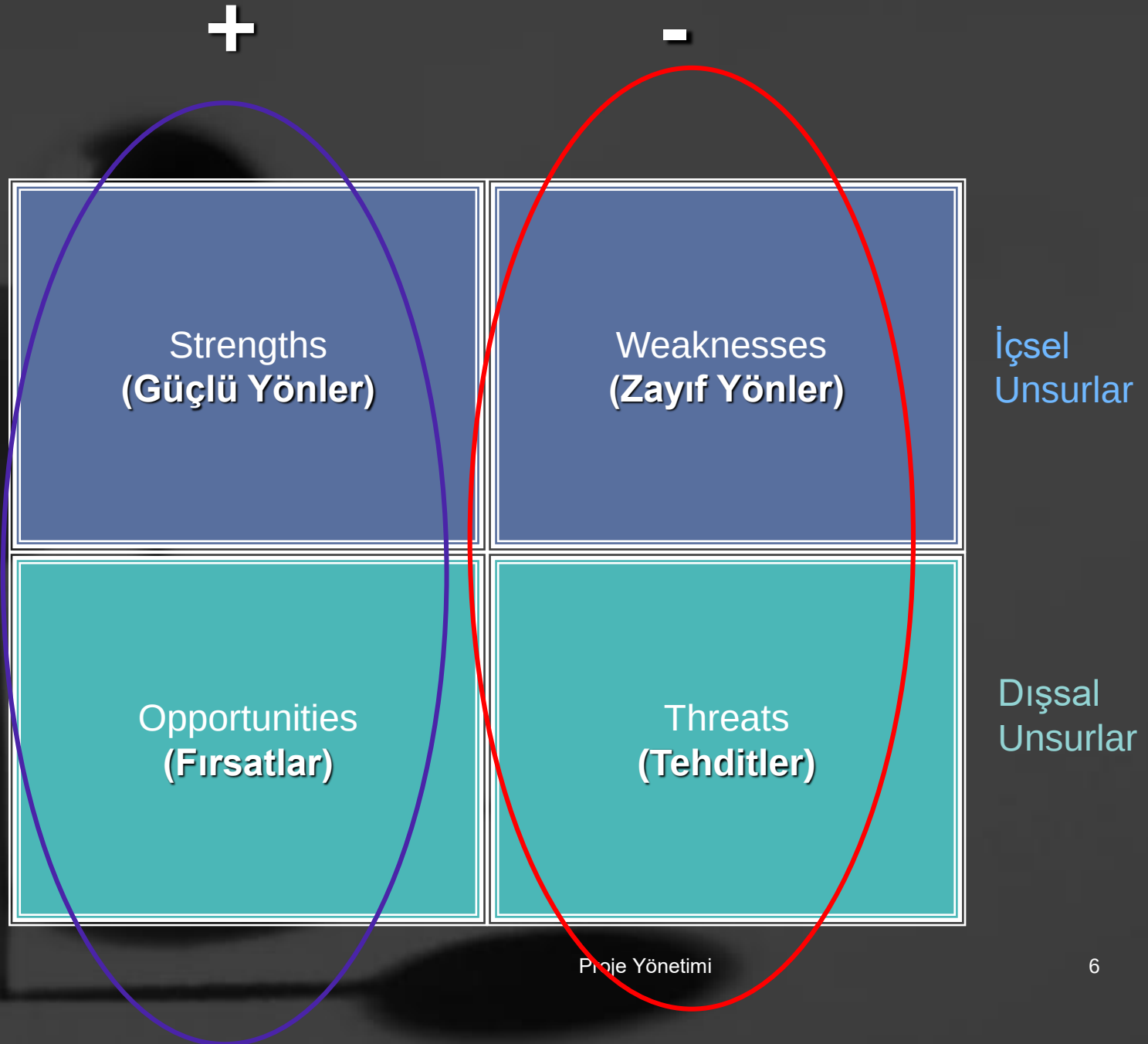
Bir yönetim aracıdır.

SWOT

**Mevcut Durum
Analizi**
(Fotoğraf Çekme)

**Bir Fikrin, Projenin
Değerlendirilmesi**
(Karar Verme)

Şekli



İçsel Unsurlar

- Teknik, mali, bilgi gibi faktörler açısından kontrol edilebilen unsurlardır.
- Bu nedenle müdahale mümkün alanlardır.

“Operasyon alanlarıdır.”

İçsel Unsurlar

Güçlü Yönler



- Ön plana çıkartılacak
- Korunacak

Zayıf Yönler



- Giderilecek
- Tedbir alınacak

- ✓ Organizasyon Yapısı ve Yönetmel kapasite
- ✓ Programlama kapasitesi
- ✓ Finansal kapasite

Dışsal Unsurlar

- Sosyolojik, ekonomik, demografik, iklimsel, ticari ve benzeri durumları içeren unsurlardır.
- Bu nedenle incelenen örgütün, sistemin, projenin dışında kontrol edilemeyen unsurlardır.

“İzleme, dikkate alma ve karar verme alanlarıdır.”

Dışsal Unsurlar

Fırsatlar



- Planların yönünü belirler
- Altyapısını oluşturur,
- Stratejileri güçlendirir.

Tehditler



- Kaçınma noktalarıdır,
- Kaçınılamayacak tehditler “varsayım” olarak kalacaktır.

Güçlü yönler

- Neleri iyi yapıyoruz?
- Avantajlarımız neler?
- Hangi kaynaklarımız var?
- Farklılıklarımız neler?
- Dışarıdan güçlü gözüken yönlerimiz?

Zayıf yönler

- Neler iyi gitmiyor?
- Neleri düzeltmemiz gerekiyor?
- Nelerin geliştirilmeye ihtiyacı var?
- Başkaları hangi konularda bizden daha iyiler?
- Dışarıdan zayıf gözüken yönlerimiz?

Fırsatlar

- Mevcut fırsatlar neler?
- Çevremizdeki mevcut gelişmeler neler?
 - Ekonomik
 - Teknolojik
 - Politik
 - Sosyokültürel
 - Çevresel
 - Hukuki

Tehditler

- Mevcut engeller neler?
- Potansiyel engeller neler olabilir?
- Rakiplerde tehdit edici gelişmeler var mı?
- Hedef kitlenin beklentilerinde değişiklikler var mı?
- Bütçe, kaynak problemleri var mı?
- Tehdit edici gelişmeler var mı?
 - Ekonomik
 - Teknolojik
 - Politik
 - Sosyokültürel
 - Çevresel
 - Hukuki

Örnek : Galatasaray

- Geniş ve yetenekli bir kadro
- Açık ve net stratejik hedefler
- Geçmişteki büyük başarılarla dayalı markalaşma
- Büyük bir taraftar kitlesine sahip olması
- Parasal problemlerin çözümlenebilmesi becerisinin yüksek olması
- Ali Sami Yen'in rakip takımlar üzerinde baskı yaratan bir ambiansa sahip olması
- Köklü bir kültür ve geçmişe sahip olması
- Güçlü ve kararlı bir yönetim
- Altyapı tesislerinin tamamlanmış olması

- Fikstür avantajı
- Yeni lig şampiyonluğu ile ezeli rakiplerle farkın açılacak, ciddi bir maddi gelir sağlayacak
- Şampiyonlar Ligi'ndeki olası başarılar, Dünya takımı olma sürecini hızlandıracak

- Mali sorunlarının olması.
- Avrupa kulüplerine göre yetersiz bütçe
- Yapılan transferlerde uyum sorunu
- Takım içi uyum ve koordinasyonun tam sağlanamamış olması
- Sportif başarıları ekonomik sonuca dönüştürememesi
- Takımda sevk ve idareyi sağlayacak lider bir oyuncu olmaması
- Defansta yer alan futbolcuların çok basit hatalar yapması
- Terim'in kafasındaki teknik ve taktik yapıya uygun yeterli sayıda ve kapasitede futbolcunun bulunmaması
- Ali Sami Yen'in, yetersiz olması ve ekonomik-teknolojik ömrünü tamamlaması

- Yeni transferlerin takım içinde bir huzursuzluğa neden olabilecek olması
- Mali problemlerin takımın moral ve motivasyonu olumsuz etkileyebilecek olması
- Ali Sami Yen'in yıkılması sürecinde, maçların Olimpiyat stadında oynanacak olmasının, yaratacağı olumsuz etkiler
- Fatih Terim ve yönetimin, taraftar nezdinde kredilerinin giderek azalması.

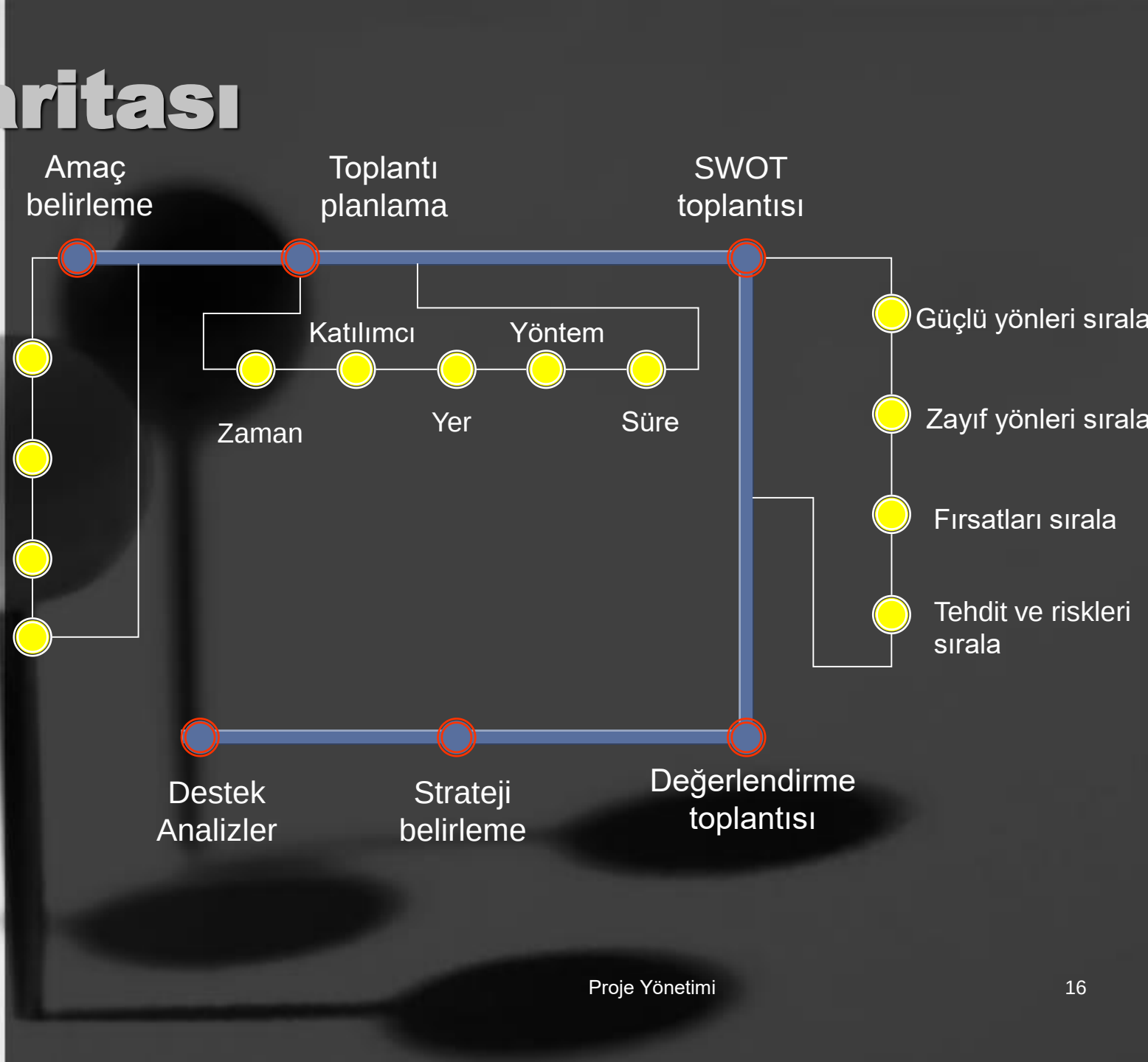
Yol har

Neden?

Nasıl?

Ne zaman?

Nerede?



Yöntem

- 1 Birlikte çalışmak.**
- 2 Gruplar halinde çalışmak.**
 - a) Her grup tümünü çalışır.
 - b) Her grup bir bölümünü çalışır.
- 3 Tablolar hazırlamak.**

Yöntem

(Örnek)

	DURUM	DURUM		GÜÇ			ZAYIFLIK			TEHDİT			FIRSAT		
		İÇSEL	DIŞSAL	-1	0	1	-1	0	1	-1	0	1	-1	0	1
1.															
2.															
3.															
4.															
5.															
6.															
7.															
8.															
9.															
10.															
11.															
12.															
13.															
14.															
15.															
16.															
17.															

- Güçlü ve zayıf yanlar için gerçekçi olmak
- Mevcut durum ile olması istenen durumu ayırt etmek
- Gri alanlardan kaçınmak
- Rekabet edebilirliği ön planda tutmak
- Karmaşıklıktan kaçınmak
- Çözümler için analiz sonrasını beklemek

Bartol, K. M., & Martin, D. C. *Management*. New York: McGraw Hill, Inc. (1991).

Broadhead, C. W. Image 2000: A vision for vocational education. To look good, we've got to be good. *Vocational Education Journal*, 66(1), 22-25. (1991).

Crispell, D. Workers in 2000. *American Demographics*, 12(3), 36-40. (1990).

Swortzel, Kirk .Journal of Vocational and Technical Education Volume 12, Number 1 Fall, 1995

<http://erc.msh.org/quality/map.cfm>

http://www.axi.ca/TCA/Sep2003/facilitationrole_1.shtml

<http://www.biltr.com/DVD/MEY/swot/SWOT%20ANALIZI.pdf>

<http://www.geocities.com/druryclass/presentation/>

<http://www.kobitek.com/makale.php?id=83>

<http://www.marketingteacher.com/>

http://www.mindtools.com/pages/article/newTMC_05.htm

<http://www.promise.org.uk/swot.htm>

<http://www.quickmba.com/strategy/swot/>